



Uninformed Search

SE-350/SE-450, Spring-2026
Artificial Intelligence (AI)

Dr. Kanwal Yousaf

Department of Software Engineering
UET Taxila

e-mail: kanwal.yousaf@uettaxila.edu.pk

Some slides material from
Chen-Yu Wei, & Emma Pierson, University of California, Berkeley



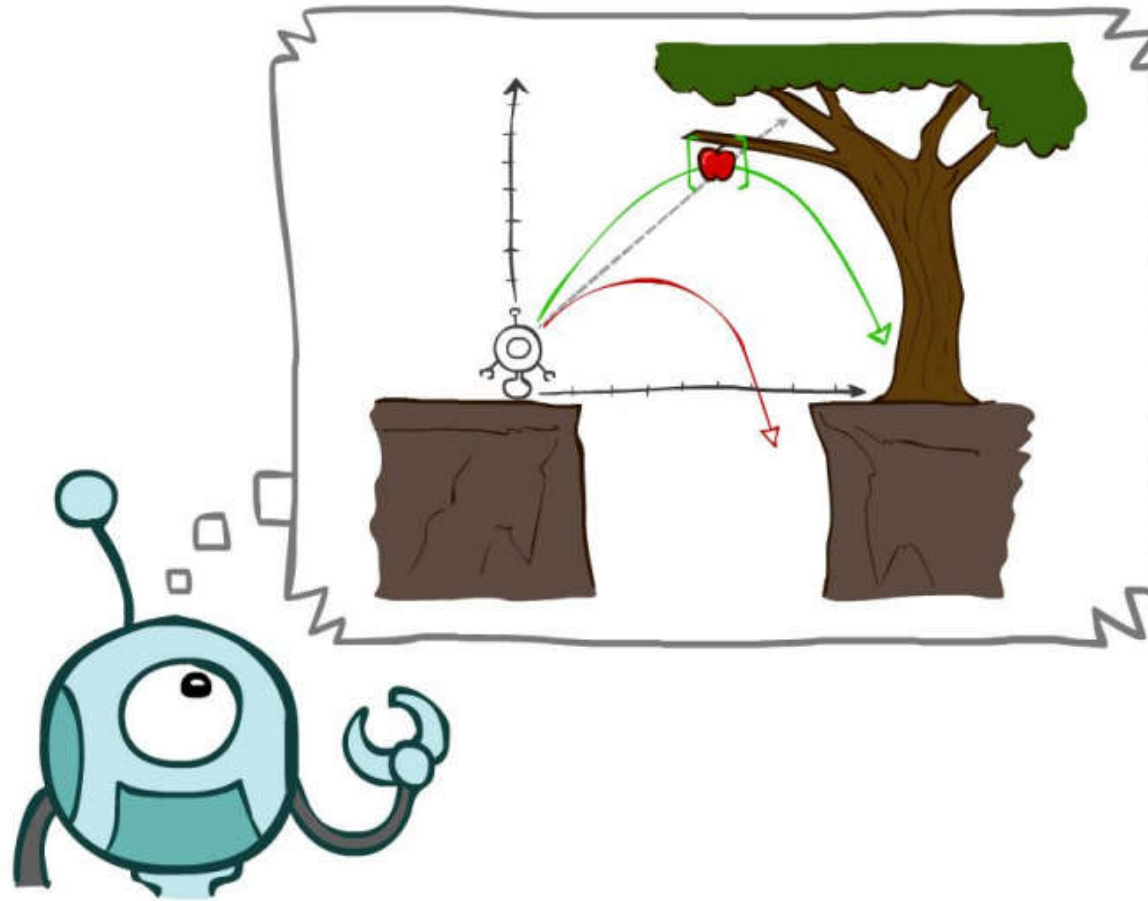
Group to join for updates

- All course material and important information will be shared in the MS Teams group below:

SE-350/SE-450 : Artificial Intelligence (Session-2023/2022)

rbuy693

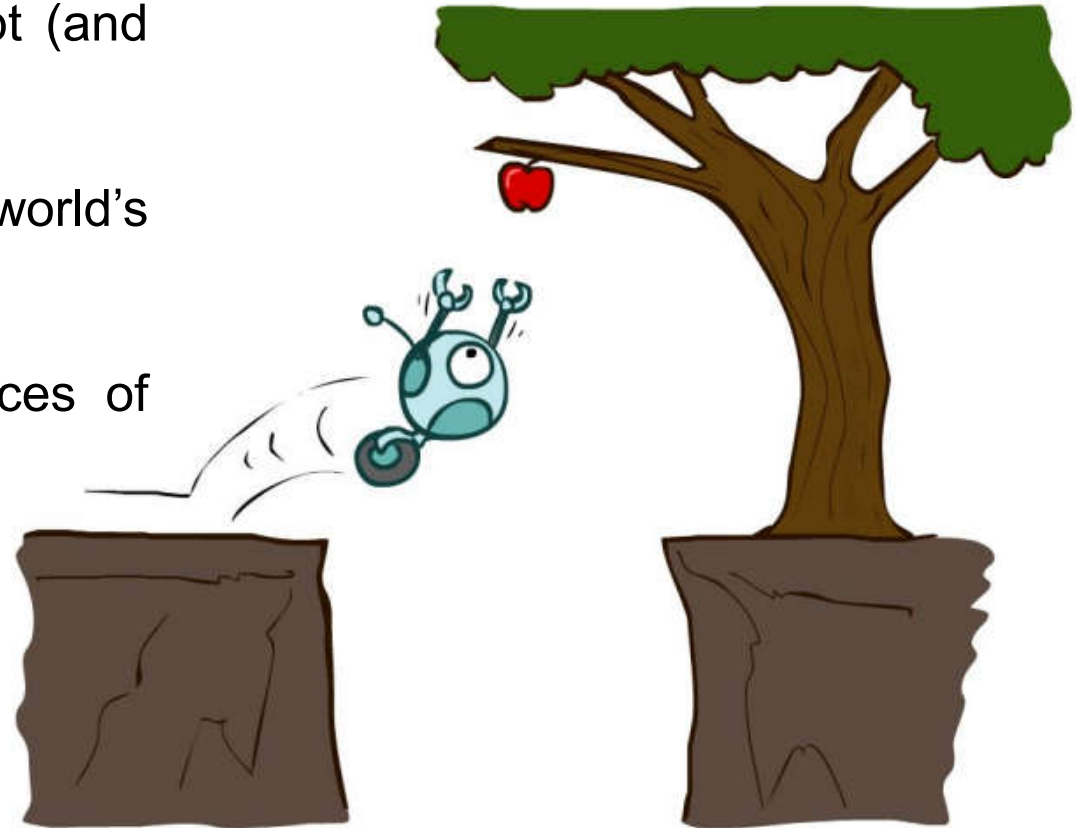
Agents that plan



Reflex agents

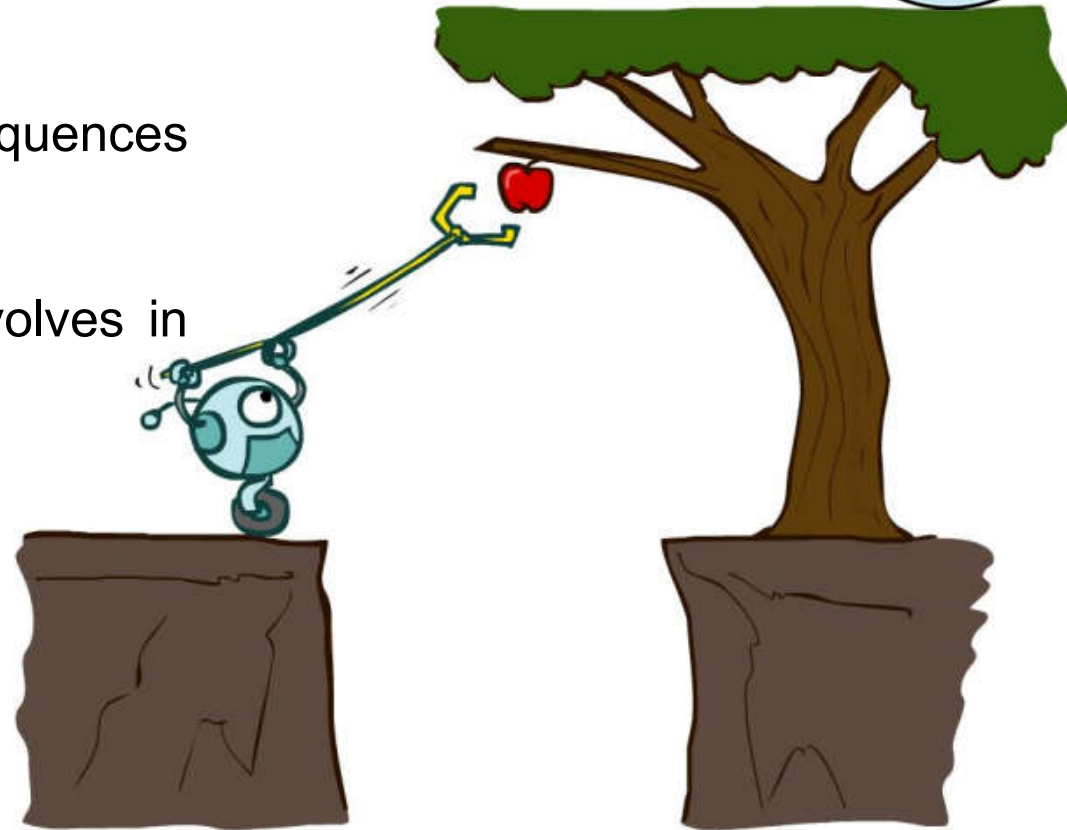
- Choose action based on current percept (and maybe memory)
- May have memory or a model of the world's current state
- Do not consider the future consequences of their actions
- Consider how the world IS

Can a reflex agent be rational?



Planning Agents

- Ask “what if”
- Decisions based on (hypothesized) consequences of actions
- Must have a model of how the world evolves in response to actions
- Must formulate a goal (test)
- Consider how the world WOULD BE
- **Optimal vs. complete planning**
- **Planning vs. replanning**



Question

- A **farmer** wants to get his **cabbage**, **goat**, and **wolf** across a river. He has a boat that only holds two. He cannot leave the cabbage and goat alone or the goat and wolf alone. How many river crossings does he need?

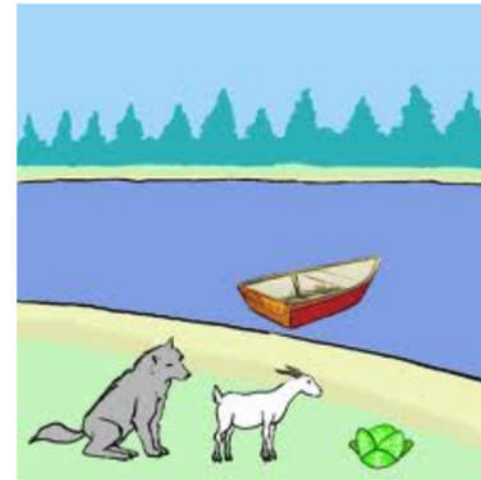
- 4
- 5
- 6
- 7
- no solution



Farmer **C**abbage **G**oat **W**olf

Model the problem

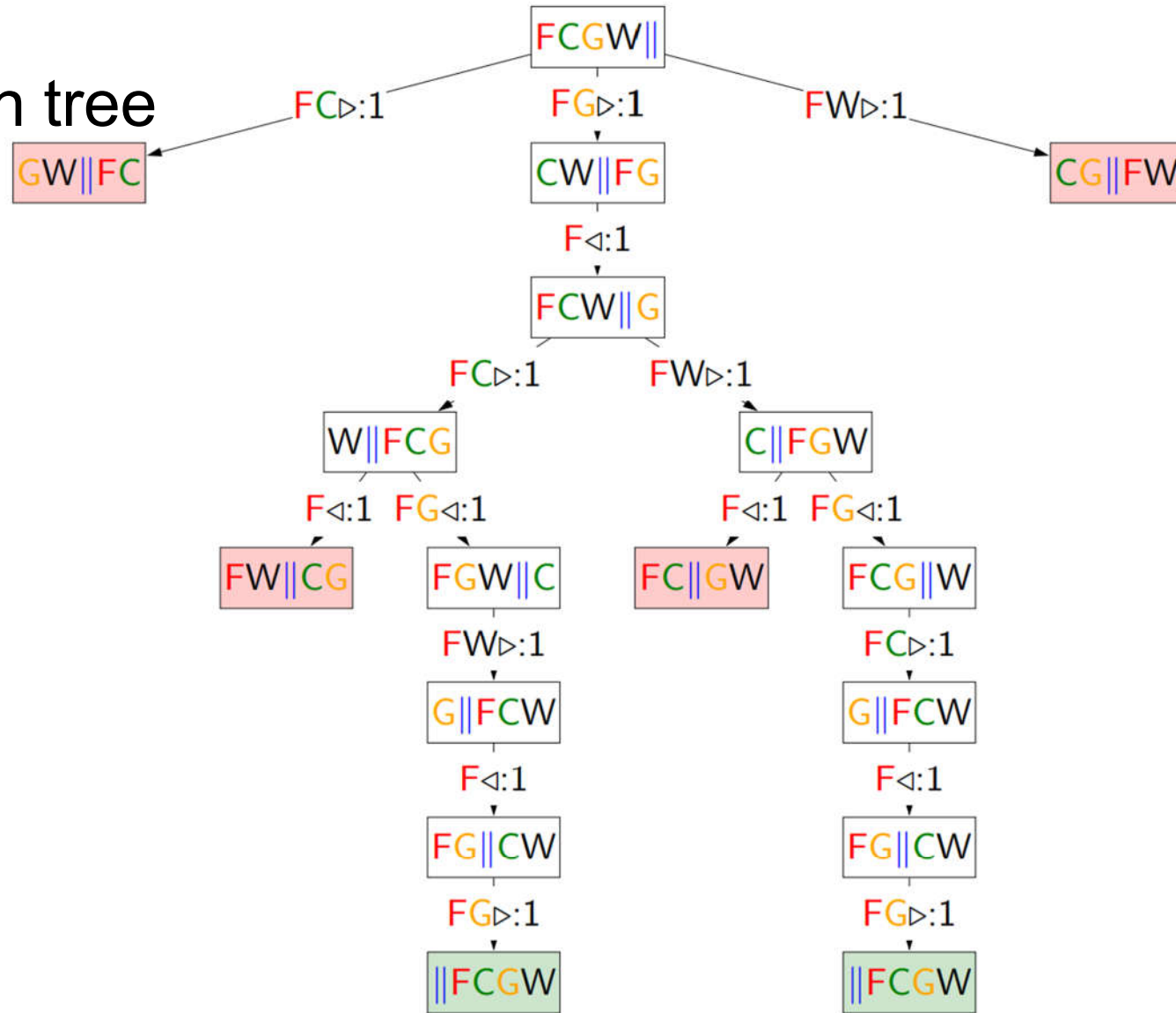
- How many different “states”?
- How many different “actions”?



Farmer Cabbage Goat Wolf

F▷	F◁
FC▷	FC◁
FG▷	FG◁
FW▷	FW◁

Build a search tree

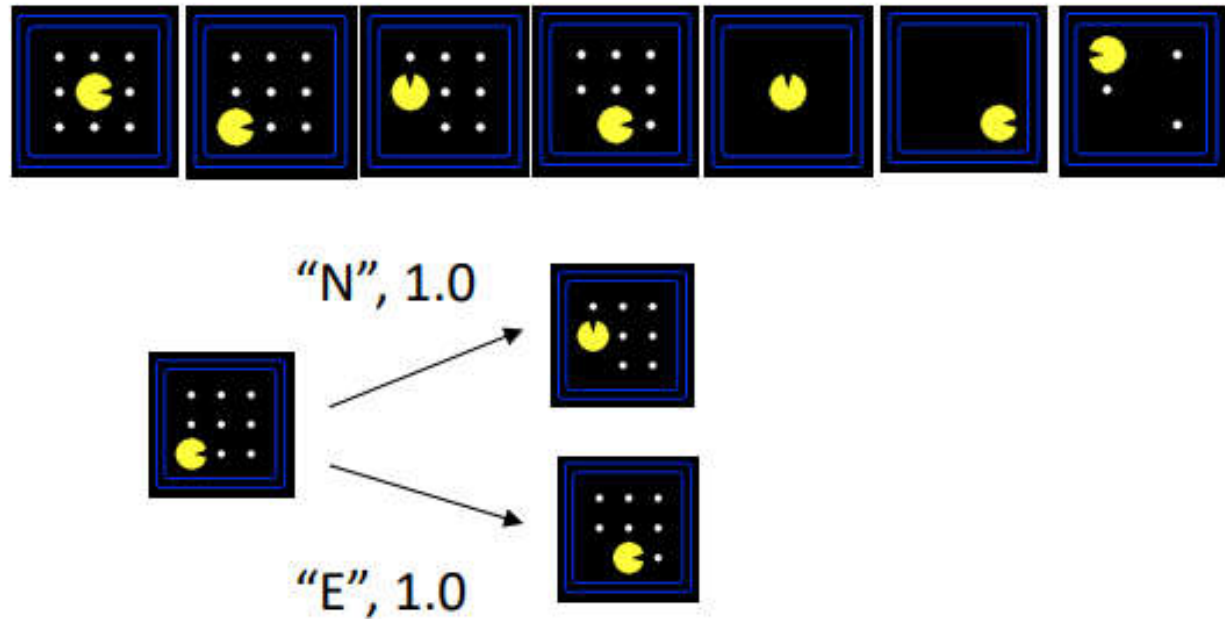


Search problem

- A search problem consists of:

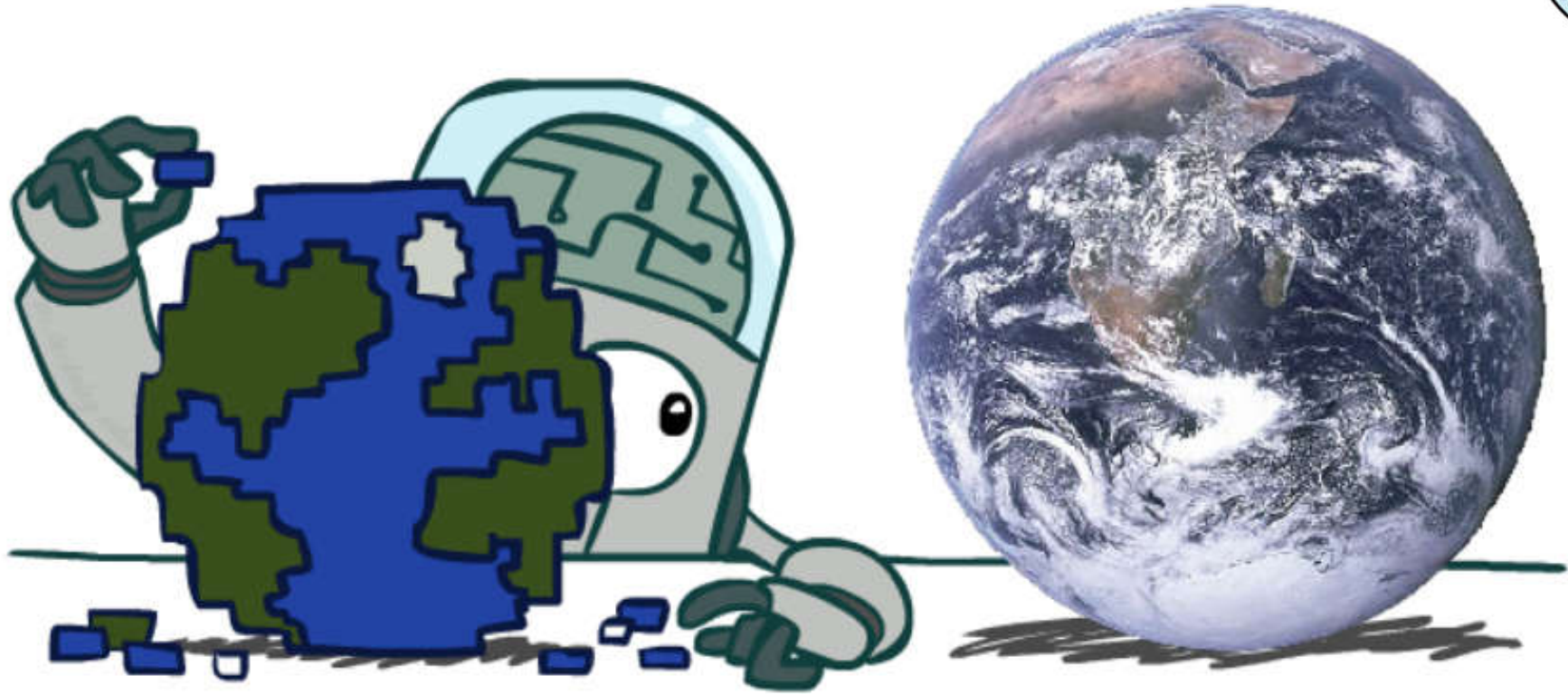
- **State space**
- **Initial state**
- **Goal test**
- **Action**
- **Successor function**

(with actions, costs)



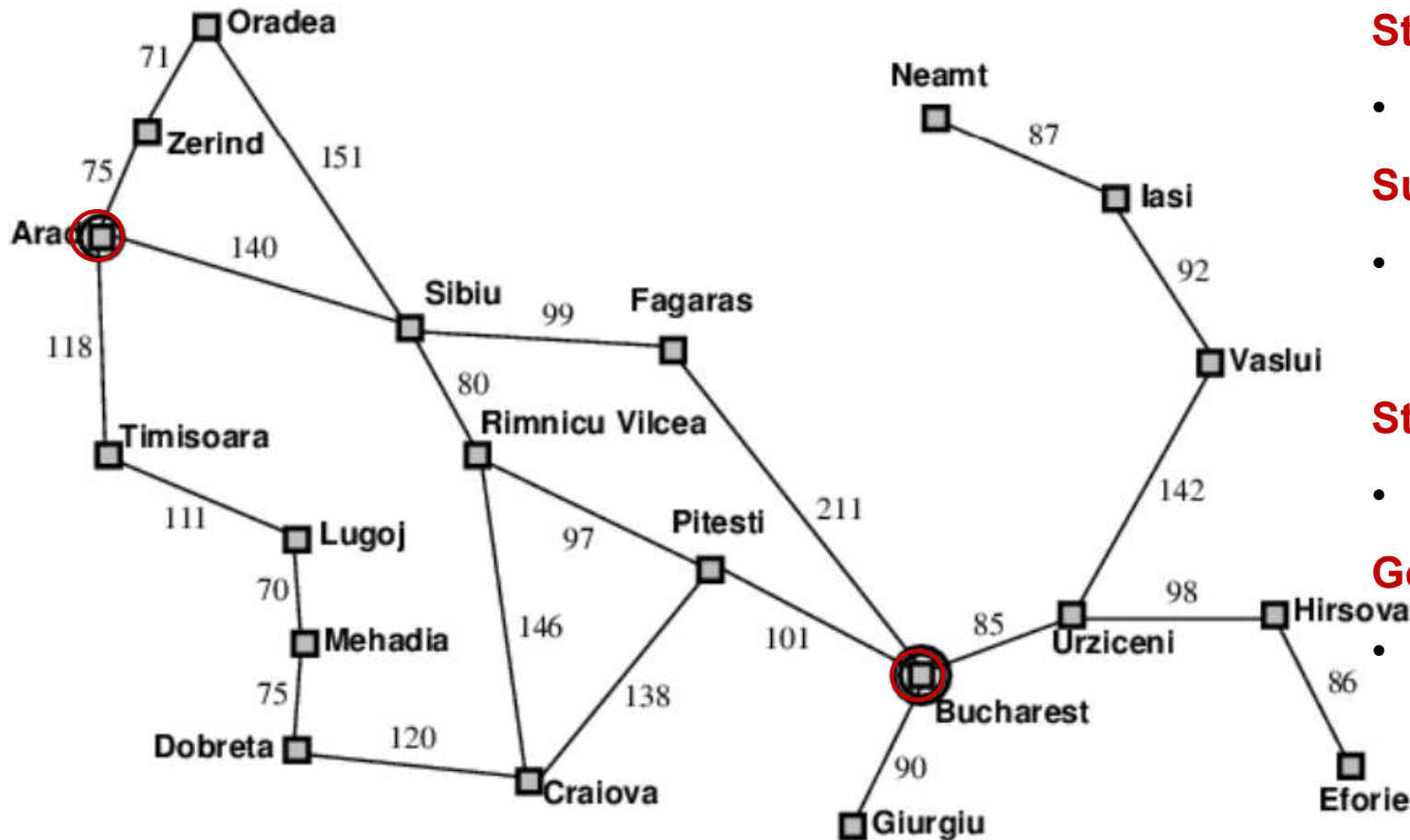
- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state.

Search problems are models



- Models are discrete approximations of the real world.

Example1: Traveling in Romania



State space:

- Cities

Successor function:

- Roads connecting adjacent city with cost = distance

Start state:

- Arad

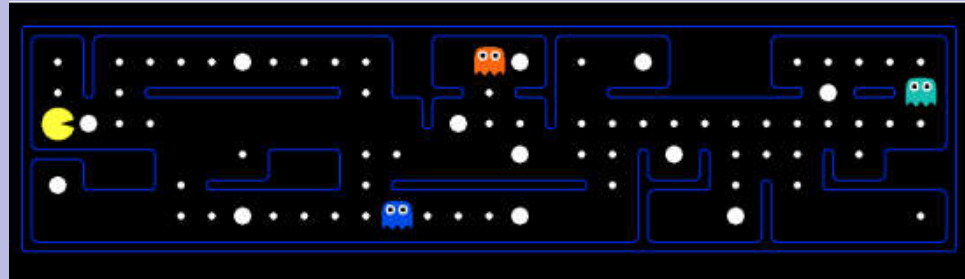
Goal test:

- Is state == Bucharest

Solution: How do we solve this problem?

Example2: PACMAN

The **world** state includes every last detail of the environment



- A **search** state keeps only the details needed for planning (abstraction)

Problem: Pathing

States: $\{(x,y), \text{location}\}$

Actions: NSEW

Successor: update location only

Goal test: $(x,y) == \text{END}$

Problem: Eat all dots

States: $\{(x,y), \text{dot (booleans)}\}$

Actions: NSEW

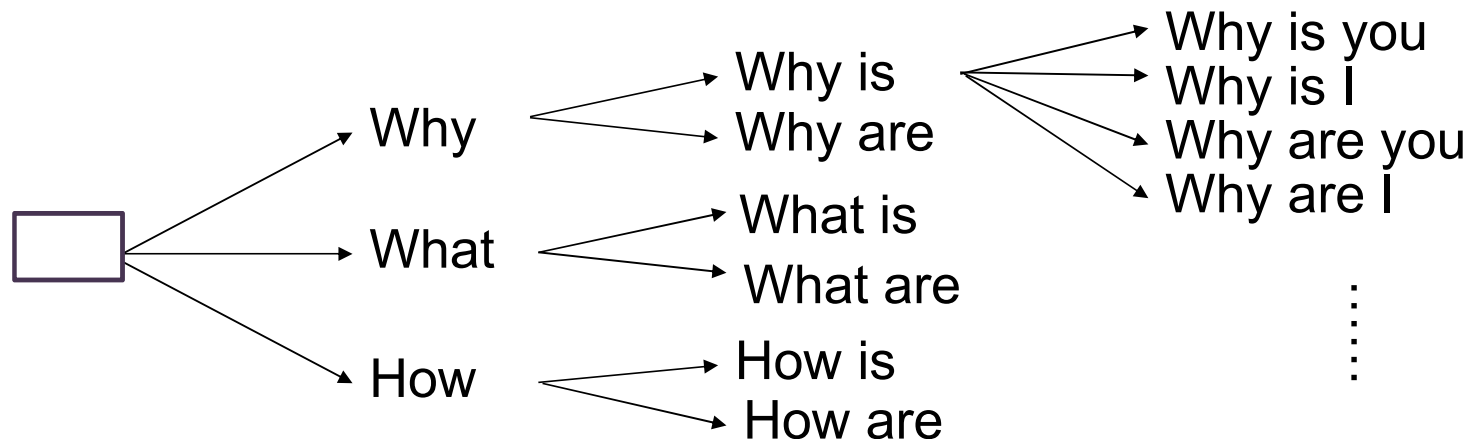
Successor: update location and dots

Goal test: No dots (all false)

Example3-Machine translation

Translate “你好嗎” to English

- **States:** current word sequence
- **Actions:** the next word
- **Successor:** the concatenation of current sequence and next word
- **Goal test:** whether the current sequence means the same as 你好嗎

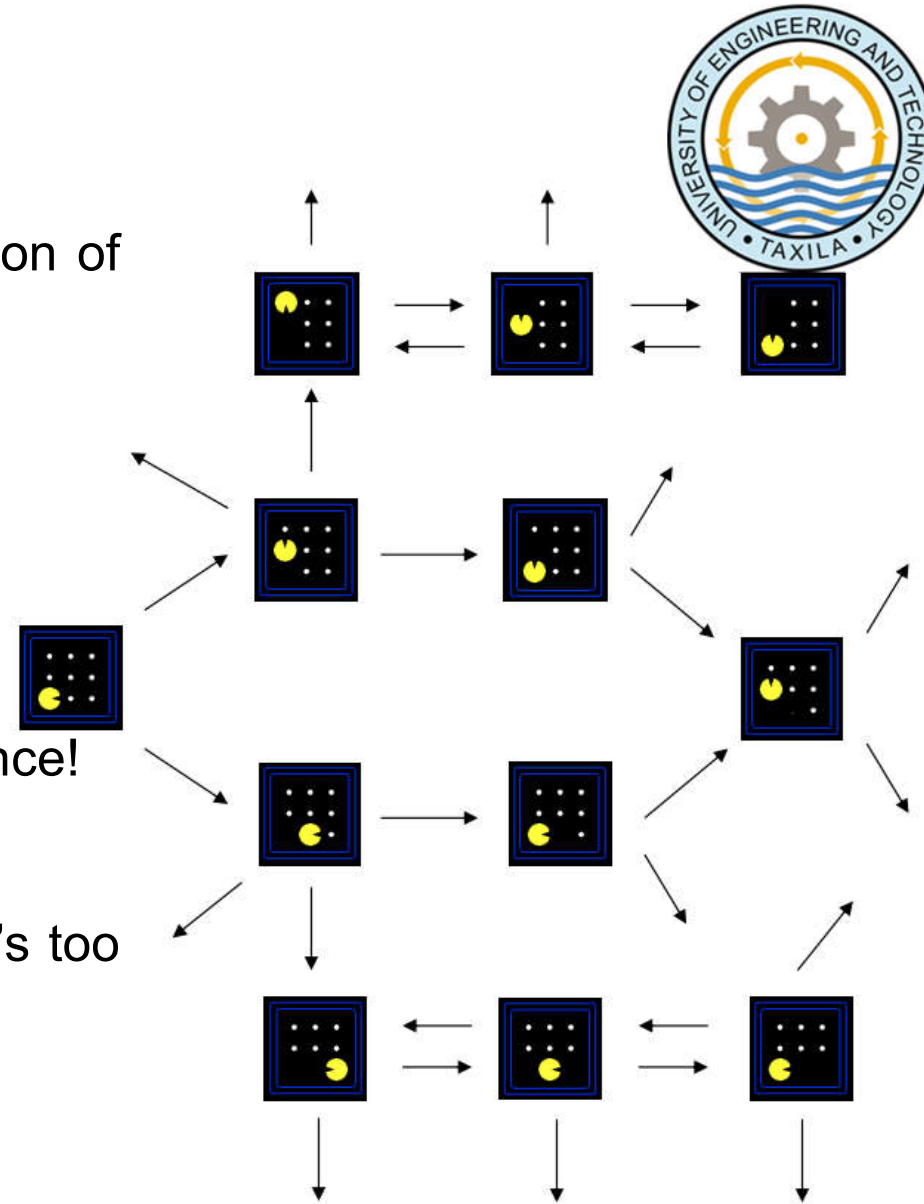




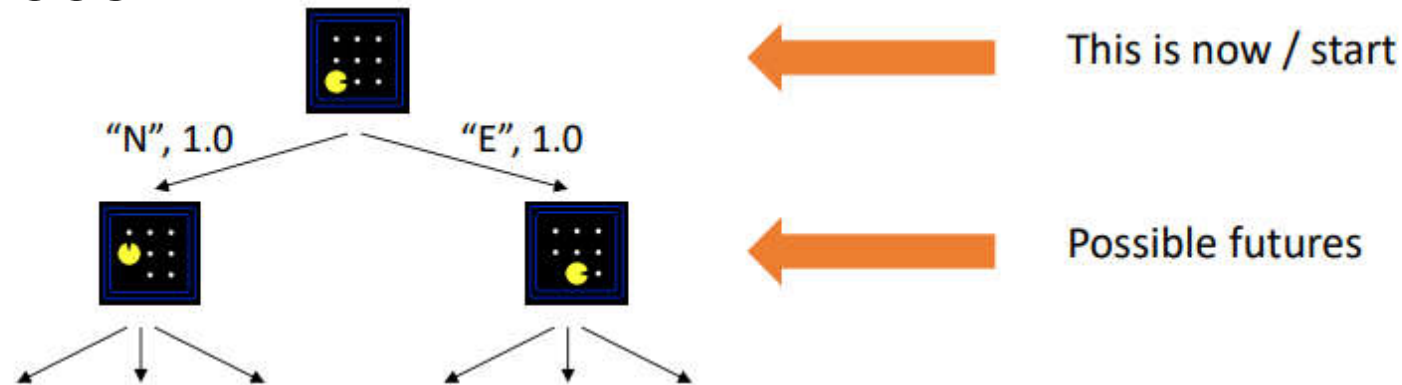
General Framework

State space graph

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea

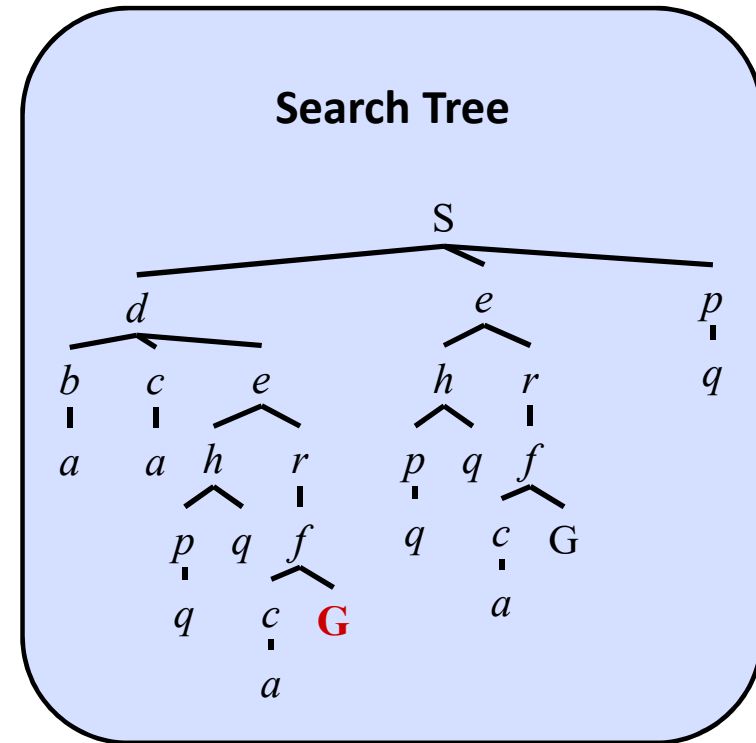
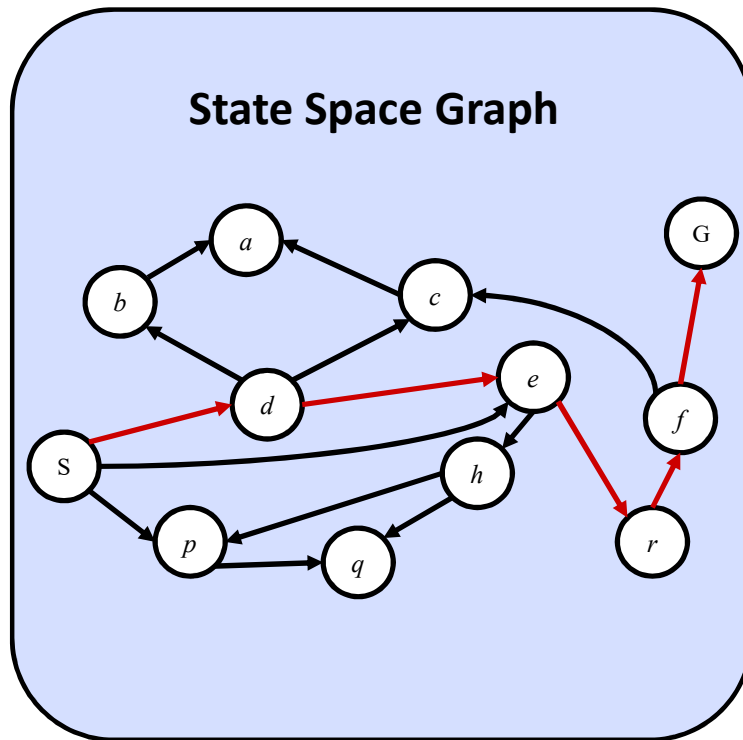


Search trees



- A search tree:
 - A “what if” tree of plans and their outcomes
 - The start state is the root node
 - Children correspond to successors
 - Nodes show states, but correspond to PLANS that achieve those states
 - For most problems, we can never actually build the whole tree

State space graph vs. Search tree



Each NODE in the search tree is an entire PATH in the state space graph.

We construct both on demand and we **construct as little as possible**.

General search tree

Expanded $\leftarrow \{ \}$

Frontier $\leftarrow \{ \text{initial_state} \}$

While **Frontier** is not empty:

Choose a node s from **Frontier** ①

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If s' has not been reached

Put s' in **Frontier**

②

Move s to **Expanded**

③

All nodes that are not in
Expanded or **Frontier**



Nodes are divided into 3 groups: **Expanded**, **Frontier**, and **Unreached**.

① Place the initial state in **Frontier**

Expanded $\leftarrow \{ \}$

Frontier $\leftarrow \{ \text{initial_state} \}$

While **Frontier** is not empty:

① Choose a node s from **Frontier**

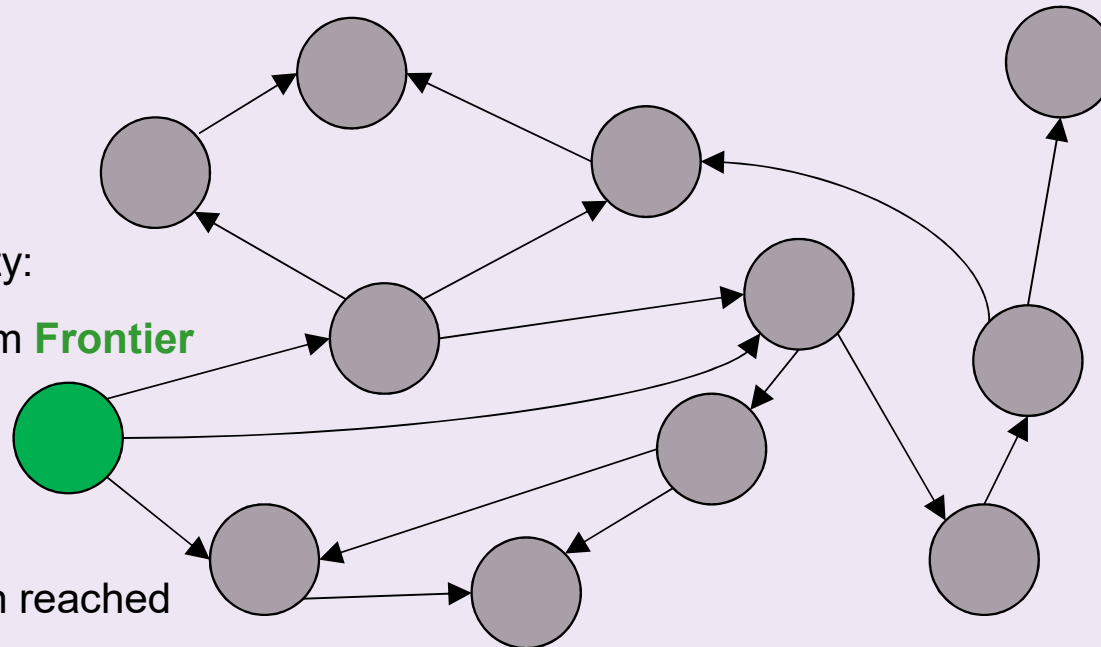
For all action a :

$s' \leftarrow \text{succ}(s, a)$

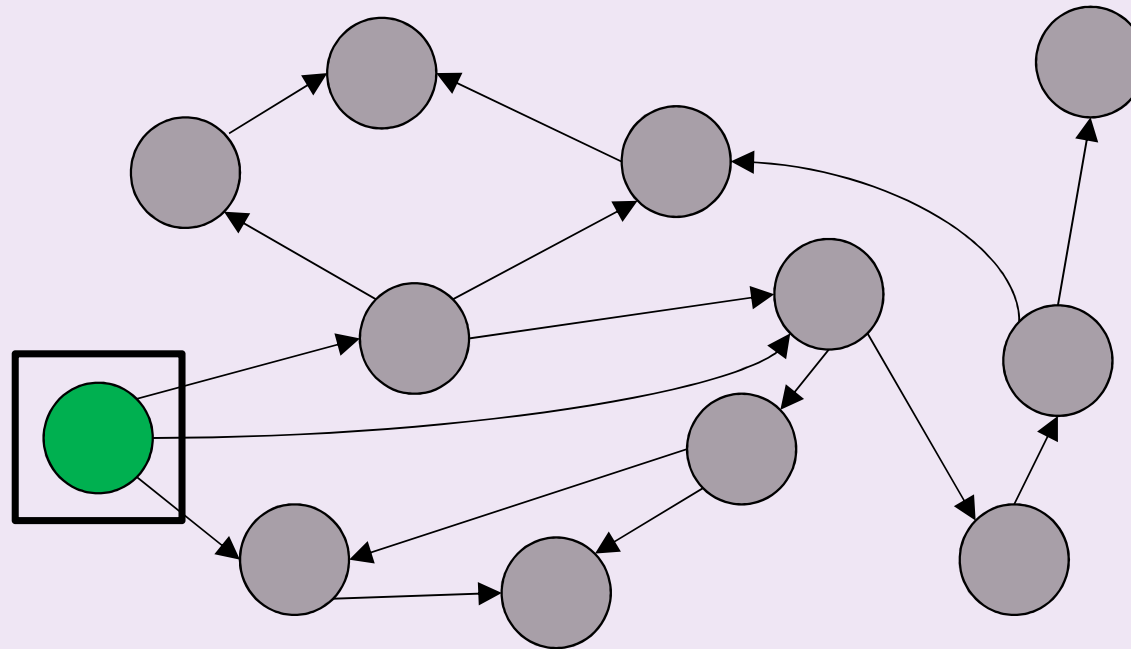
If s' has not been reached

Put s' in **Frontier**

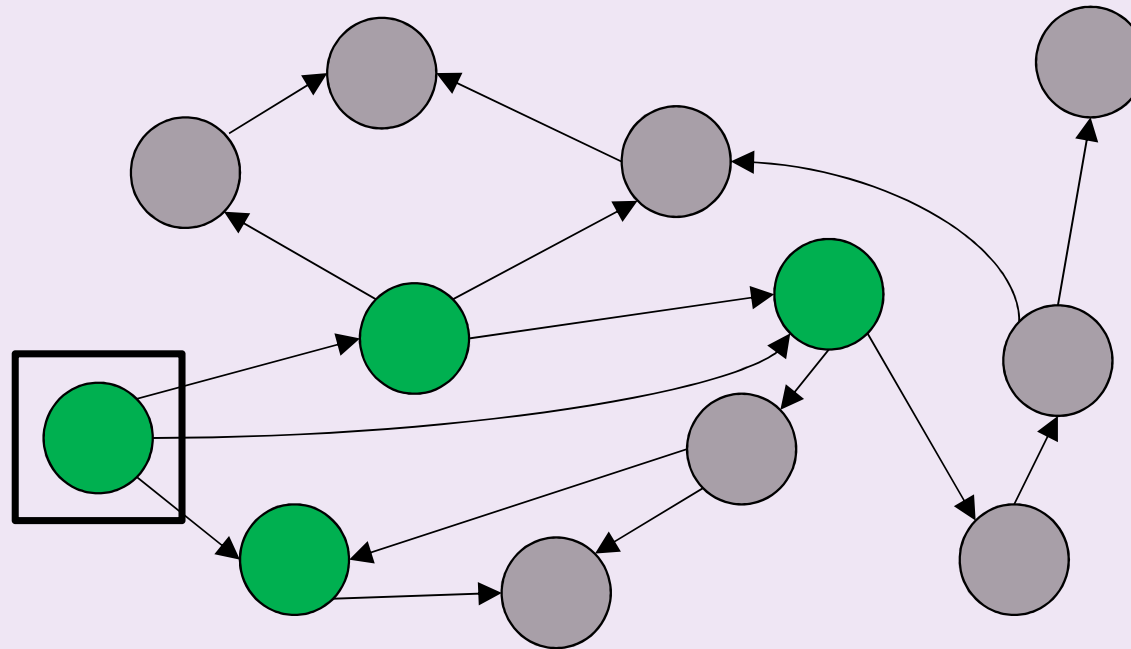
③ Move s to **Expanded**



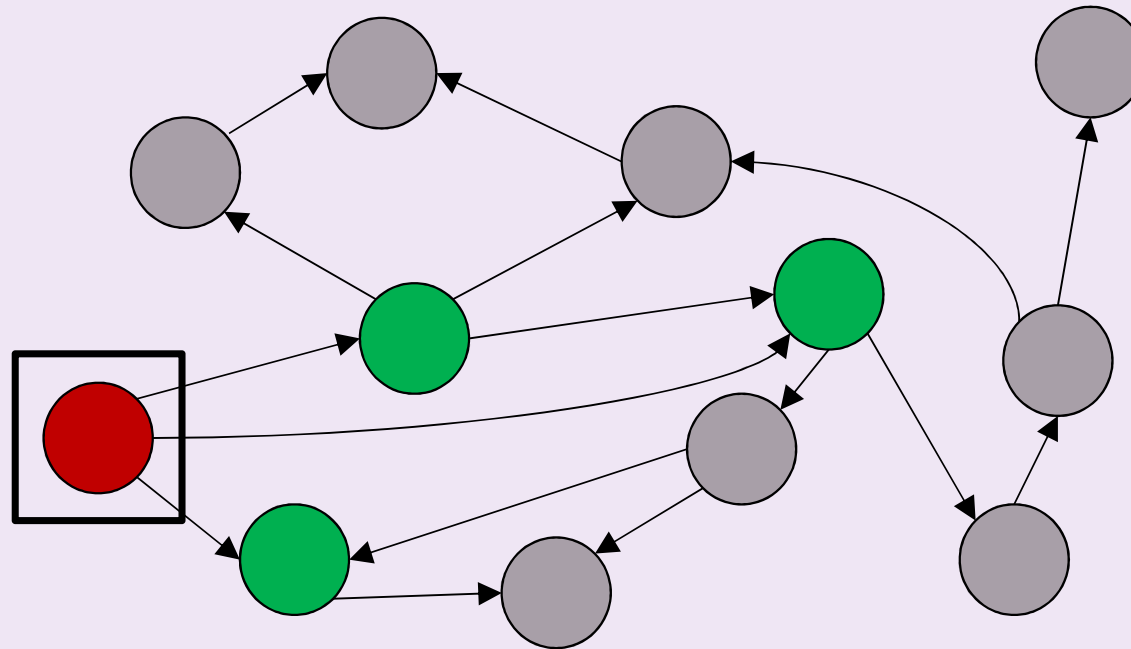
- ① Choose a node s from **Frontier**



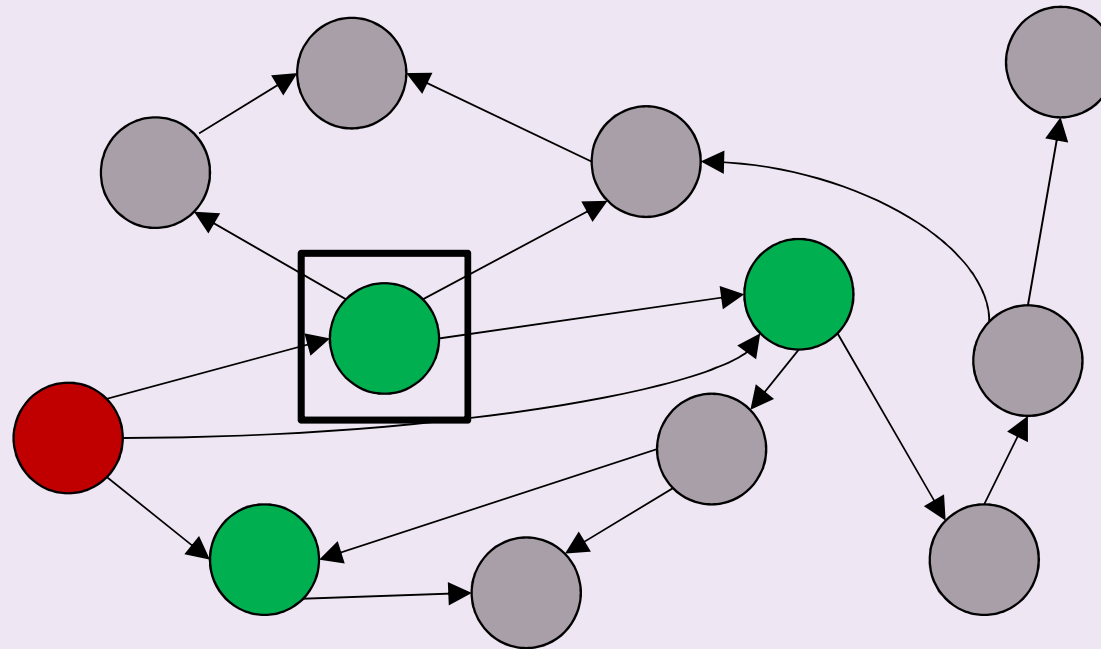
- ② Move all unreached successors of s to **Frontier**



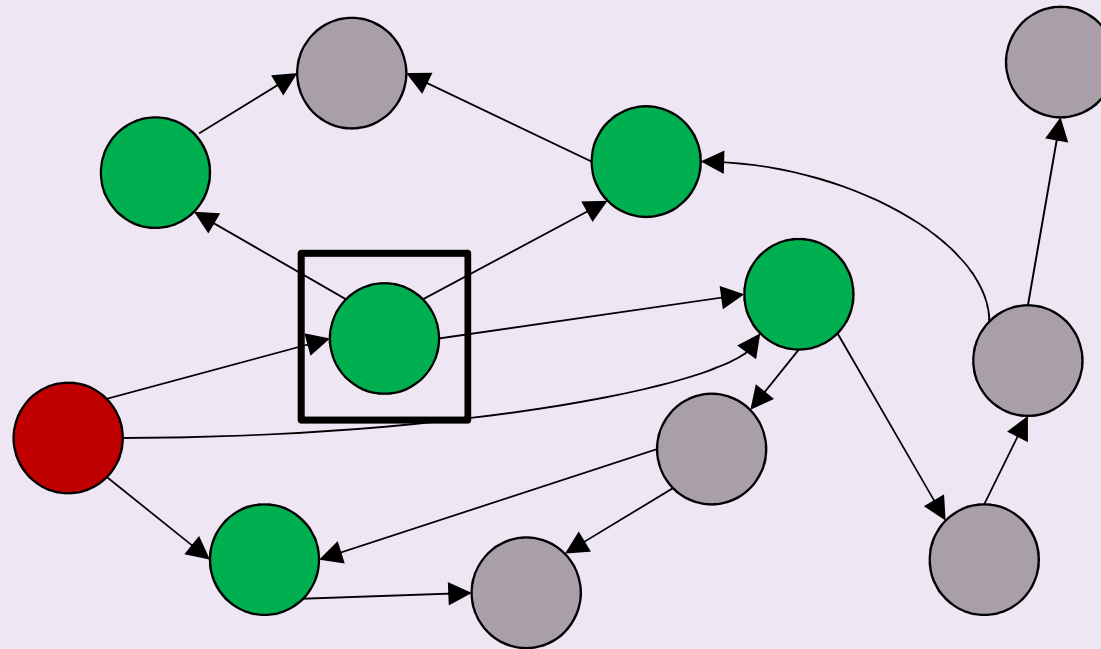
③ Move s to **Expanded**



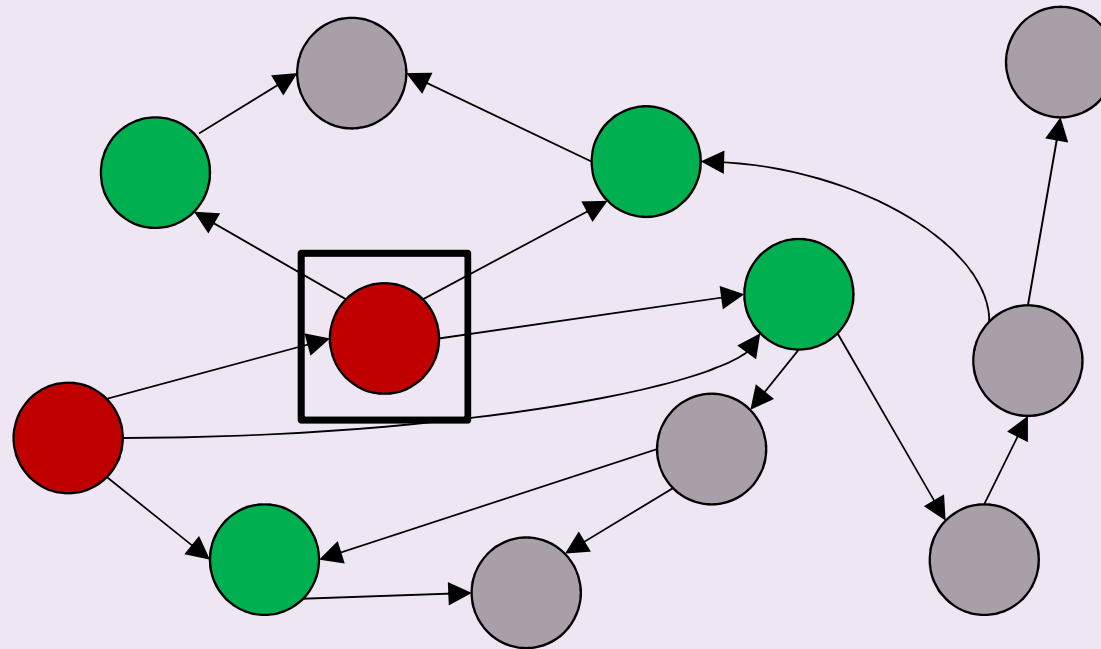
① Choose a node s from **Frontier**



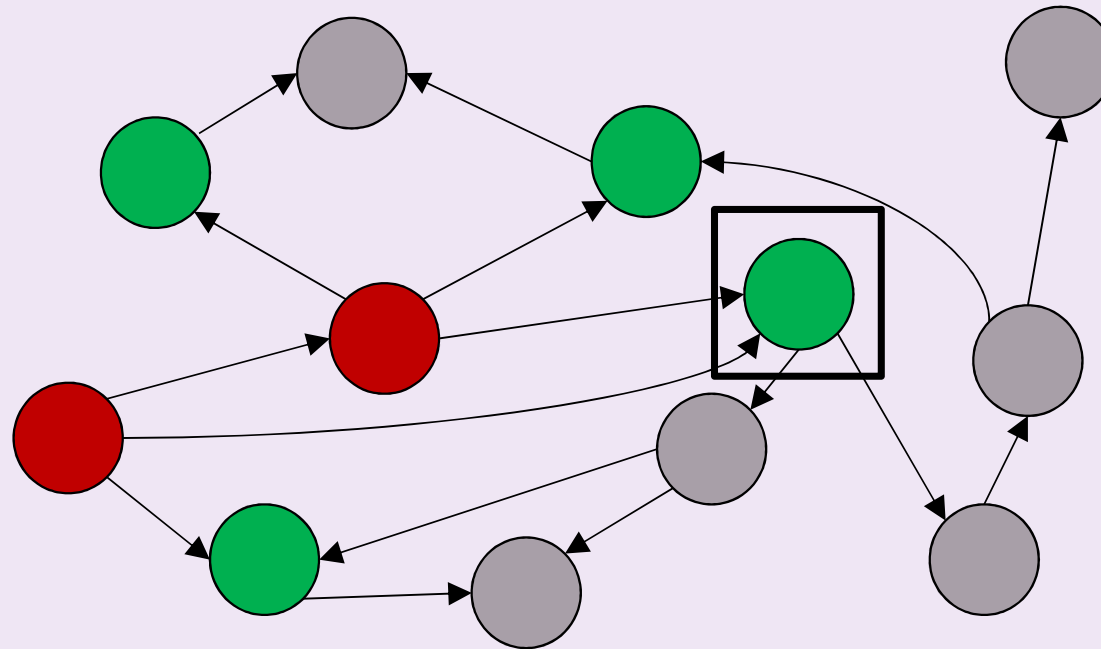
- ② Move all unreached successors of s to **Frontier**



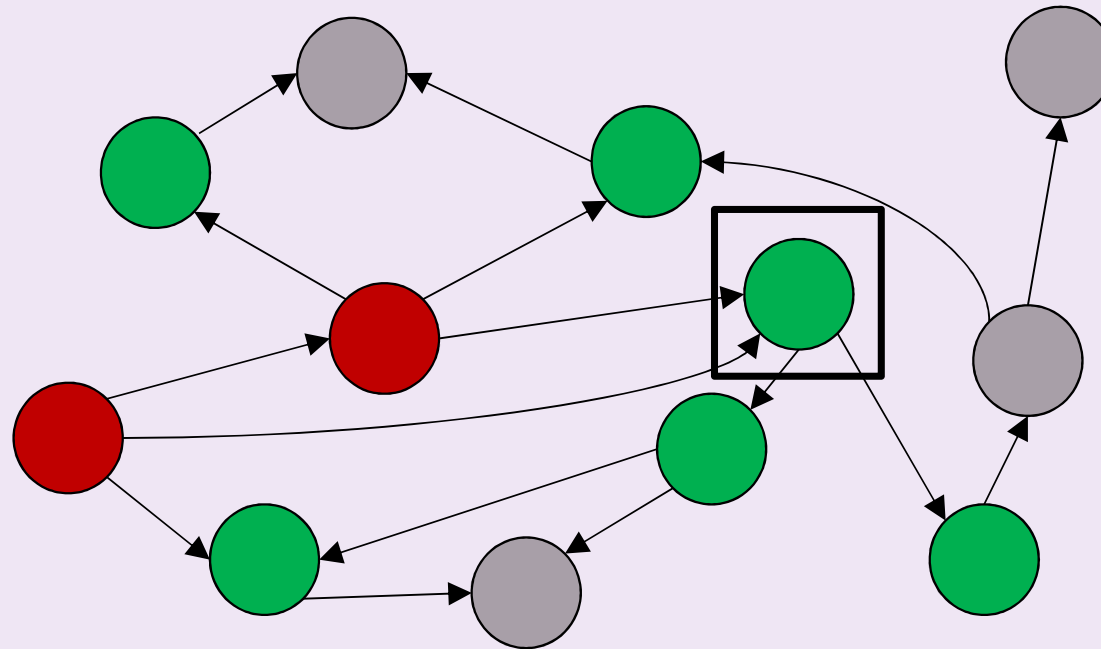
③ Move s to **Expanded**



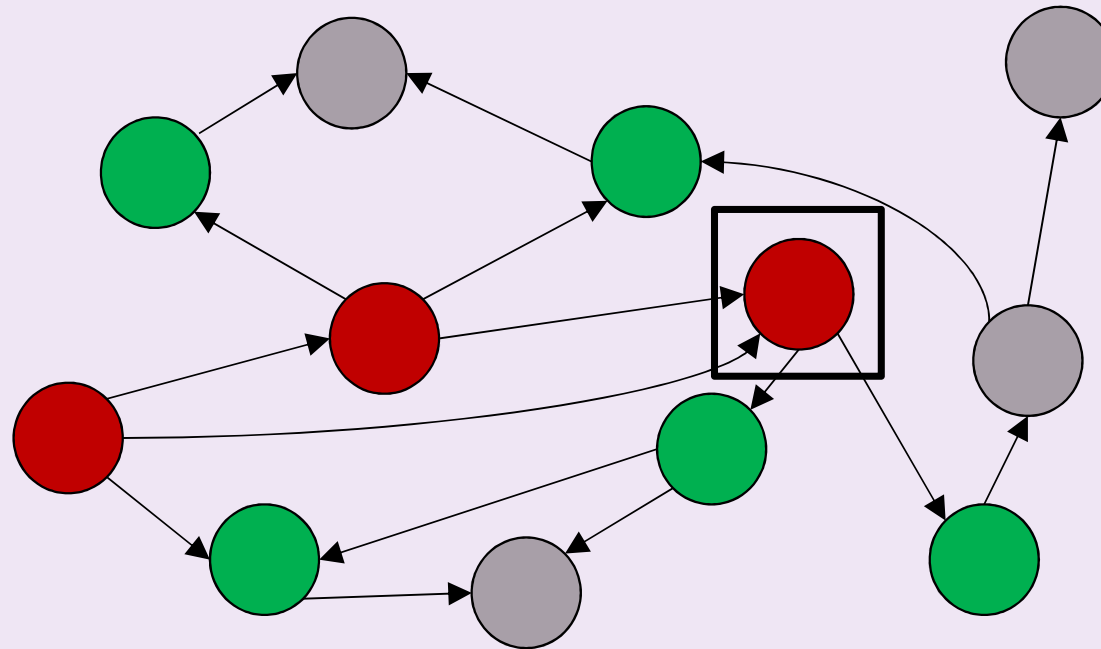
① Choose a node s from **Frontier**



- ② Move all unreached successors of s to **Frontier**



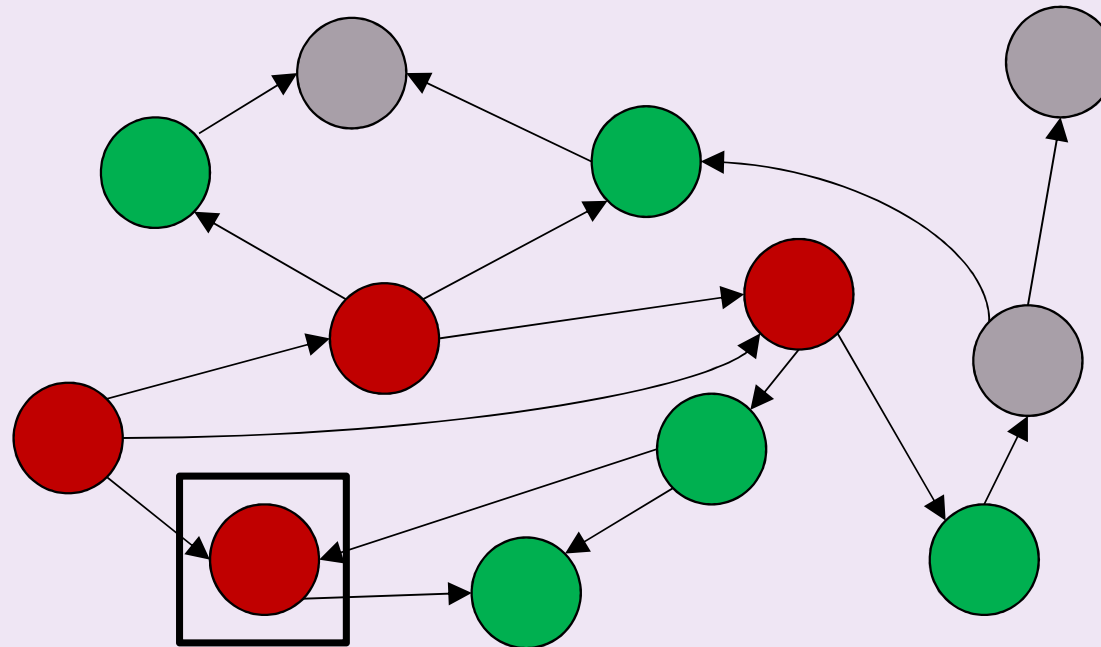
③ Move s to **Expanded**







③ Move s to **Expanded**



Implementation

Expanded $\leftarrow \{ \}$

Frontier $\leftarrow \{ \text{initial_state} \}$

While **Frontier** is not empty:

Choose a node s from **Frontier** ①

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If s' has not been reached ②

Put s' in **Frontier**

Move s to **Expanded** ③

Choose and then remove it

Frontier $\leftarrow \{ \text{initial_state} \}$

While **Frontier** is not empty:

Pop a node s from **Frontier**

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If not **Reached** $[s']$:

Put s' in **Frontier**

Reached $[s'] \leftarrow \text{True}$

Termination when goal is encountered

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

If s is a goal state, terminate

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If not **Reached** $[s']$:

Put s' in **Frontier**

Reached $[s'] \leftarrow \text{True}$

Late Goal Test

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If not **Reached** $[s']$:

If s' is a goal state, terminate

Put s' in **Frontier**

Reached $[s'] \leftarrow \text{True}$

Early Goal Test



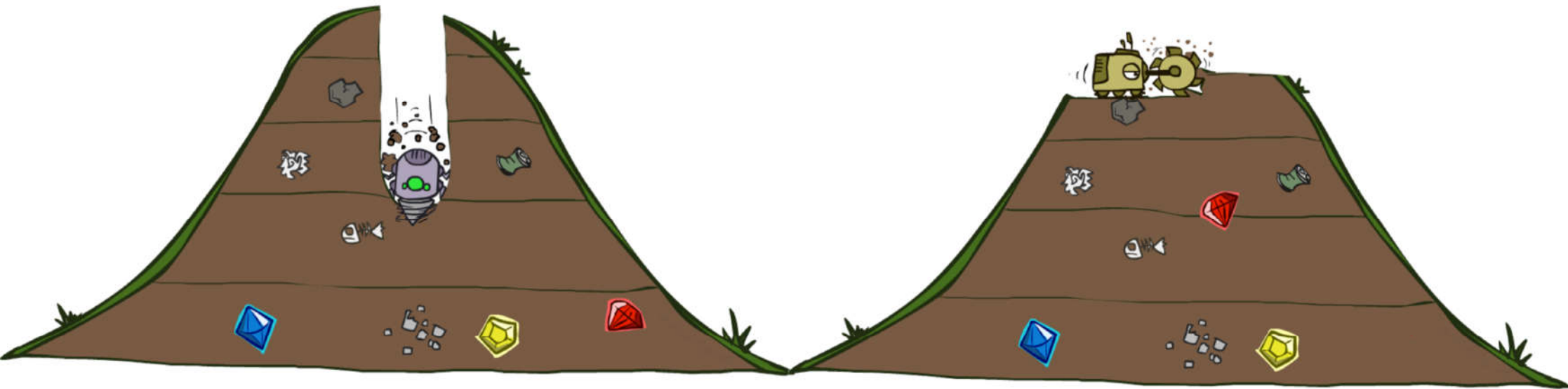
Termination when goal is encountered

- Early Goal Test allows quicker termination when a goal is found.
 - Breadth First Search
 - Depth First Search
- However, when actions are associated with costs and we want to find a **minimum cost solution** (i.e., cost-sensitive), we may have to use the Late Goal Test.
 - Uniform Cost Search (Dijkstra Algorithm)



Uninformed Search

DFS and BFS



Depth First Search

Breadth First Search

DFS

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier** differ here

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If not **Reached** $[s']$:

If s' is a goal state, terminate

Put s' in **Frontier**

Reached $[s'] \leftarrow$ True

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop the **newest** node s from **Frontier**

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If not **Reached** $[s']$:

If s' is a goal state, terminate

Put s' in **Frontier**

Reached $[s'] \leftarrow$ True

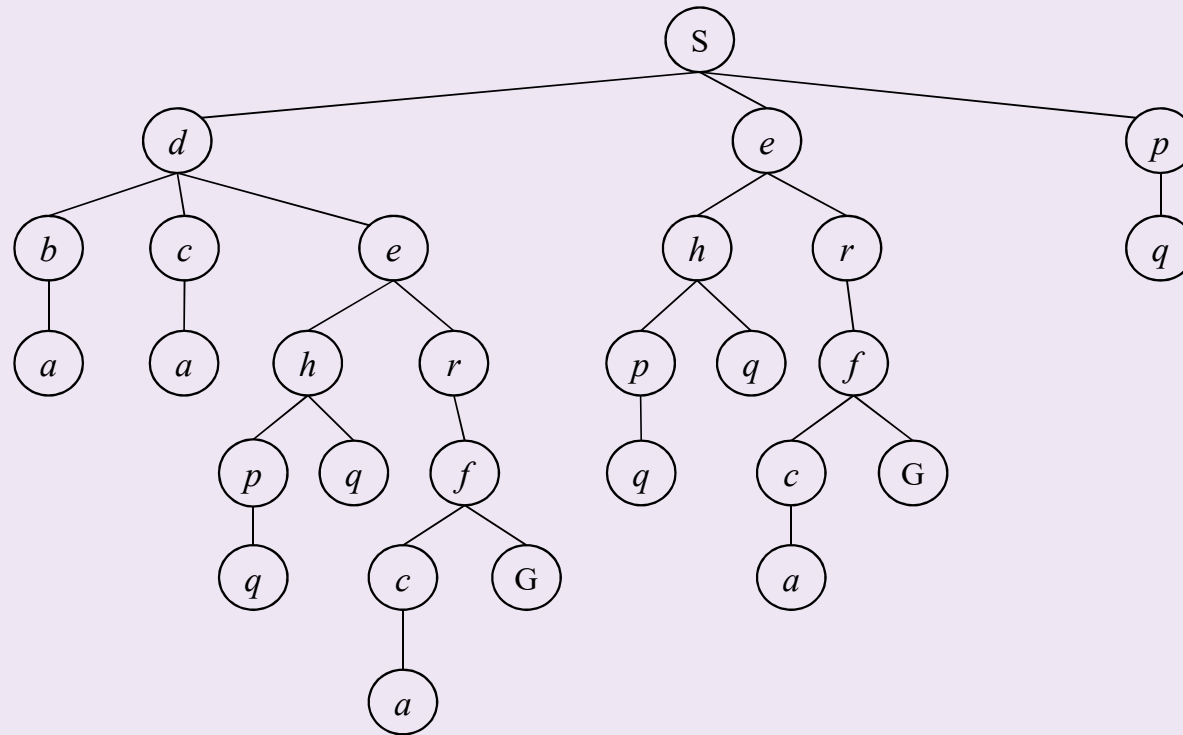
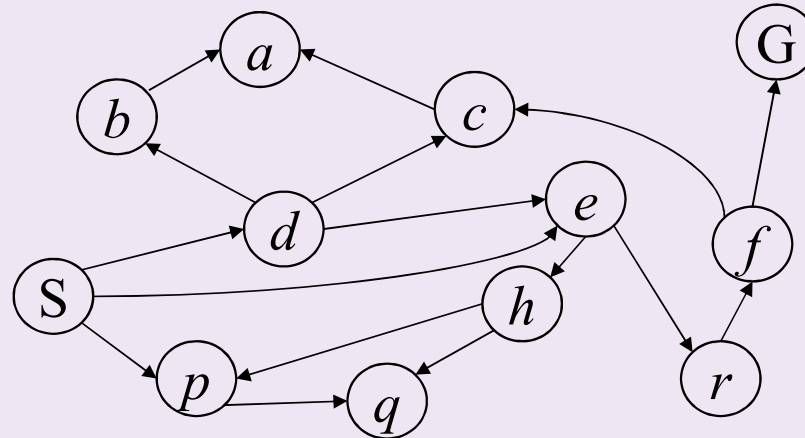
DFS

Strategy:

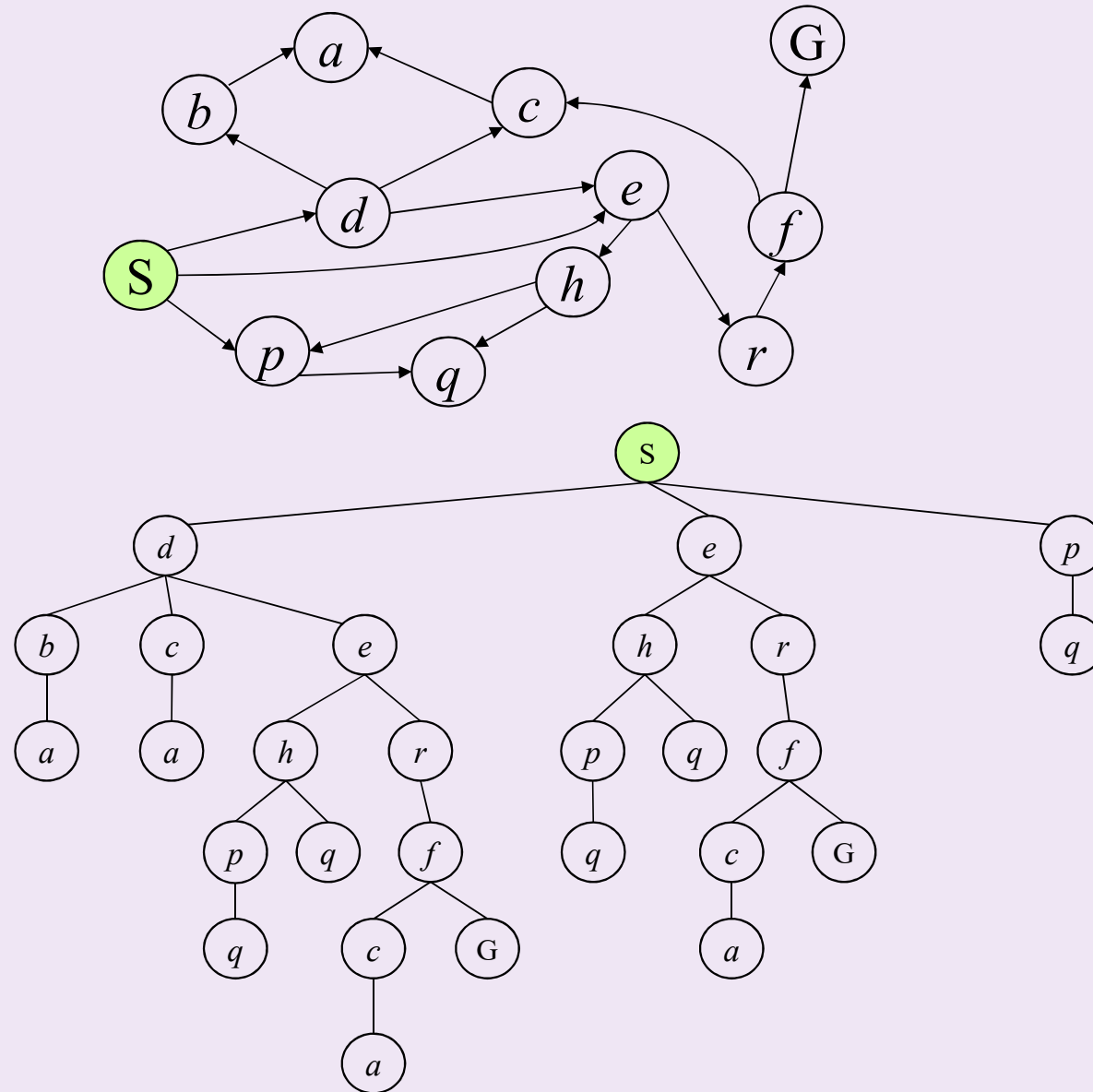
expand a deepest node first

Implementation:

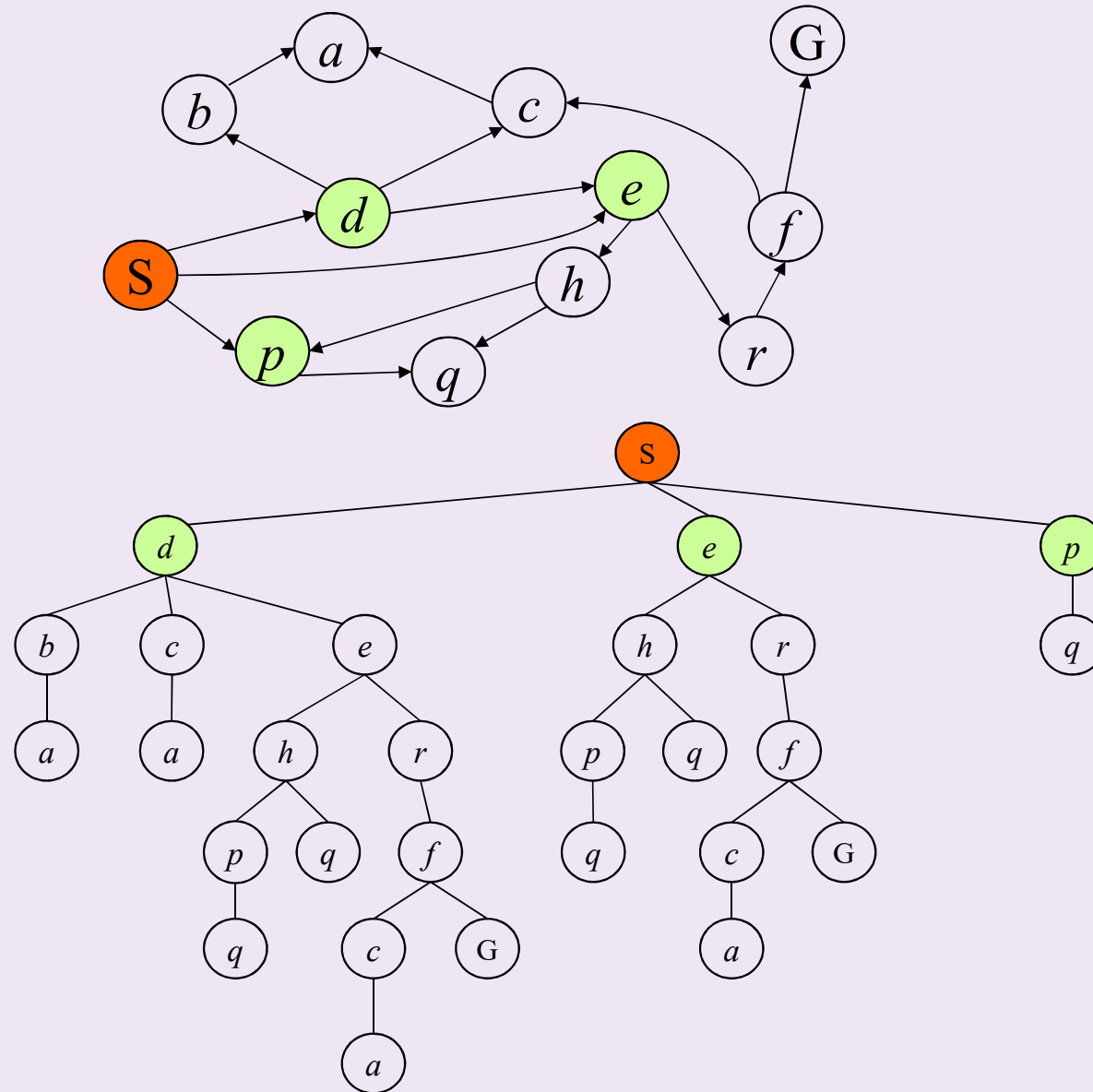
Frontier is a LIFO stack



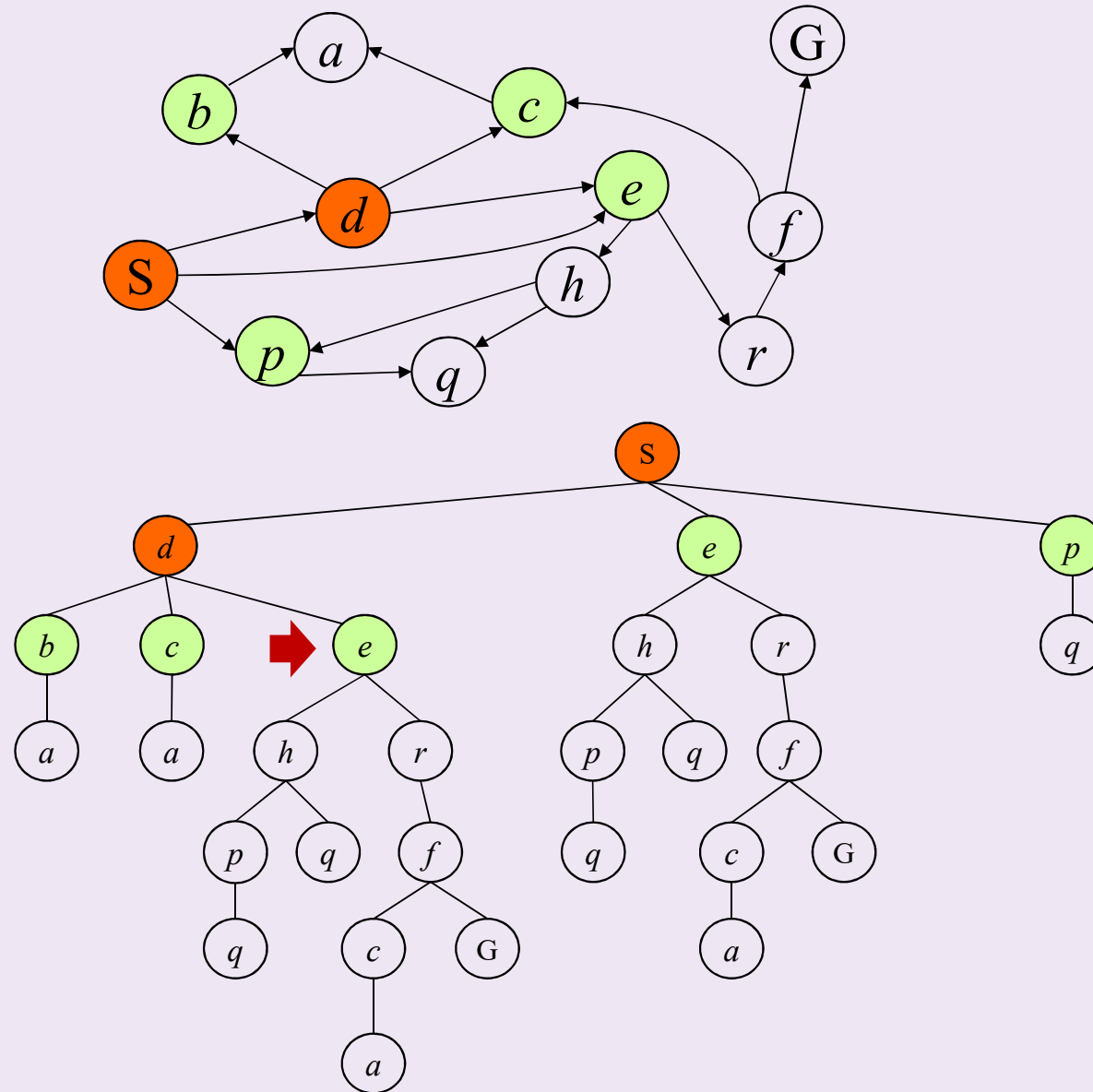
DFS (cont...)



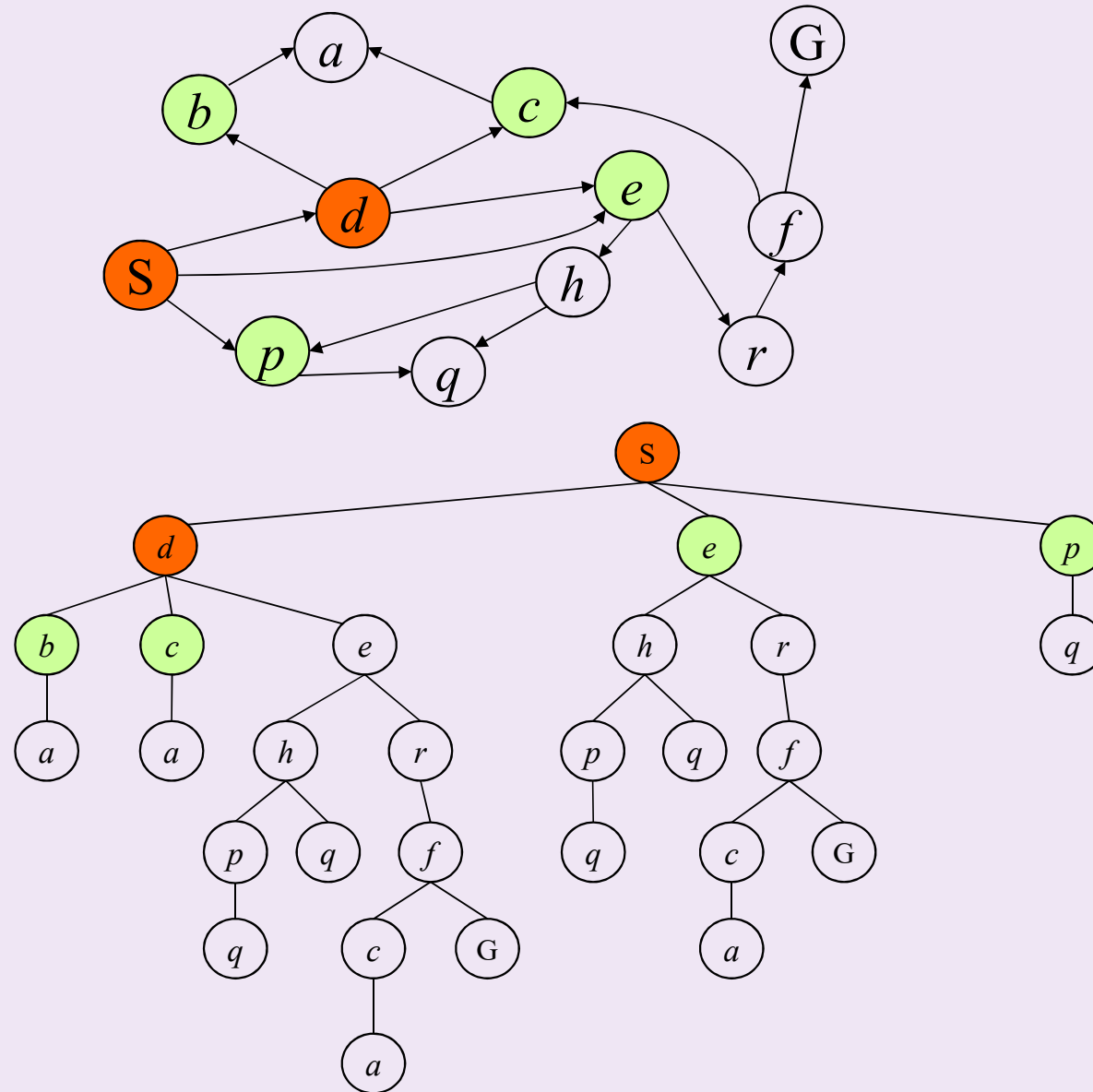
DFS (cont...)



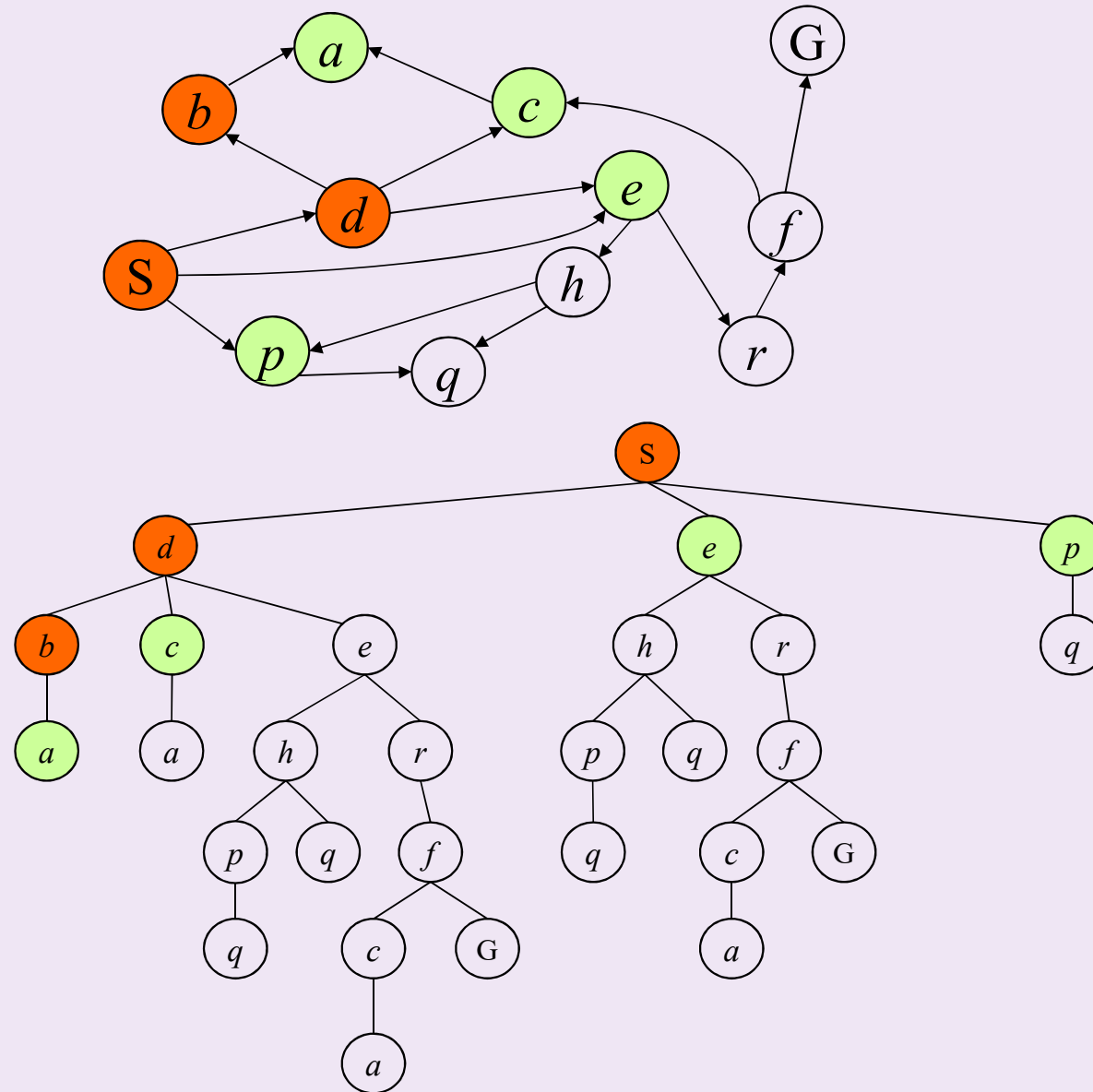
DFS (cont...)



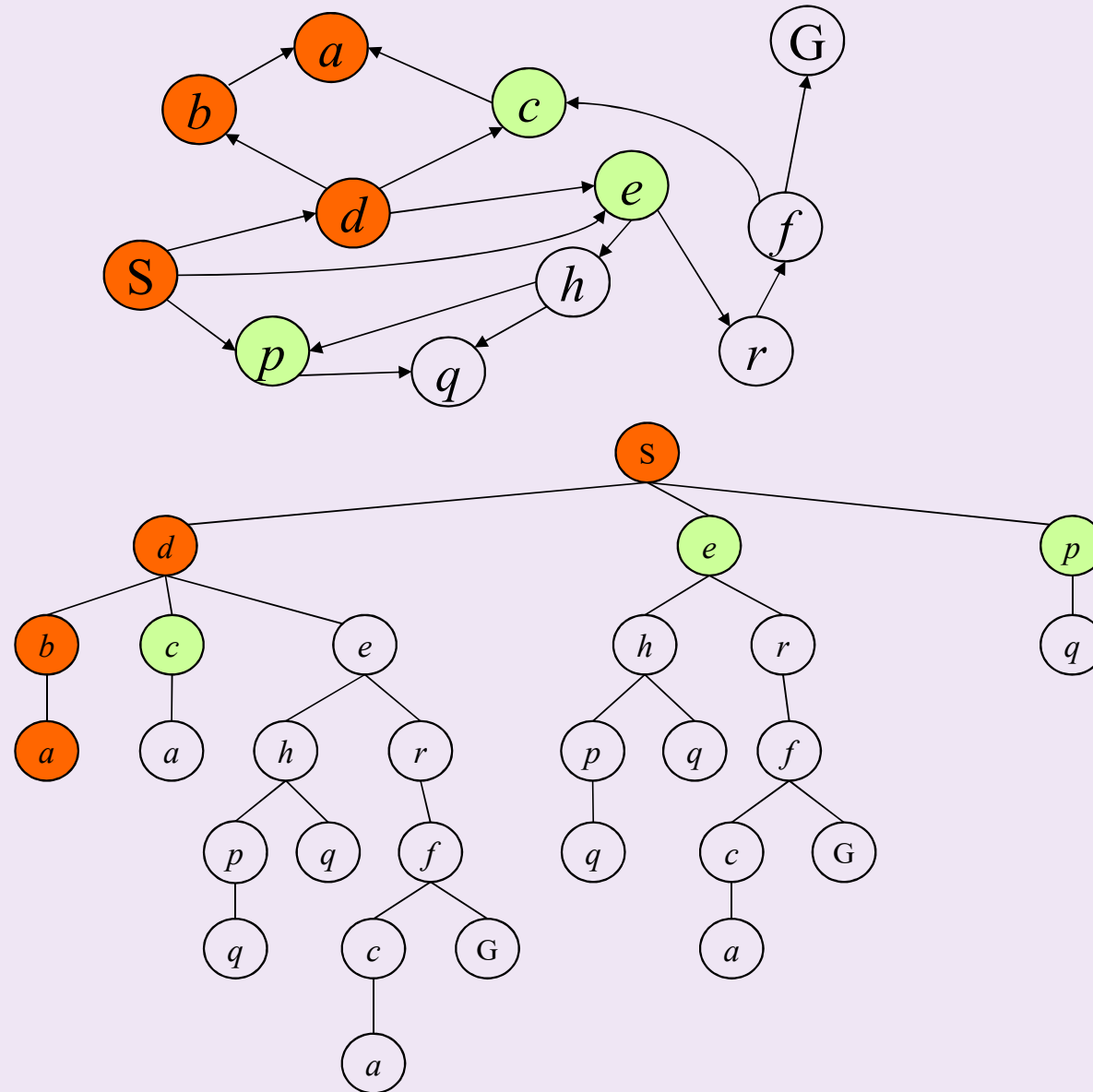
DFS (cont...)



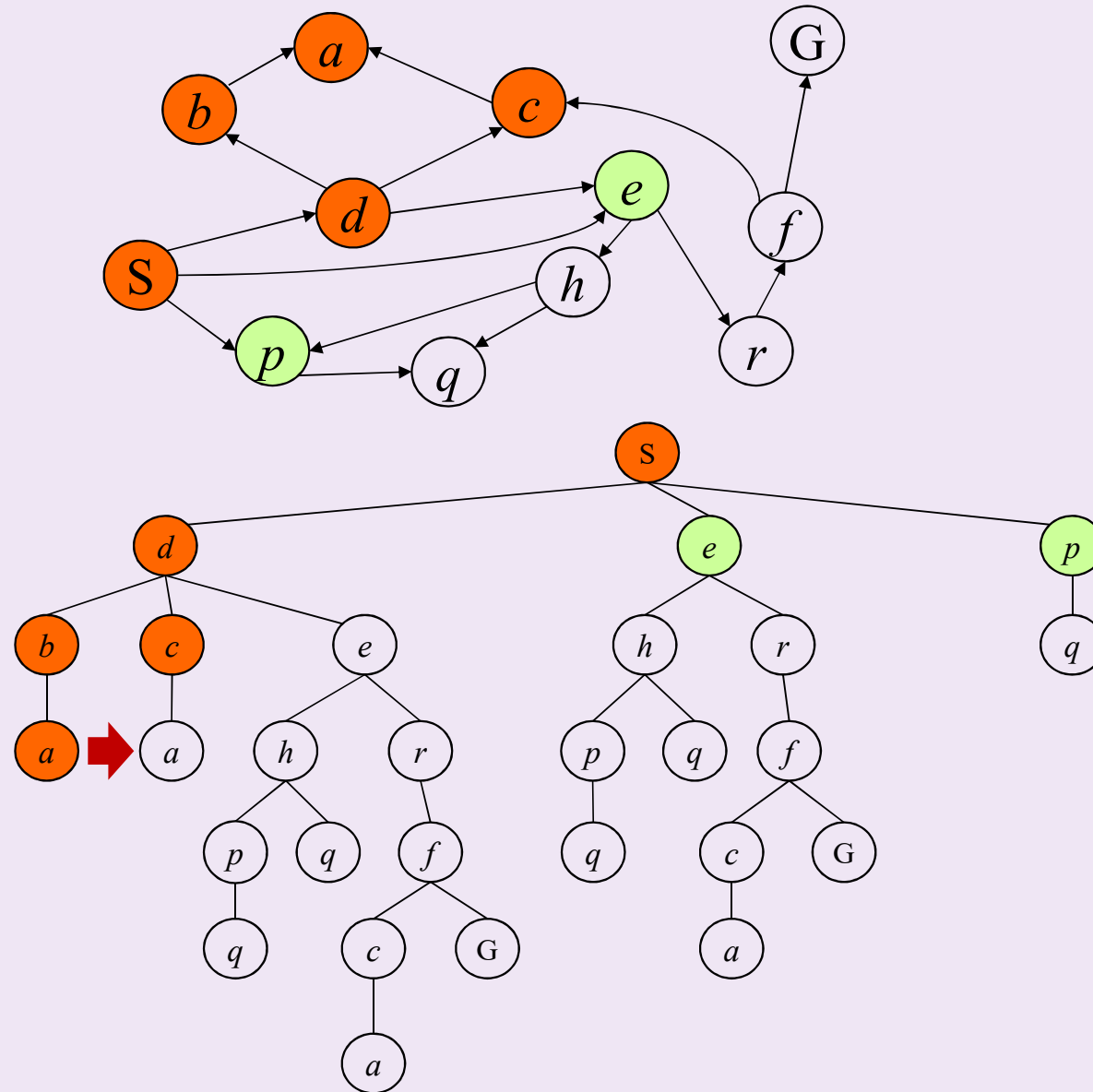
DFS (cont...)



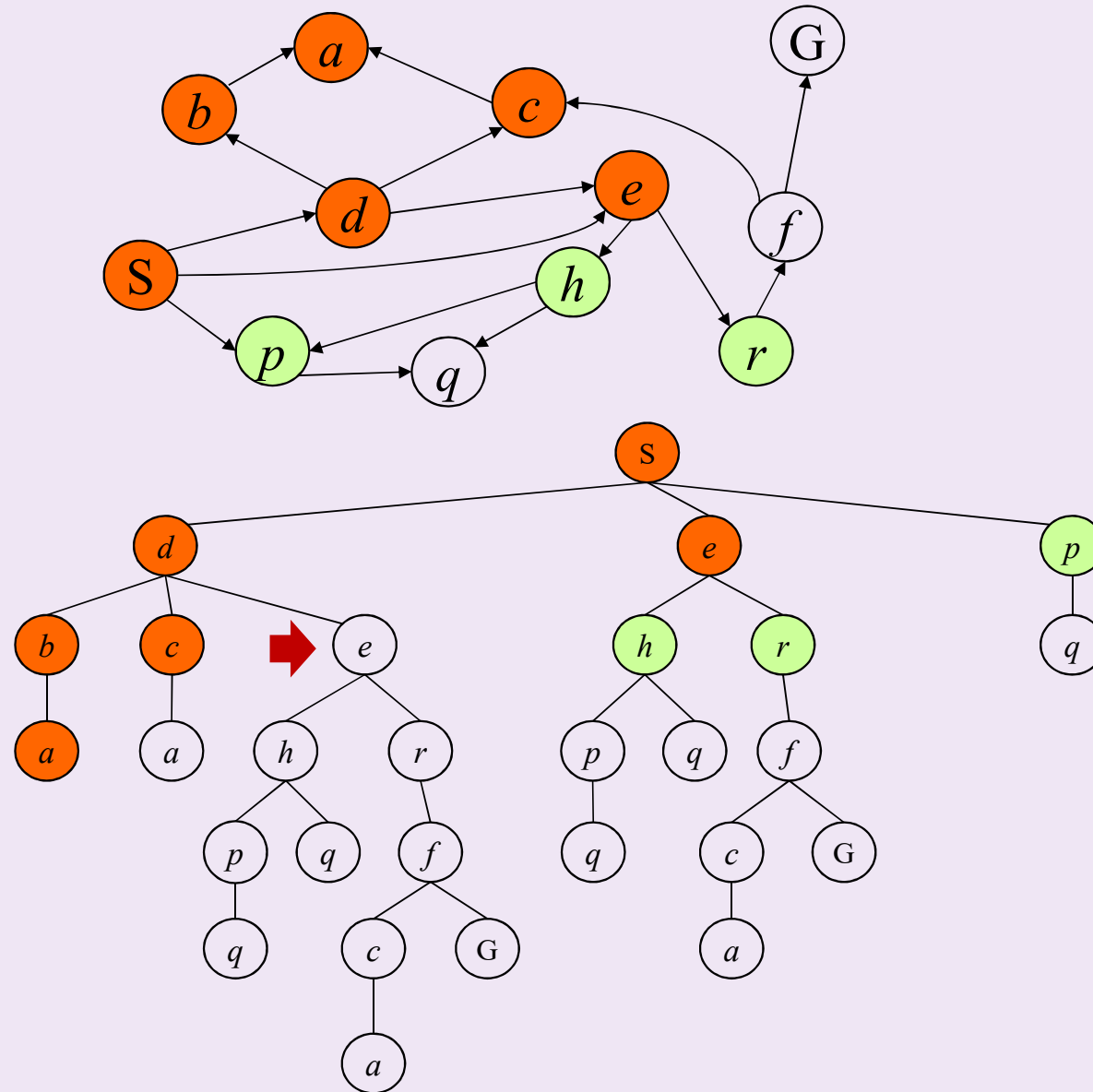
DFS (cont...)



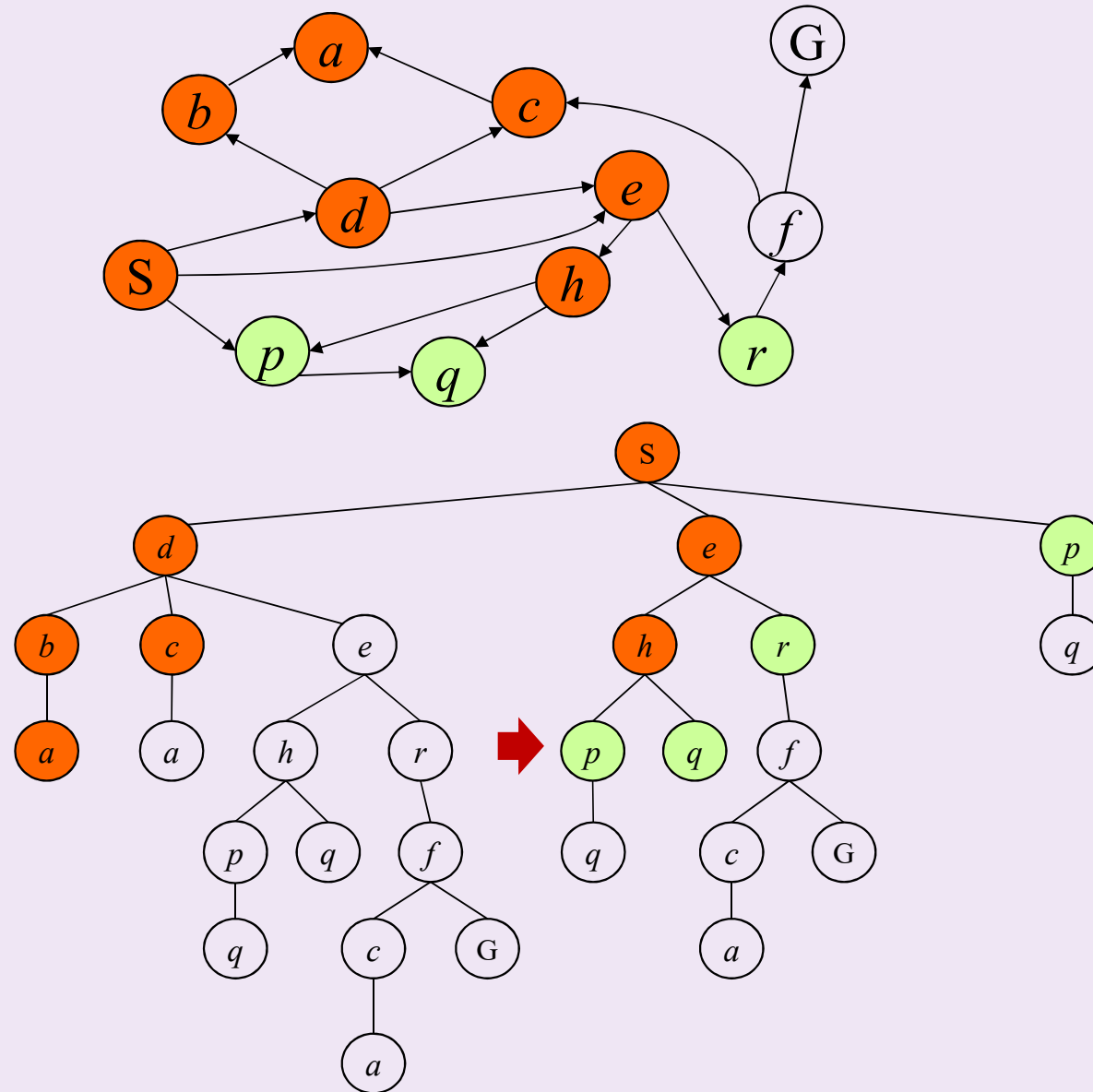
DFS (cont...)



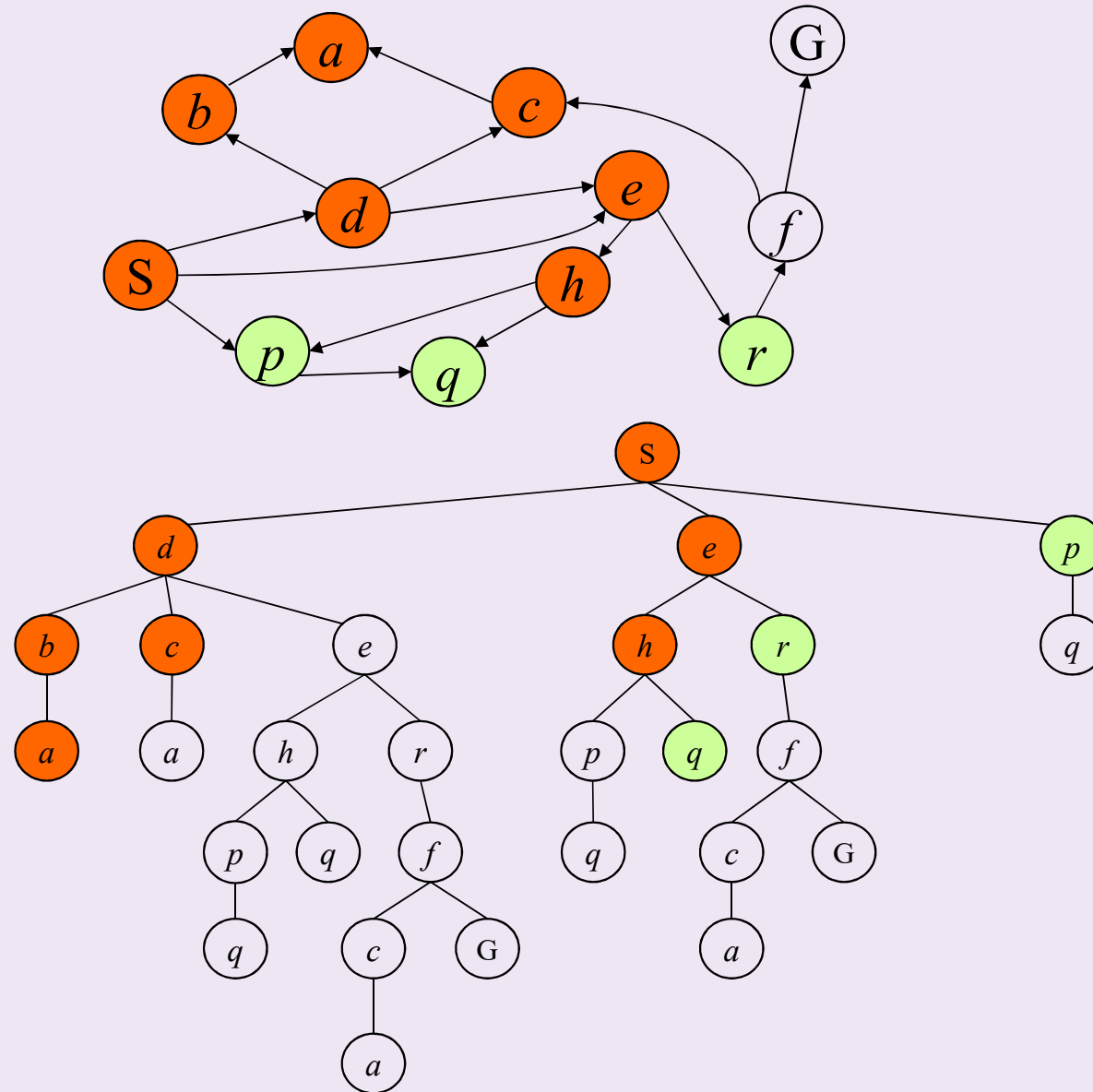
DFS (cont...)



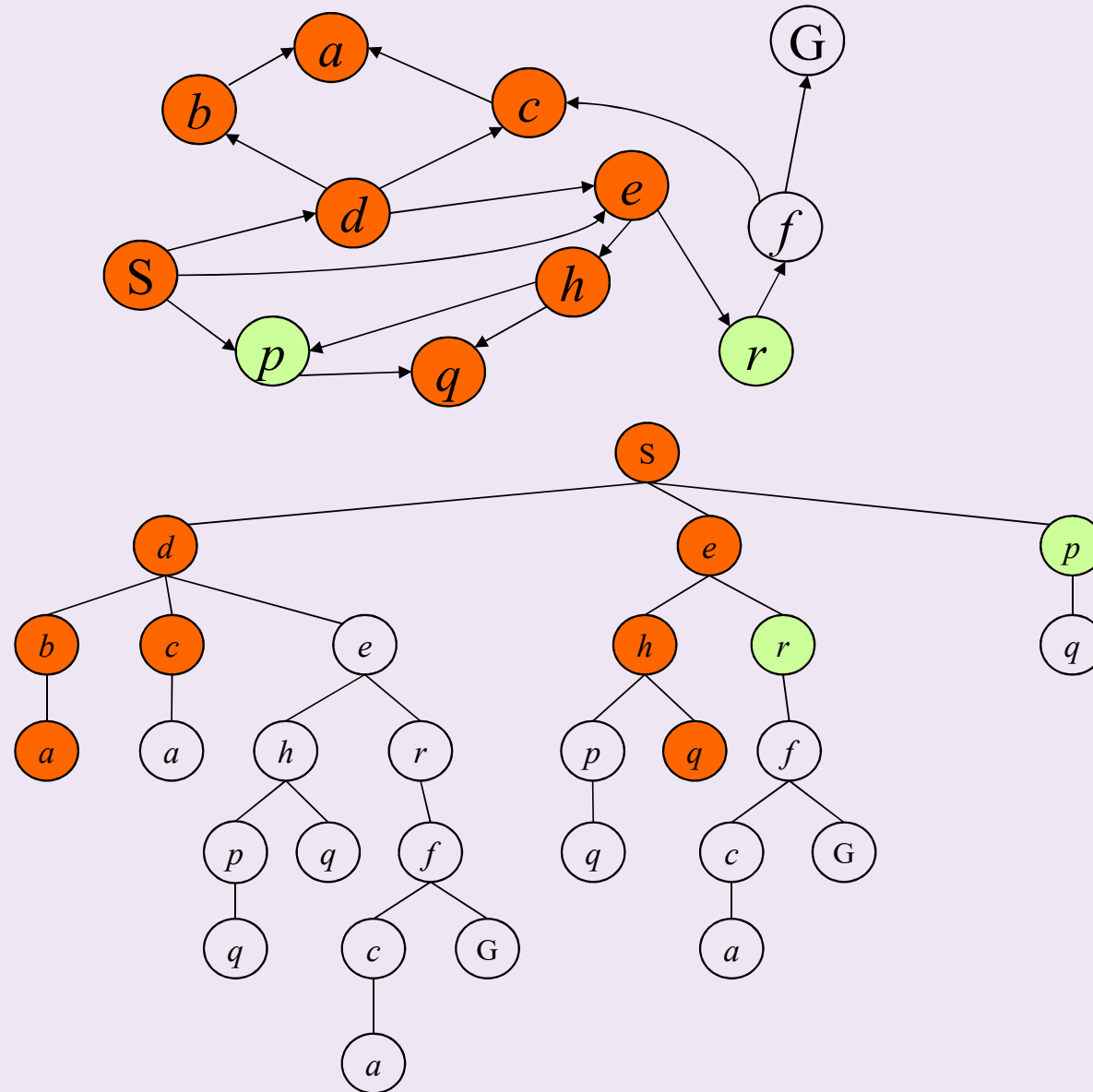
DFS (cont...)



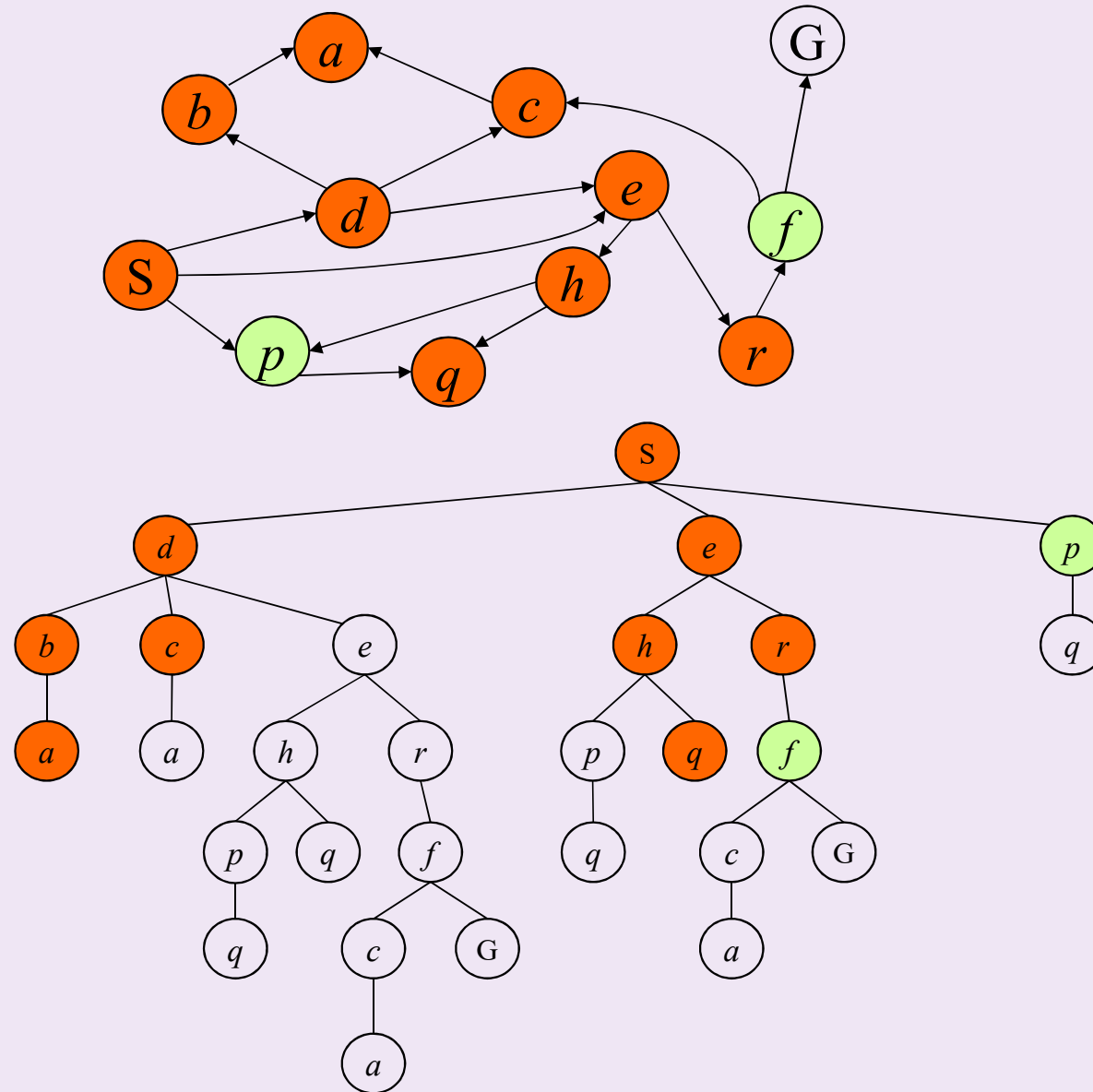
DFS (cont...)



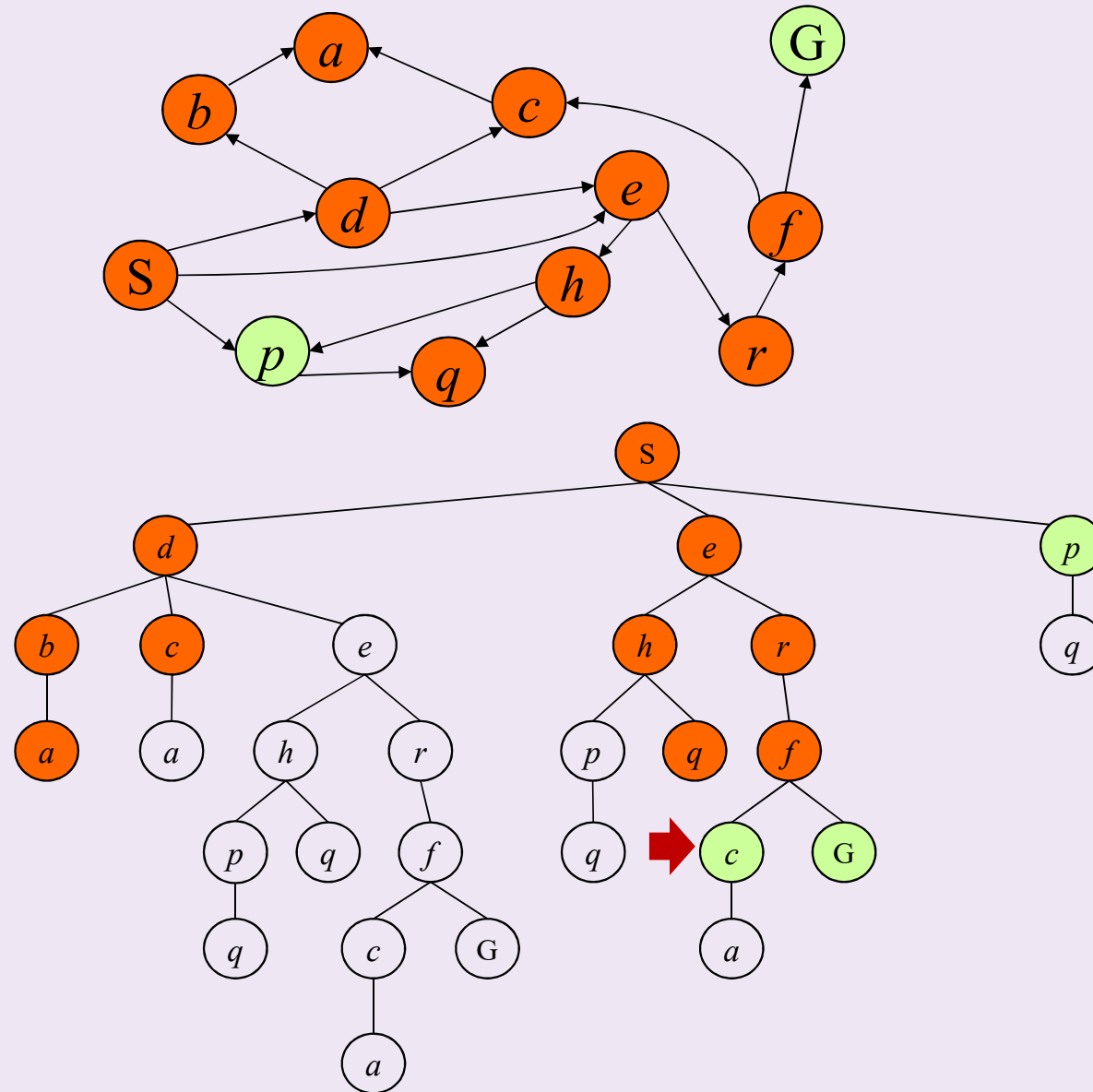
DFS (cont...)



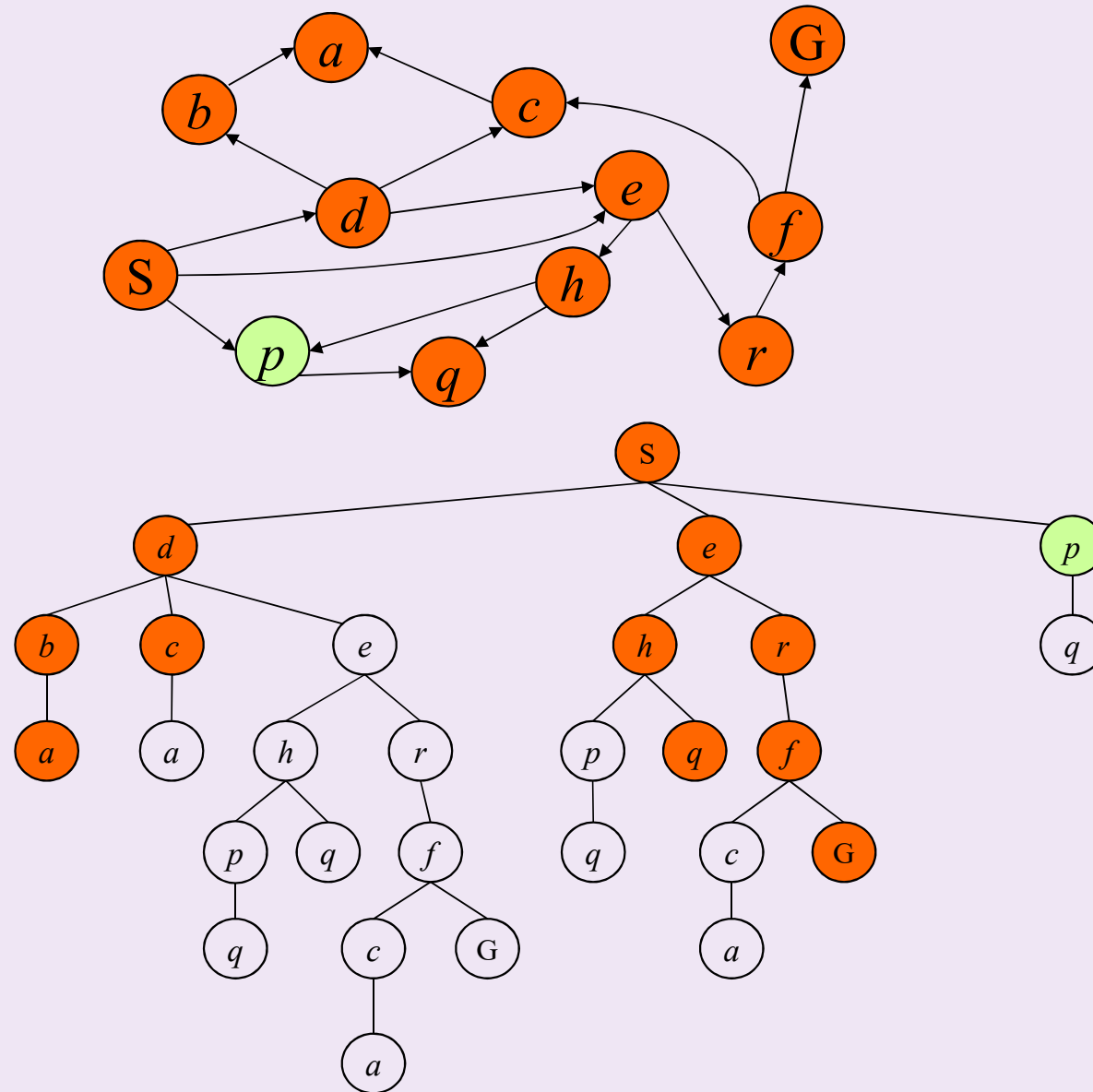
DFS (cont...)



DFS (cont...)



DFS (cont...)



Step	Pop / Expand	Children (in push order: reverse alpha)	Pushed (if not visited)	Stack after push (top = right)	Discovered / Visited so far	Notes
1	S	p, e, d	p, e, d	p e d	S	start
2	d	e, c, b	e (already), c, b	p e c b	S d	
3	b	a	a	p e c a	S d b	
4	a	—	—	p e c	S d b a	dead end
5	c	a	— (a visited)	p e	S d b a c	dead end
6	e	r, h	r, h	p r h	S d b a c e	
7	h	q, p	q (p already)	p r q	S d b a c e h	
8	q	—	—	p r	S d b a c e h q	dead end
9	r	f	f	p f	S d b a c e h q r	important branch opens
10	f	G, c	G (c already)	p G	S d b a c e h q r f	goal found
11	G	—	—	p	S d b a c e h q r f G	goal reached – can stop

BFS

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop the **newest** node s from **Frontier**

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If not **Reached** $[s']$:

If s' is a goal state, terminate

Put s' in **Frontier**

Reached $[s'] \leftarrow$ True

DFS

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop the **oldest** node s from **Frontier**

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If not **Reached** $[s']$:

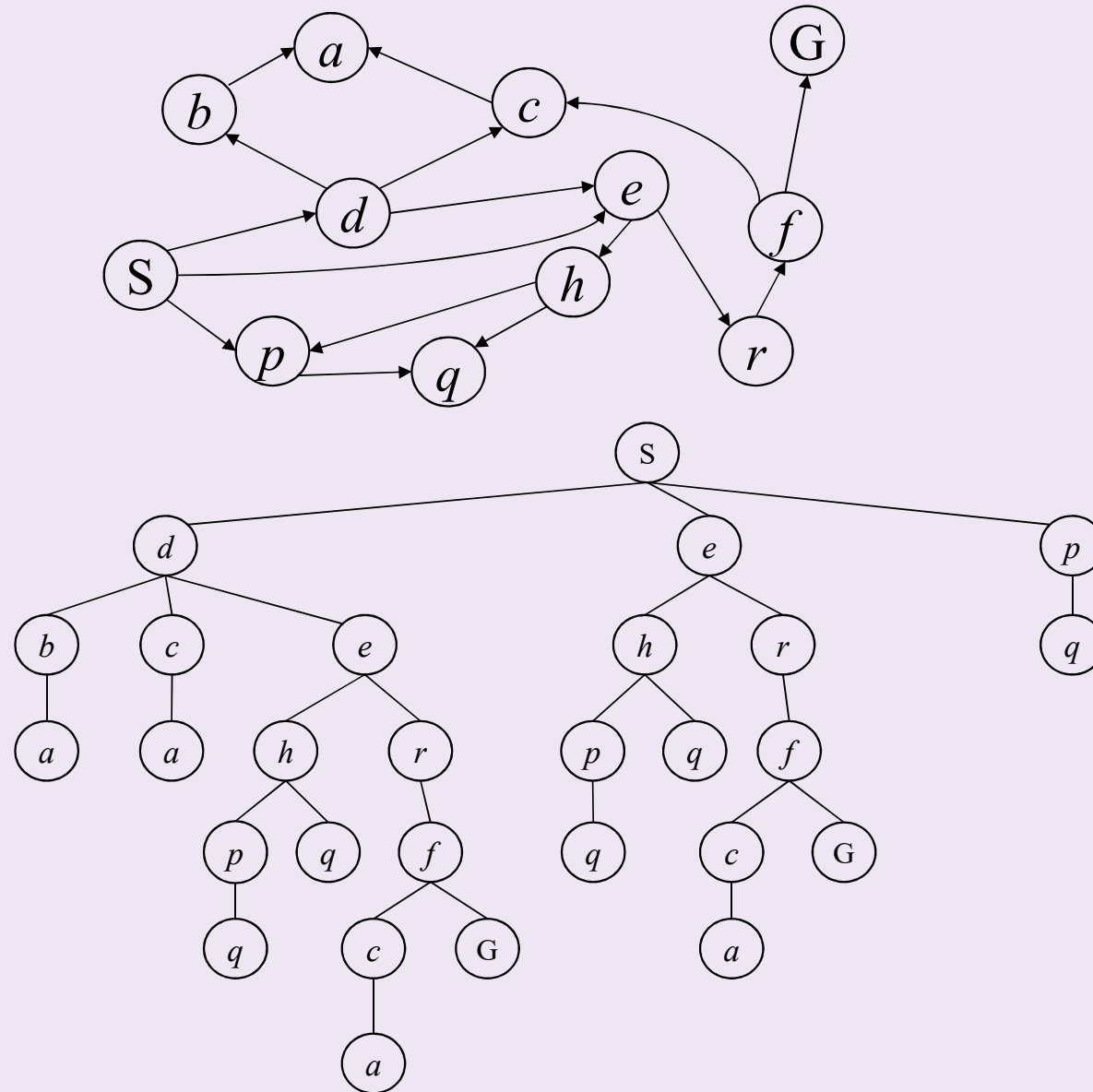
If s' is a goal state, terminate

Put s' in **Frontier**

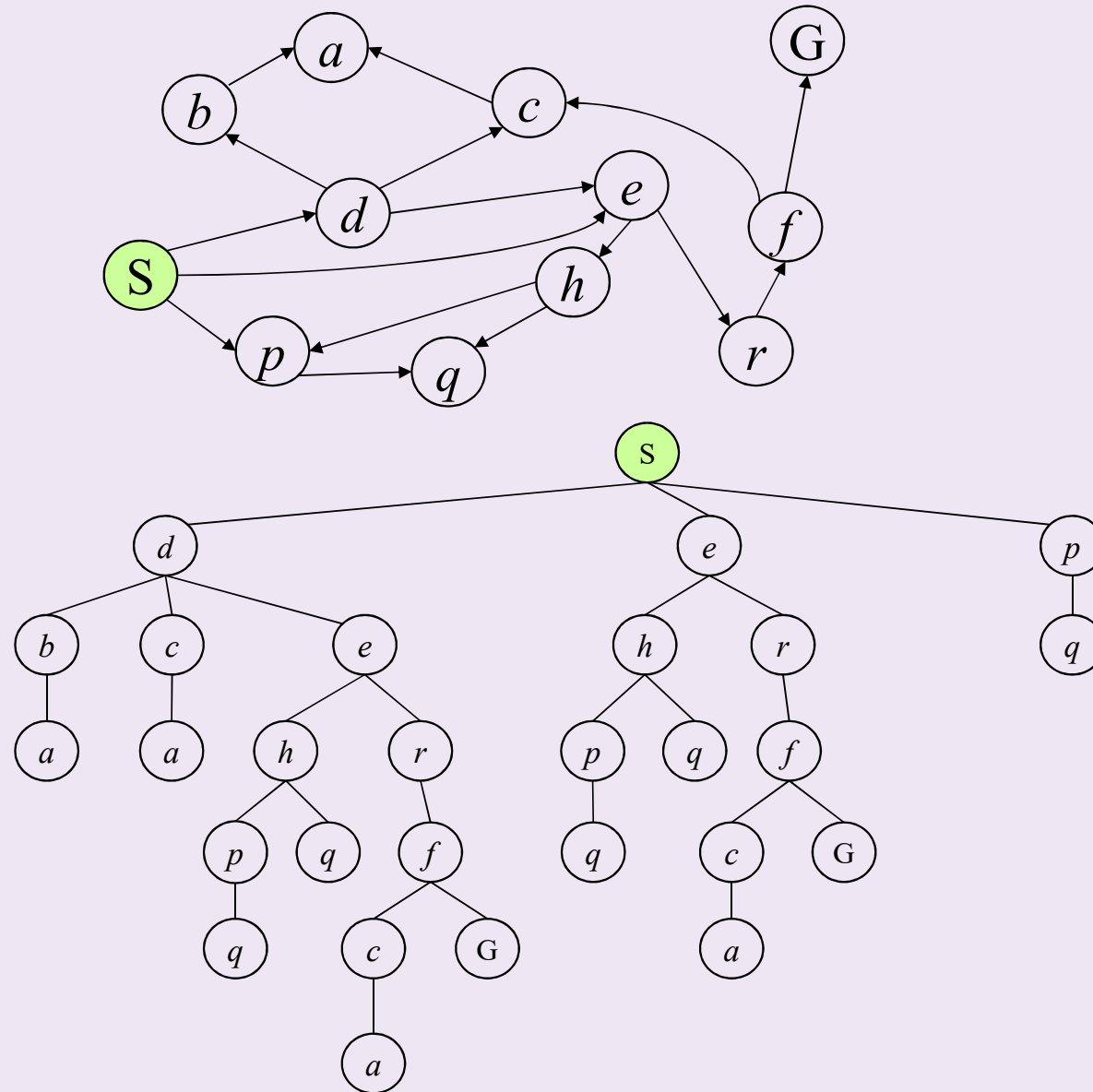
Reached $[s'] \leftarrow$ True

BFS

BFS

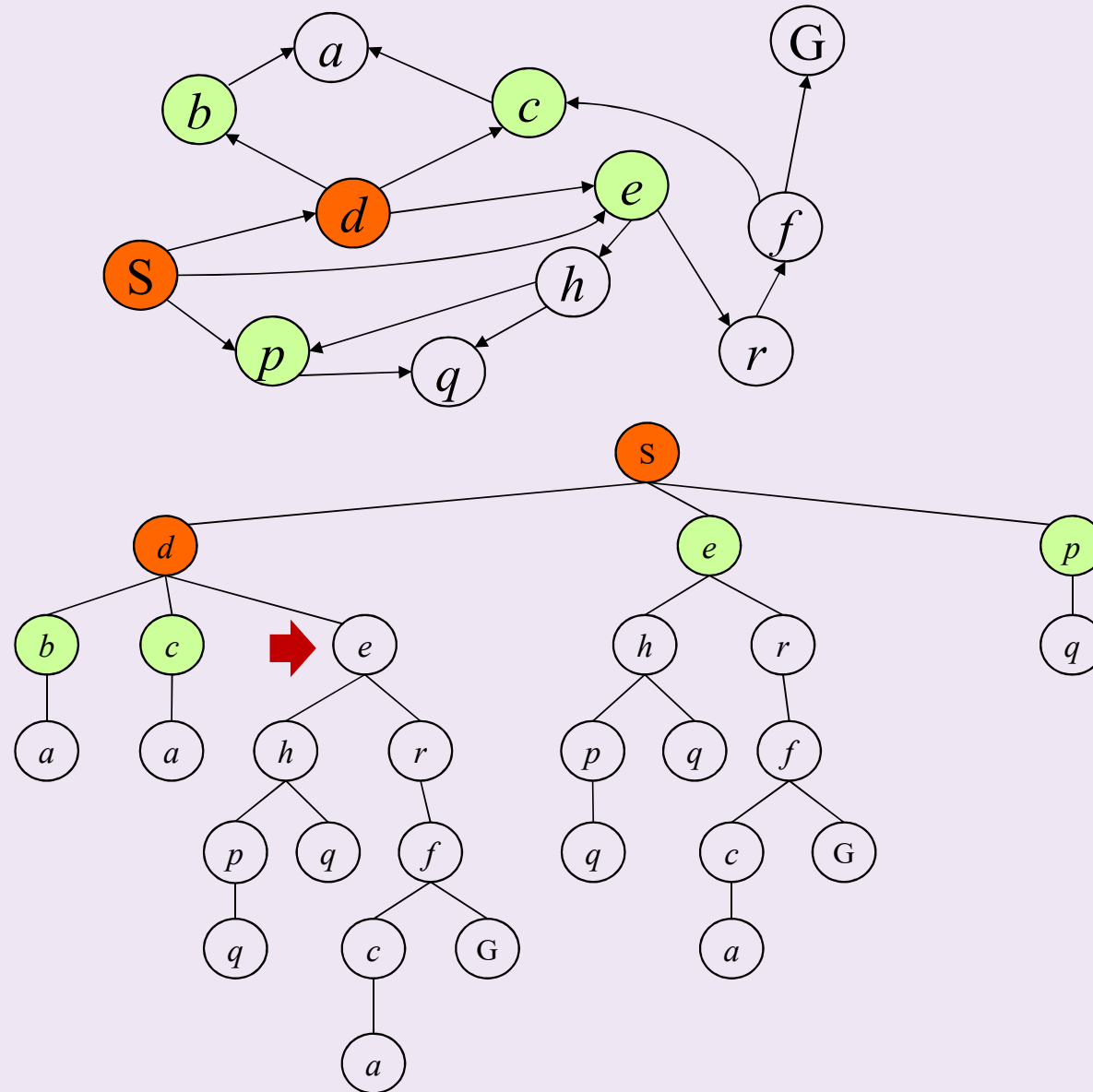


BFS (cont...)

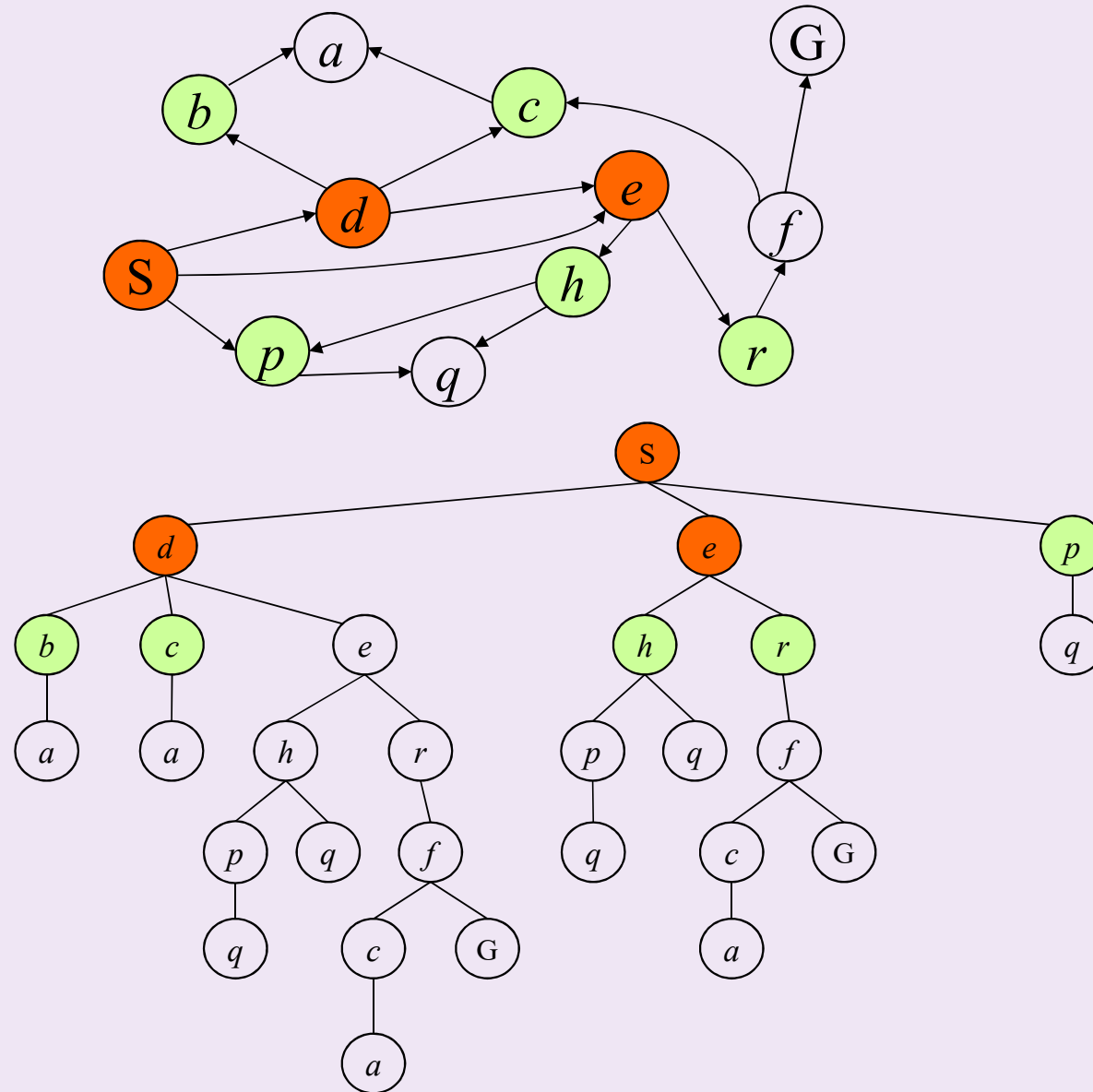




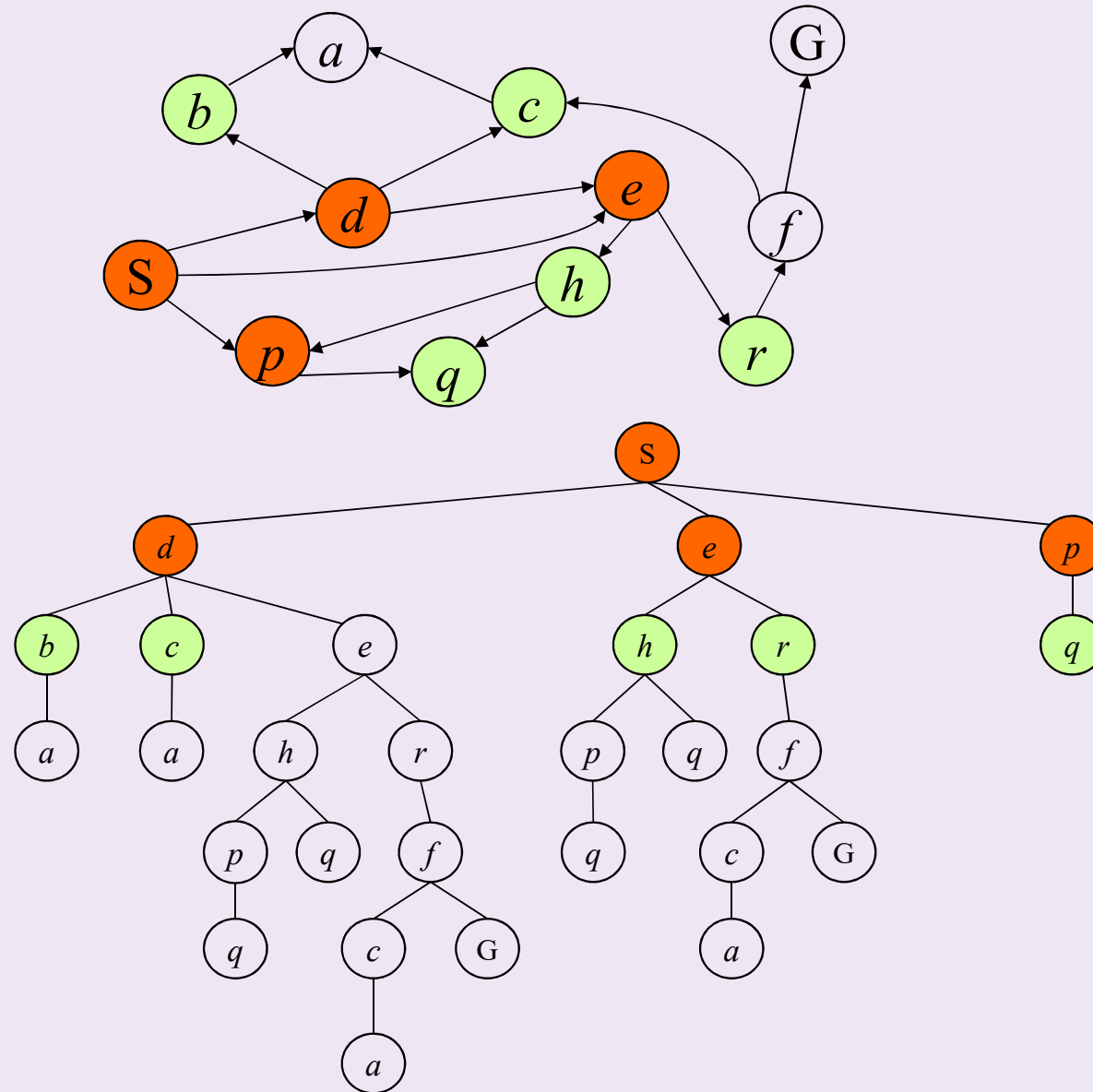
BFS (cont...)



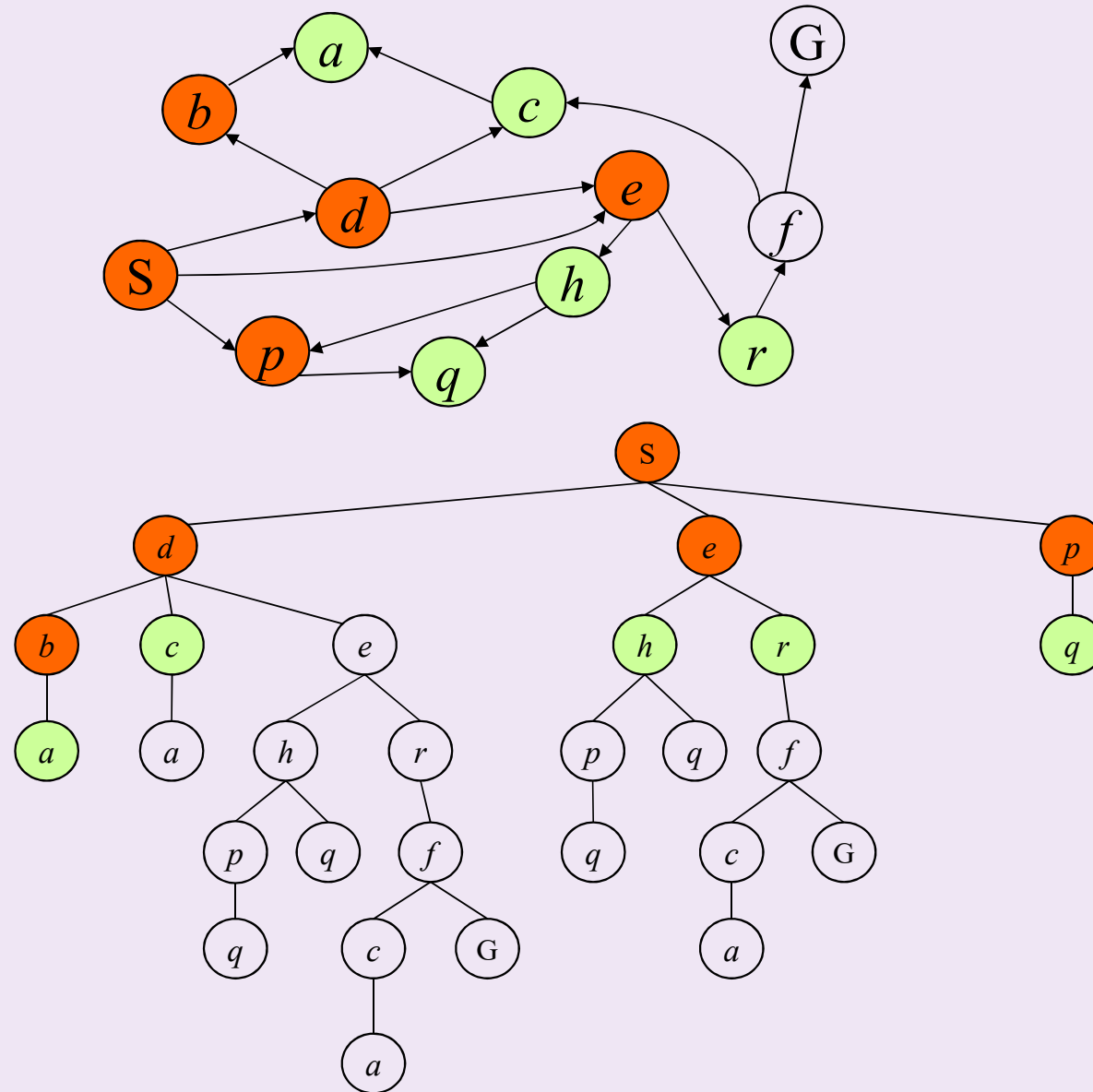
BFS (cont...)



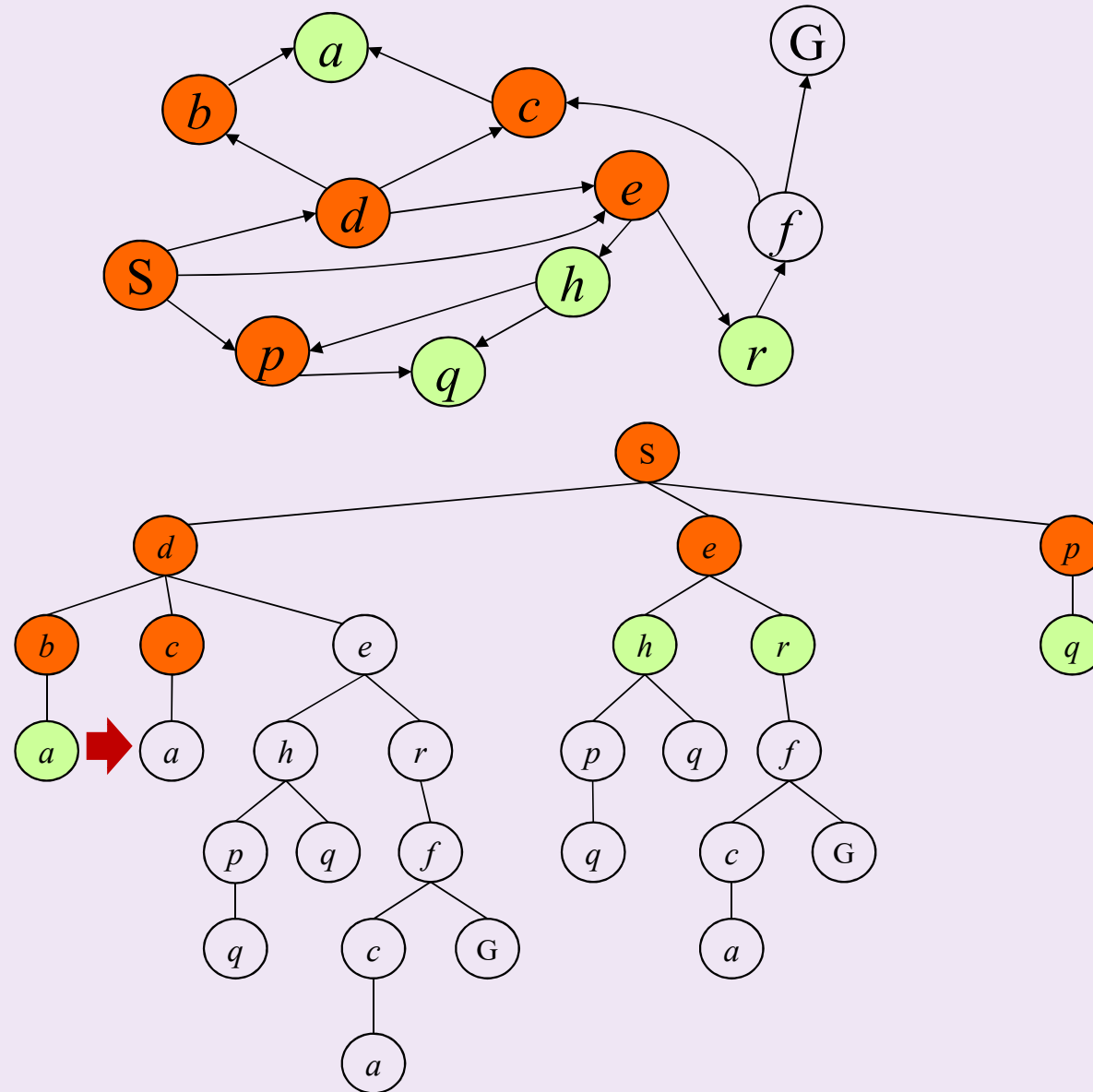
BFS (cont...)



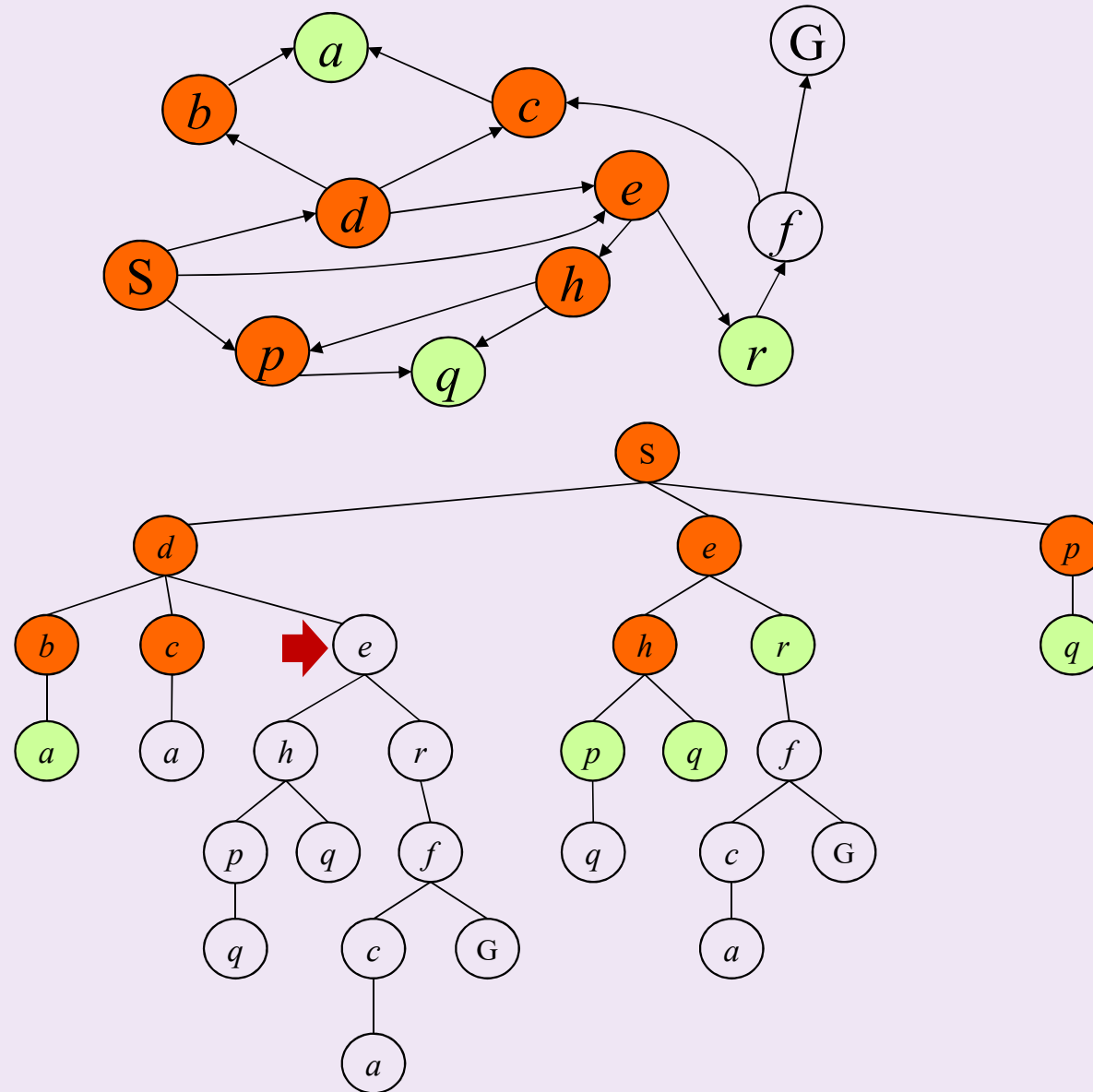
BFS (cont...)



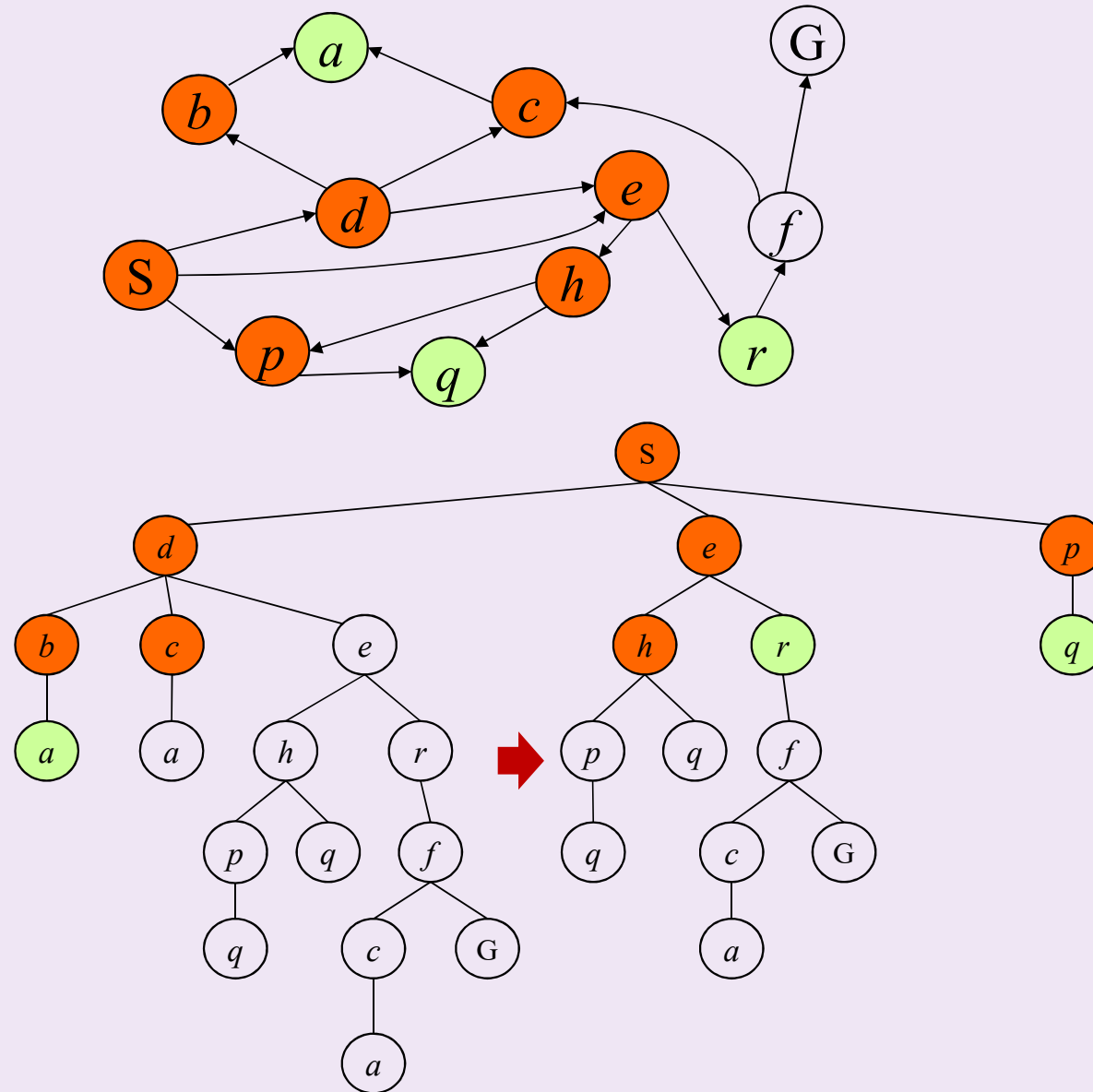
BFS (cont...)



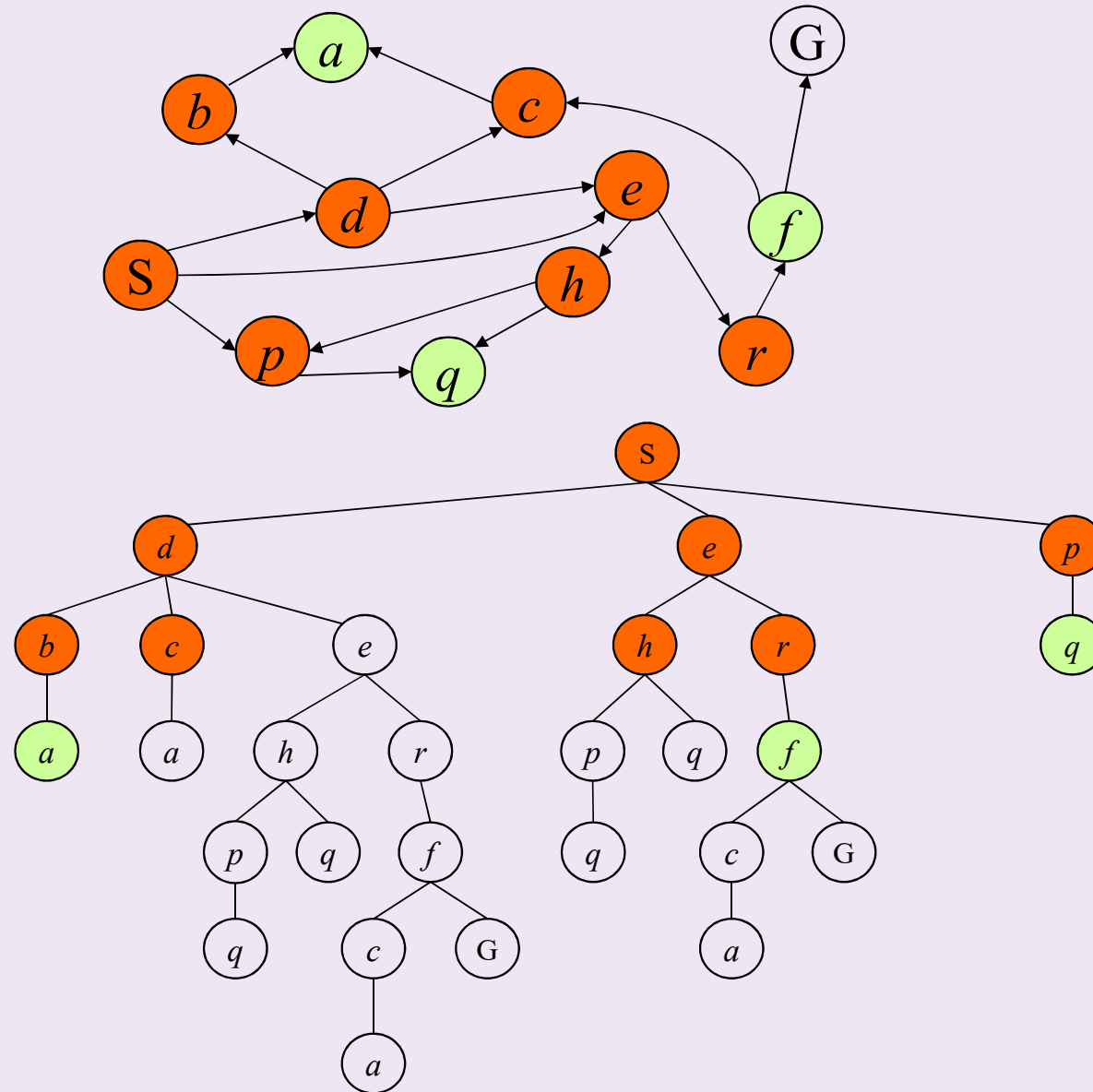
BFS (cont...)



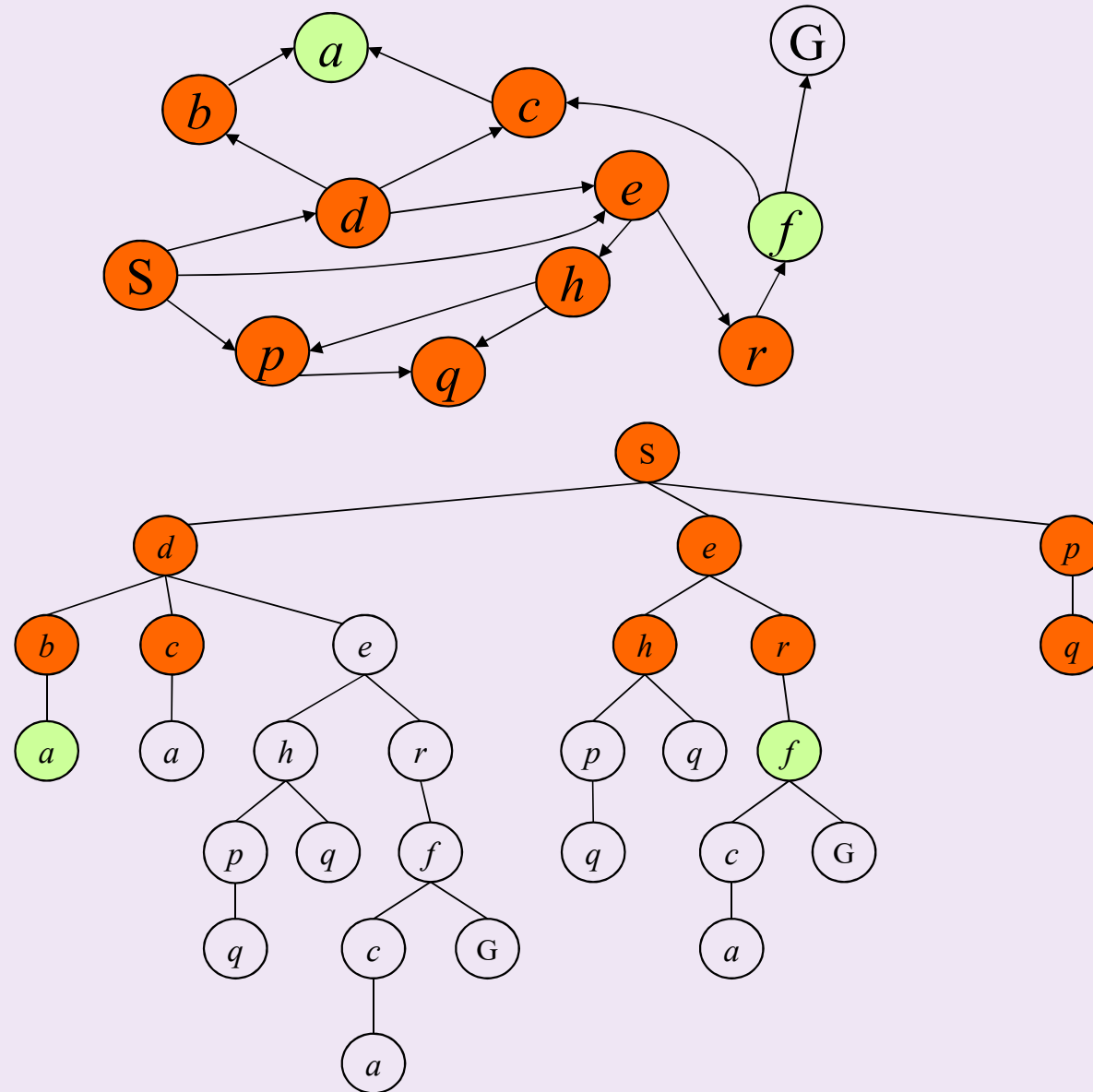
BFS (cont...)



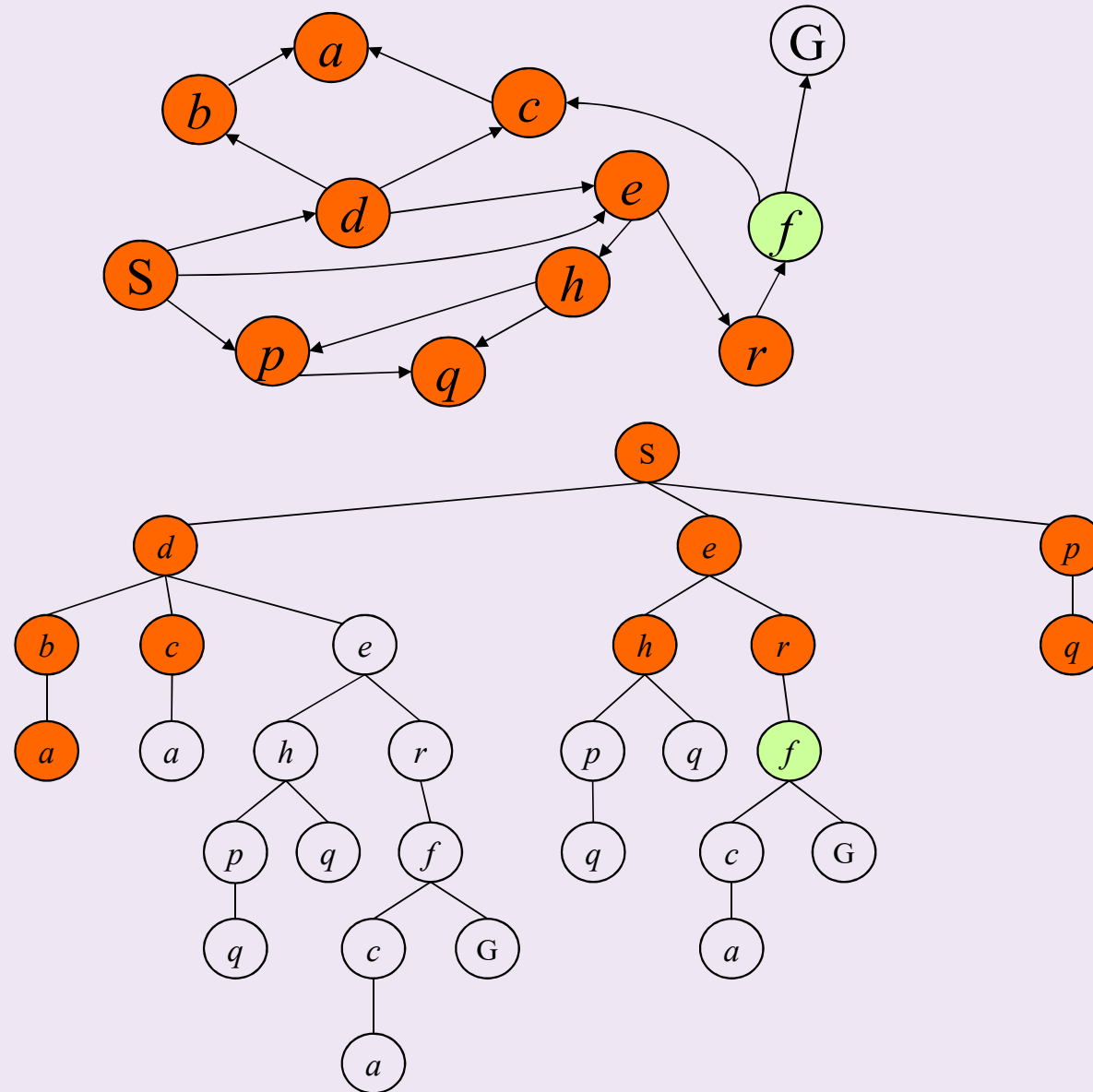
BFS (cont...)



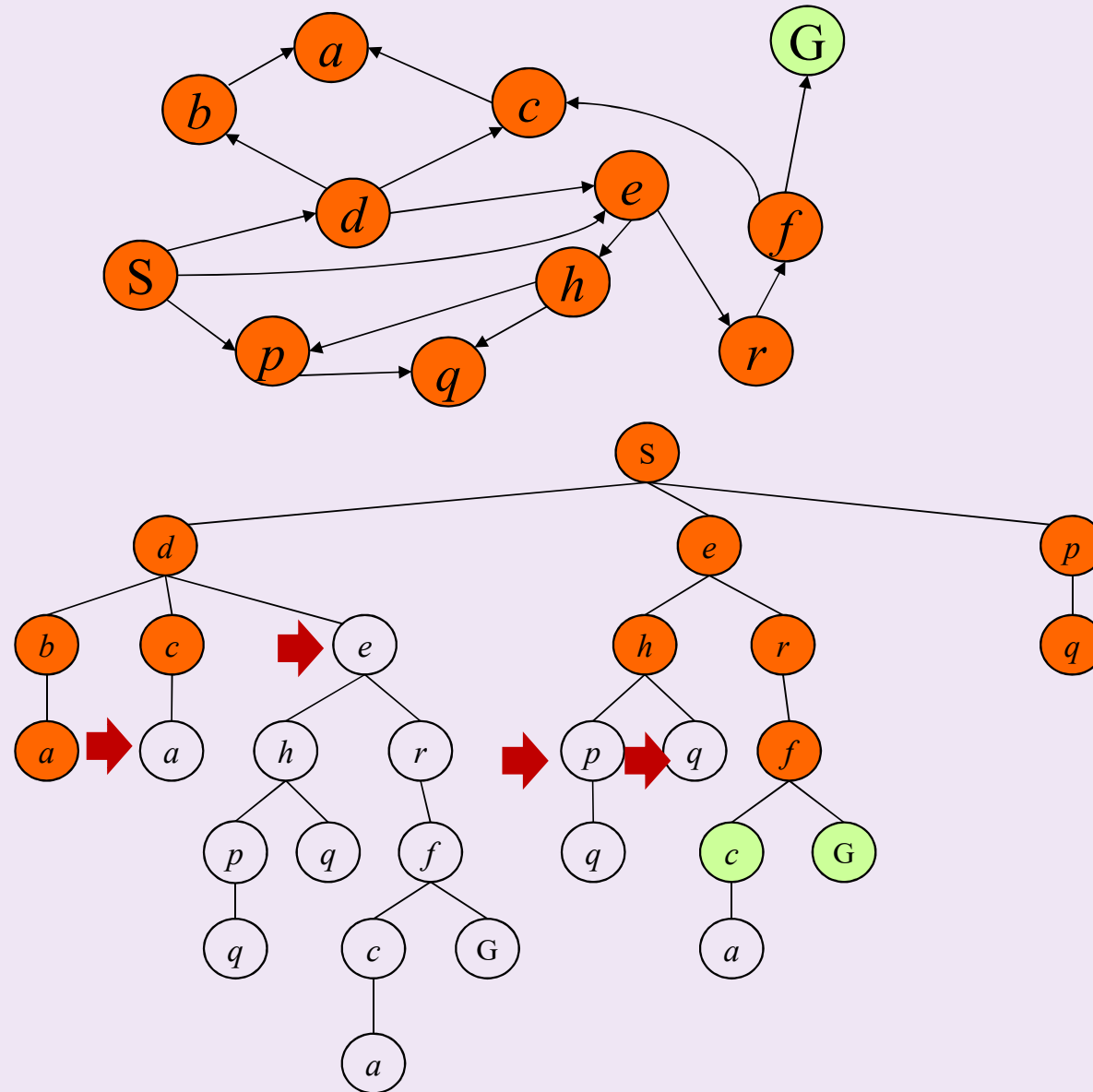
BFS (cont...)



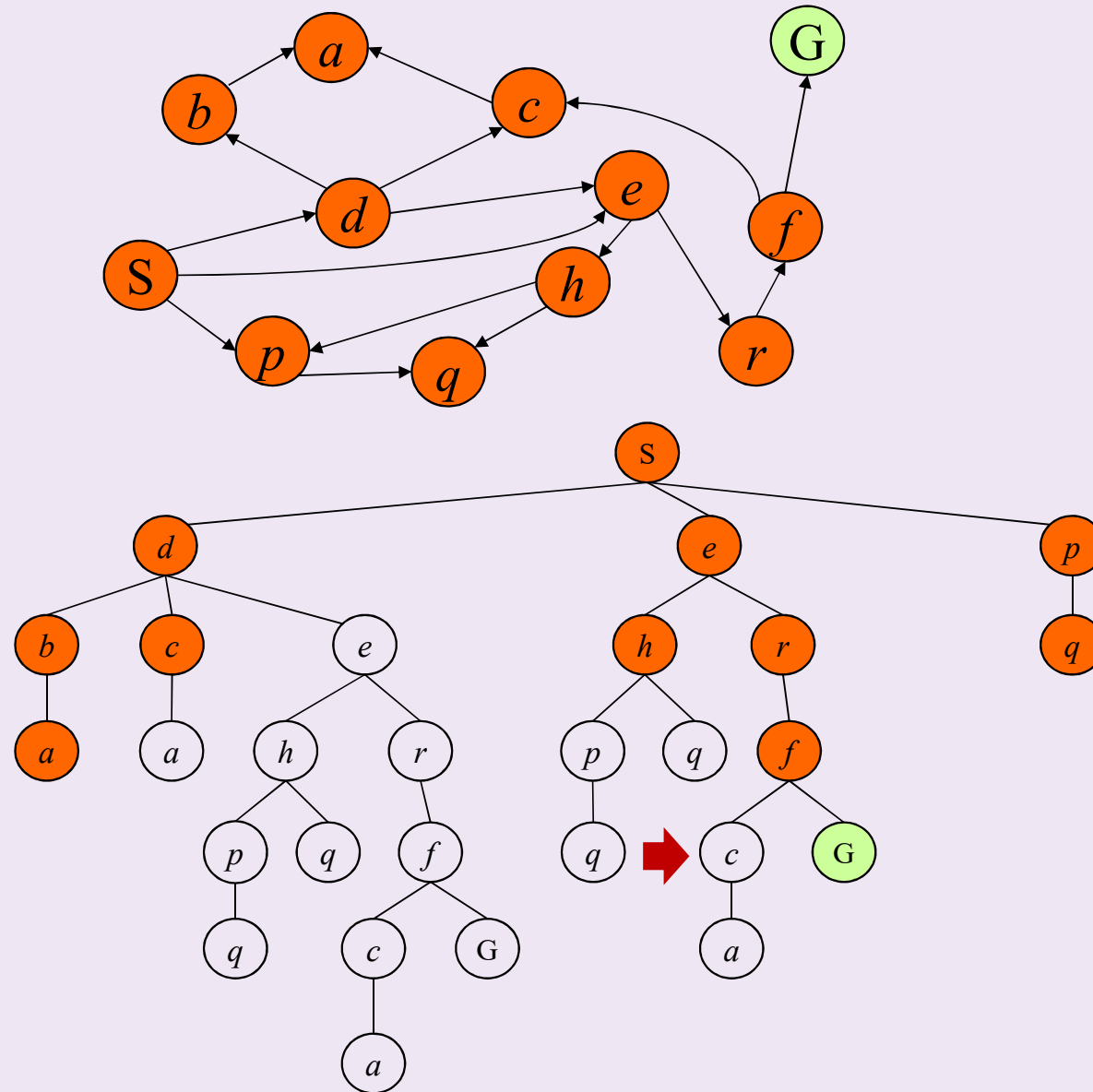
BFS (cont...)



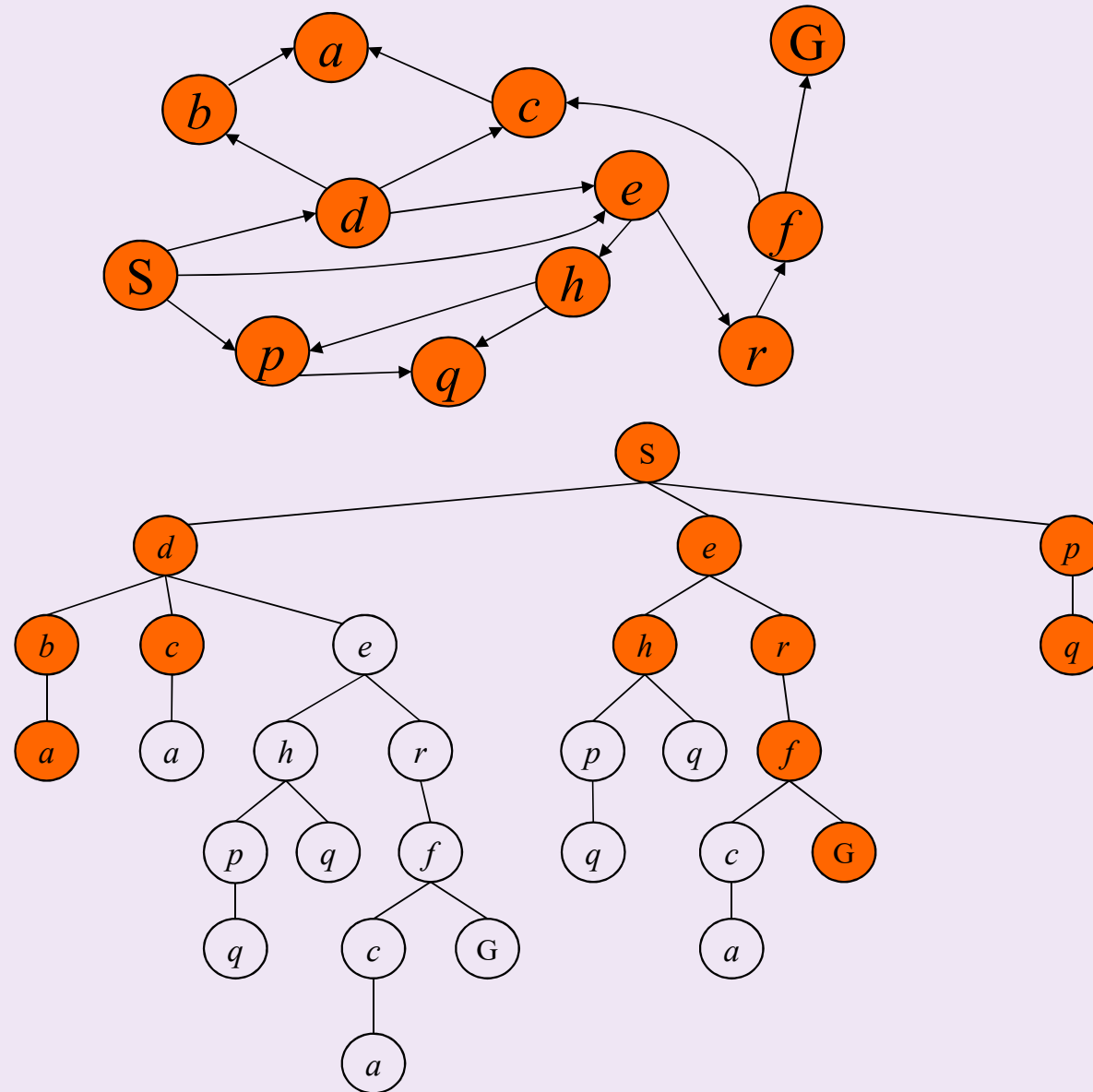
BFS (cont...)



BFS (cont...)



BFS (cont...)





Step	Dequeue / Expand	Neighbors (in alpha order)	Enqueued (if not yet discovered)	Queue after enqueue (front ← left)	Discovered / Visited so far	Level	Notes
1	S	d, e, p	d, e, p	d e p	S	0	start
2	d	b, c, e	b, c (e already)	e p b c	S d	1	
3	e	h, r	h, r	p b c h r	S d e	1	
4	p	q	q	b c h r q	S d e p	1	
5	b	a	a	c h r q a	S d e p b	2	
6	c	a	— (a already)	h r q a	S d e p b c	2	
7	h	p, q	— (both already)	r q a	S d e p b c h	2	dead-end branch
8	r	f	f	q a f	S d e p b c h r	2	key step — opens path to G
9	q	—	—	a f	S d e p b c h r q	2	dead end
10	a	—	—	f	S d e p b c h r q a	3	dead end
11	f	c, G	G (c already)	G	S d e p b c h r q a f	3	goal found
12	G	—	—	(empty)	S d e p b c h r q a f G	4	goal reached — stop

Search algorithm properties

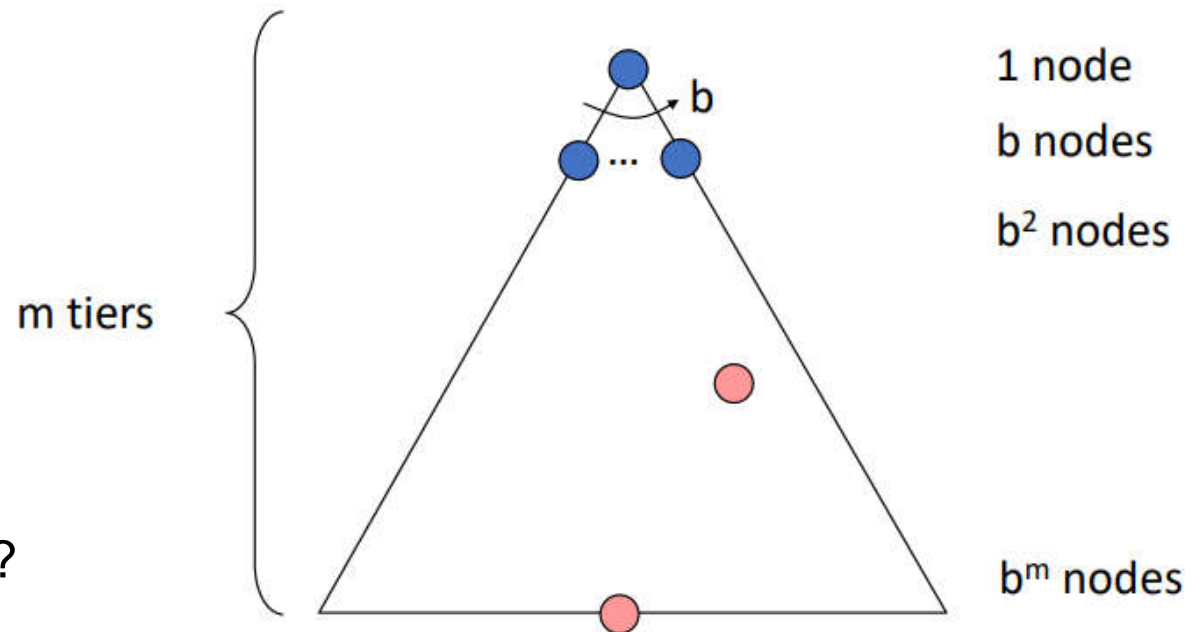
- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?

- Cartoon of search tree:

- b is the branching factor
- m is the maximum depth
- solutions/goals at various depths

- Number of nodes in entire tree?

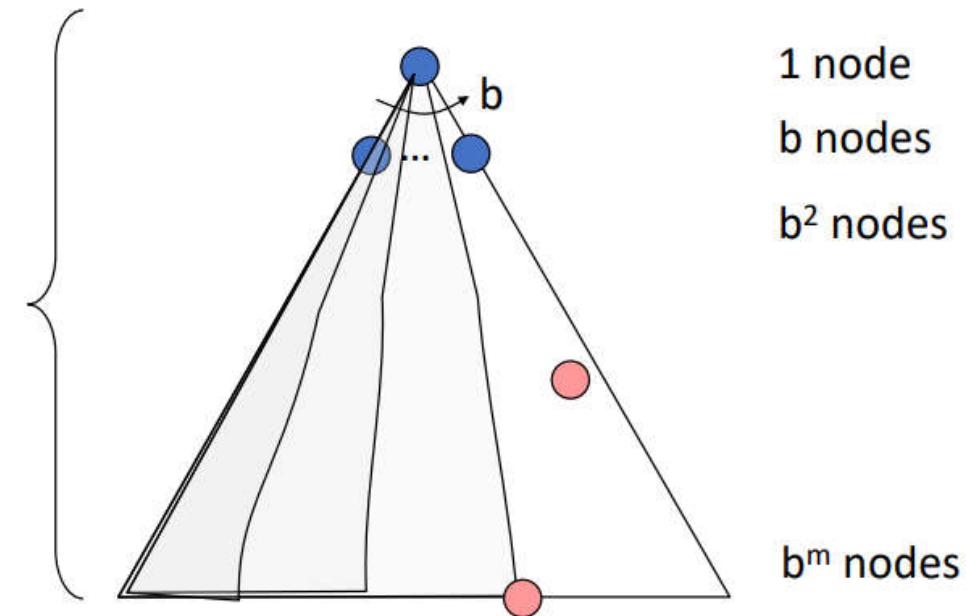
$$- 1 + b + b^2 + \dots + b^m = O(b^m)$$



DFS properties

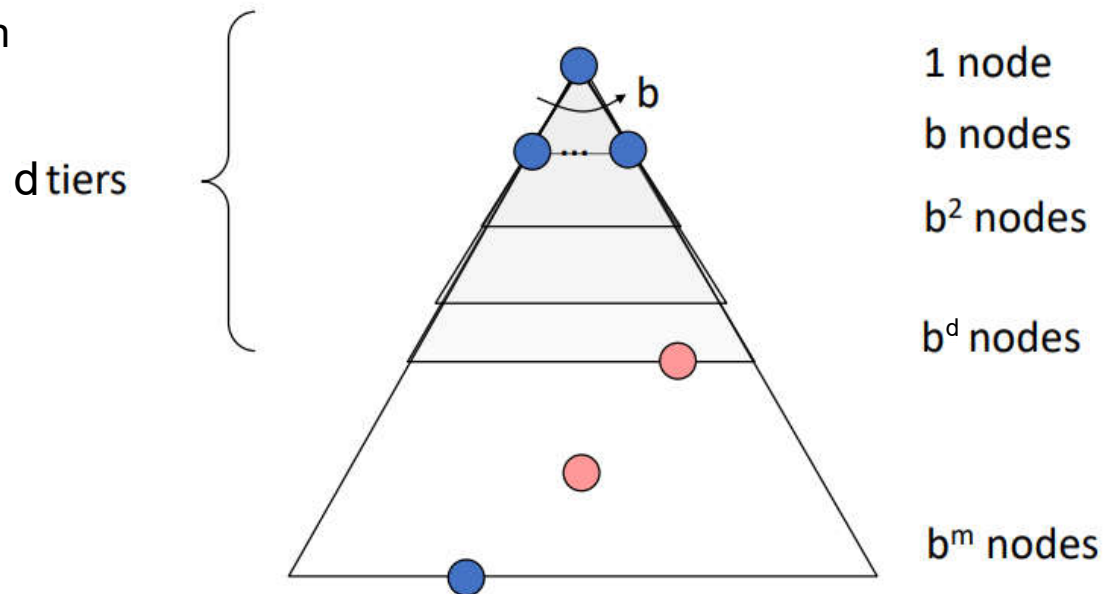
- What nodes DFS expand?
 - Some left prefix of the tree.
 - Could process the whole tree!
 - If m is finite, takes time $O(b^m)$
- How much space does the frontier take?
 - Only has siblings on path to root, so $O(b \cdot m)$
- Is it complete?
 - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
 - No, it finds the “leftmost” solution, regardless of depth or cost

m tiers



BFS properties

- What nodes does BFS expand?
 - Processes all nodes above shallowest solution
 - Let depth of shallowest solution be d
 - Search takes time $O(b^d)$
- How much space does the frontier take?
 - Has roughly the last tier, so $O(b^d)$
- Is it complete?
 - d must be finite if a solution exists, so yes!
- Is it optimal?
 - Only if costs are all 1 (more on costs later)



DFS vs. BFS

- When will BFS outperform DFS?
- When will DFS outperform BFS?



Which one is DFS/BFS?



DFS

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop the **newest** node s from **Frontier**

For all action a :

$s' \leftarrow \text{succ}(s, a)$

If s' is not an ancestor of s :

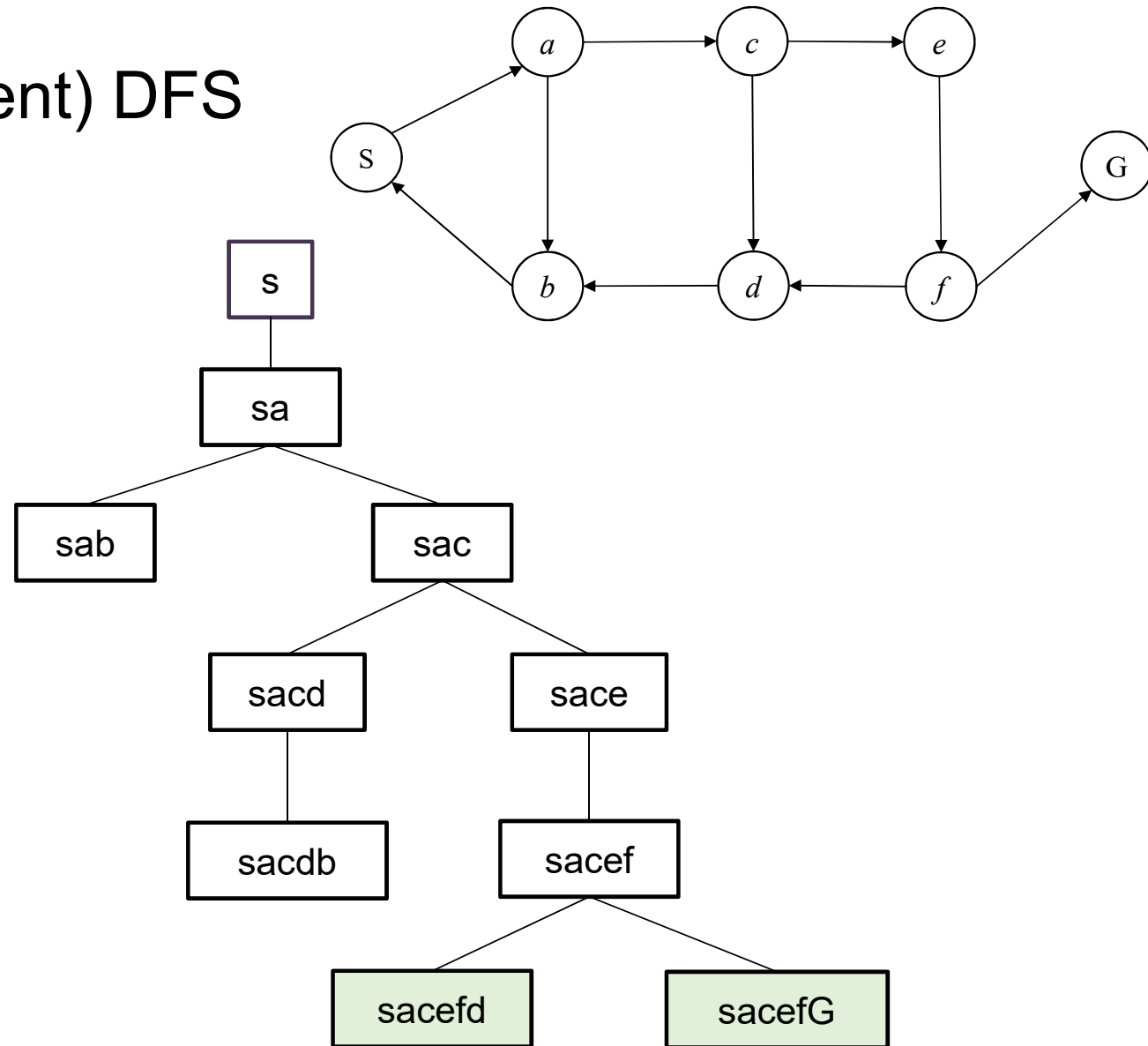
If s' is a goal state, terminate

Push s' to **Frontier**

A Memory Efficient Version of DFS for **Cyclic** Graphs

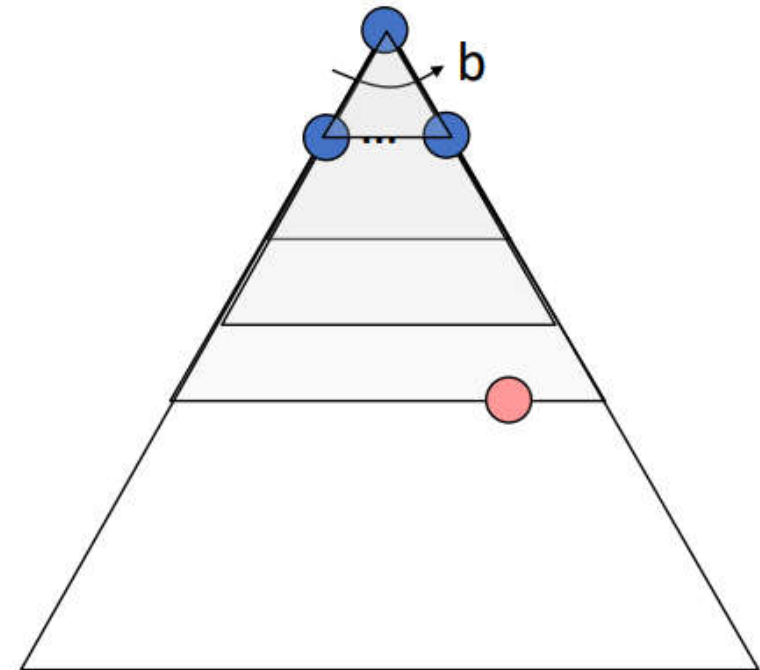
(Memory Efficient) DFS

handling cycles



Iterative deepening search (IDS)

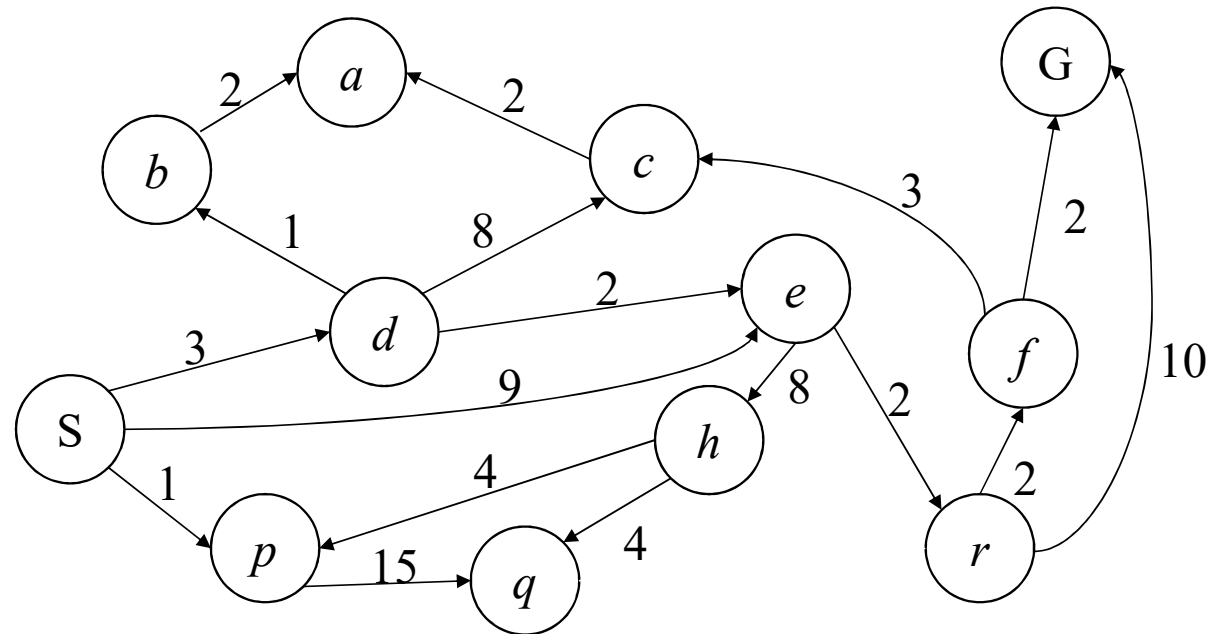
- **Idea:** get DFS's space advantage with BFS's time / shallow-solution advantages
 - Run a DFS with depth limit 1. If no solution...
 - Run a DFS with depth limit 2. If no solution...
 - Run a DFS with depth limit 3.
- Isn't that wastefully redundant?
 - Generally, most work happens in the lowest level searched, so not so bad



Time & space complexity

	Time	Space
DFS (memory-efficient version)	$m \cdot b^m$	$b \cdot m^2$
BFS	b^d	b^d
IDS	$O\left(\sum_{i=1}^d i \cdot b^i\right) \leq$ $O\left(d \cdot \sum_{i=1}^d b^i\right) \leq$ $O(d \cdot b^d)$	$b \cdot d^2$

Cross-sensitive search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path. We will now cover
a similar algorithm which does find the least-cost path.

Uniform cost search (UCS) (Dijkstra)

Frontier $\leftarrow$ { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

 If s is a goal state, then terminate

 For all action a :

$s' \leftarrow \text{succ}(s, a)$

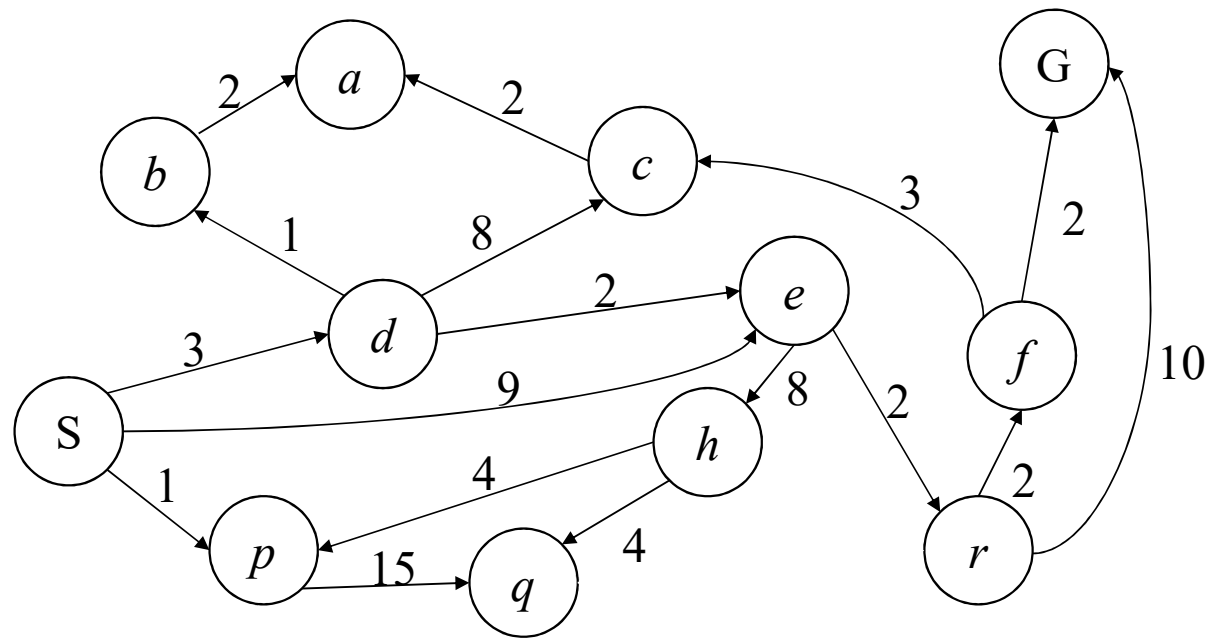
 If not **Reached**[s']:

 Put s' in **Frontier**

Reached[s'] $\leftarrow$ True

$g(s') \leftarrow \min \{ g(s'), g(s) + \text{cost}(s, a) \}$

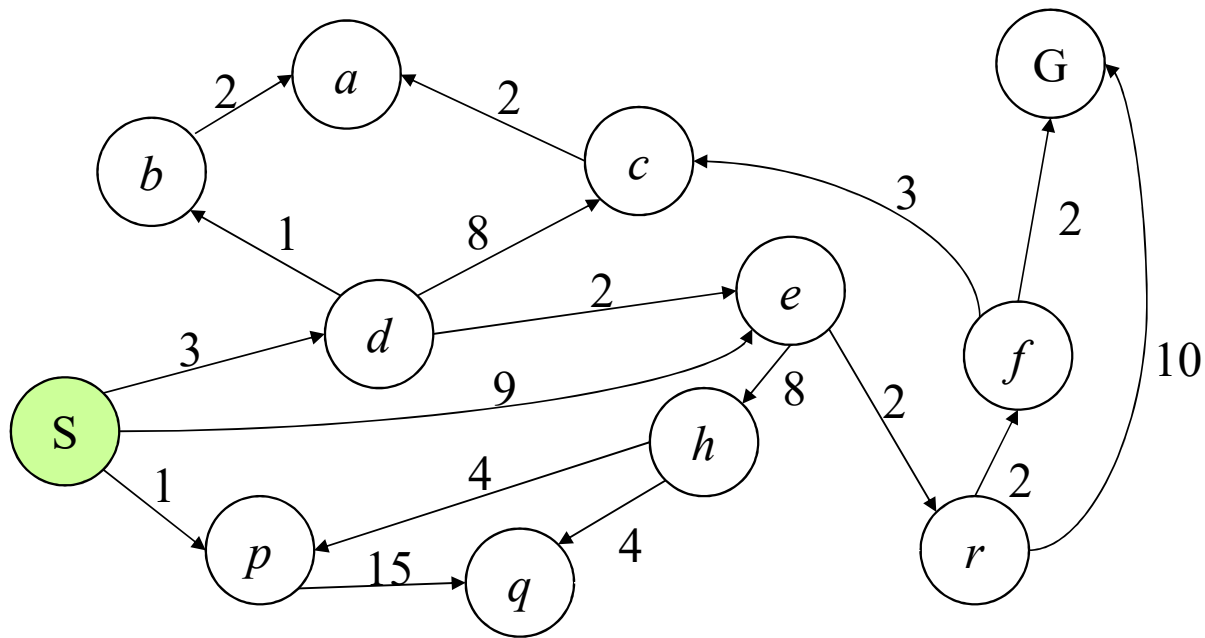
UCS



x	$g(x)$
S	
a	
b	
c	
d	
e	
f	
h	
p	
q	
r	
G	



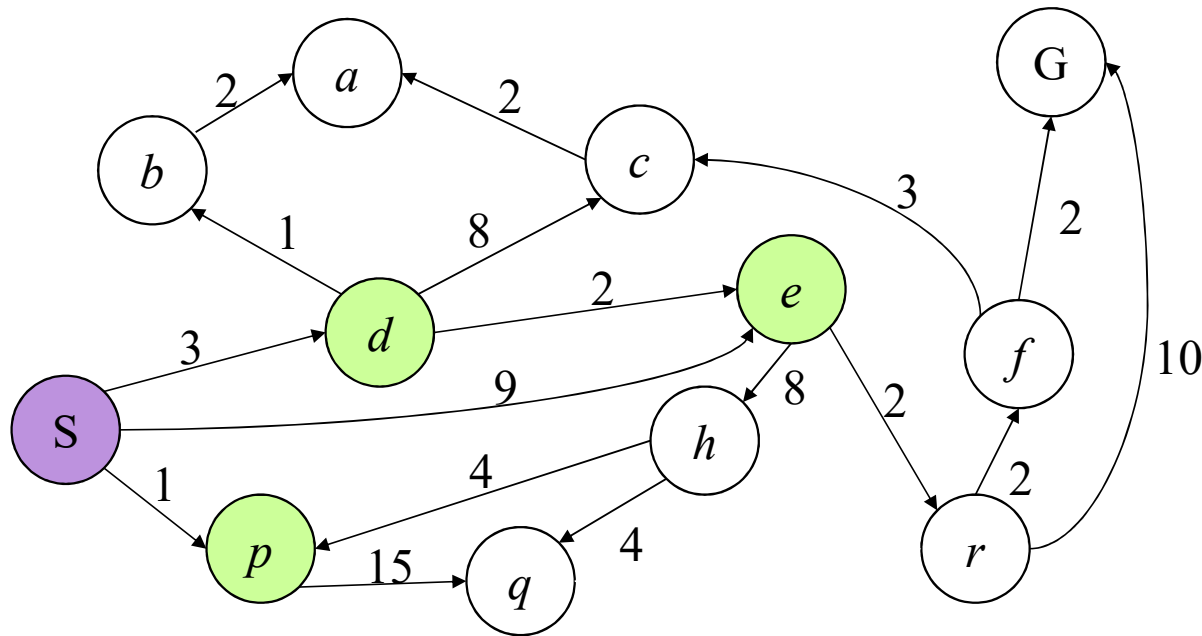
UCS (cont...)



x	$g(x)$
S	0
a	∞
b	∞
c	∞
d	∞
e	∞
f	∞
h	∞
p	∞
q	∞
r	∞
G	∞



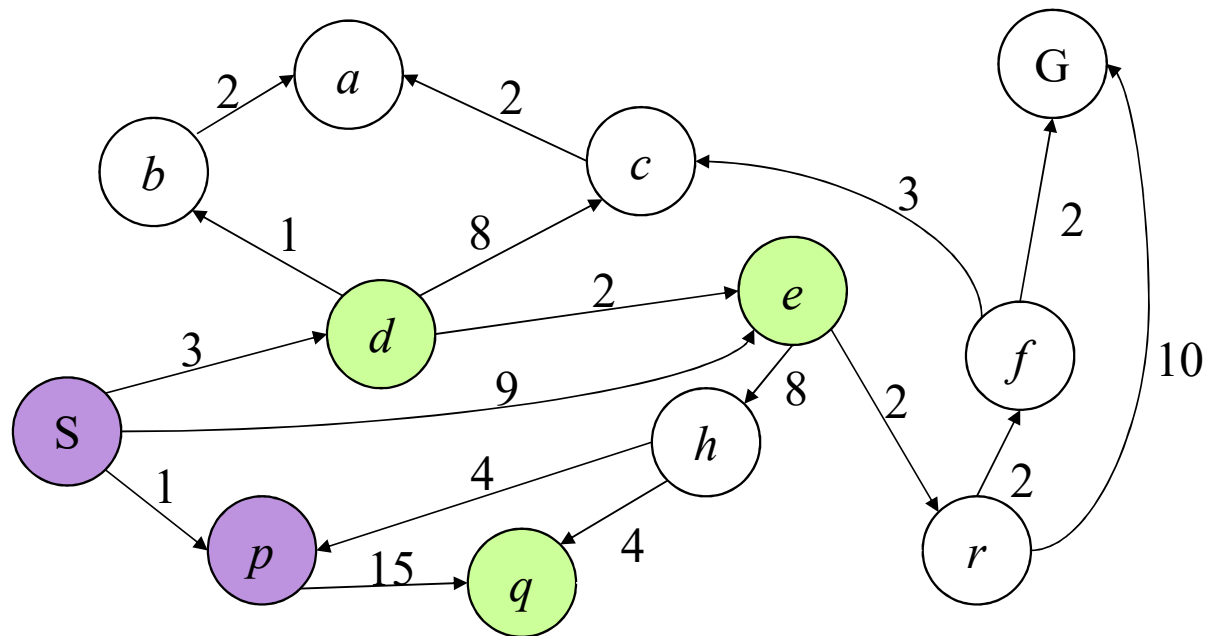
UCS (cont...)



x	$g(x)$
S	0
a	∞
b	∞
c	∞
d	3
e	9
f	∞
h	∞
p	1
q	∞
r	∞
G	∞



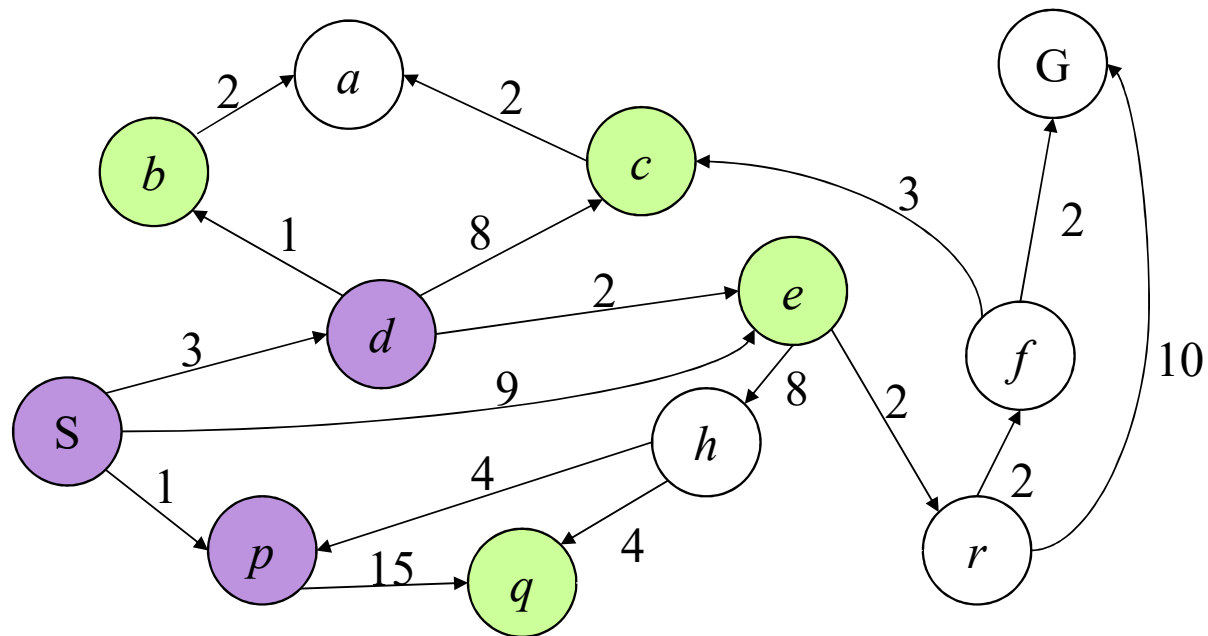
UCS (cont...)



x	$g(x)$
S	0
a	∞
b	∞
c	∞
d	3
e	9
f	∞
h	∞
p	1
q	16
r	∞
G	∞



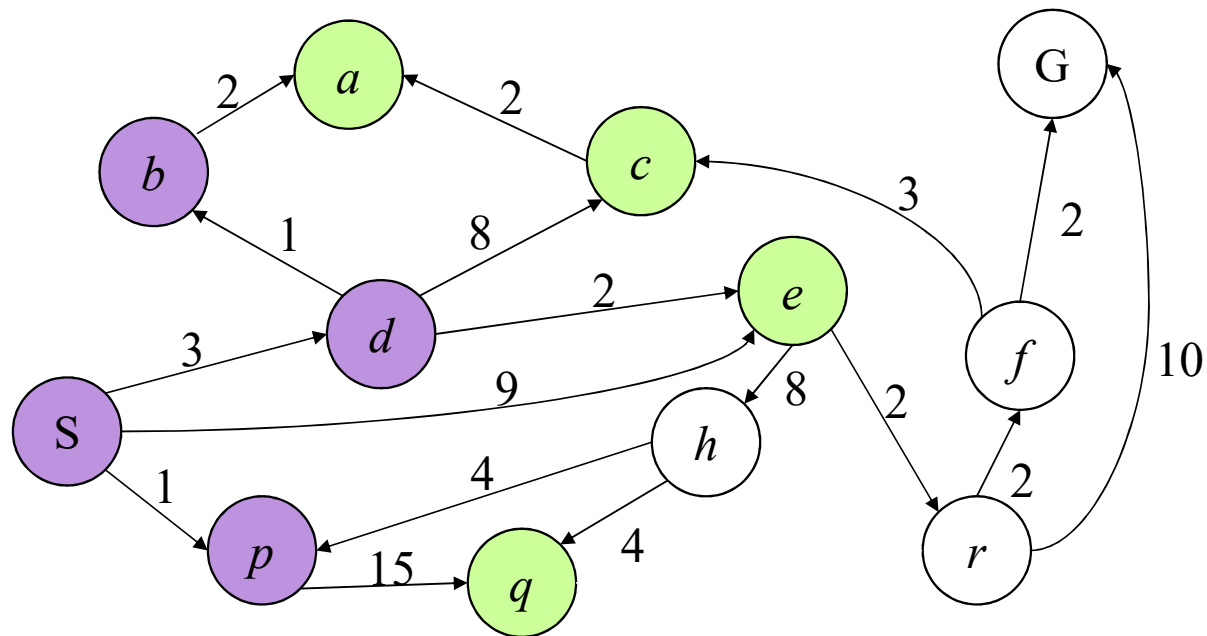
UCS (cont...)



x	$g(x)$
S	0
a	∞
b	4
c	11
d	3
e	5
f	∞
h	∞
p	1
q	16
r	∞
G	∞



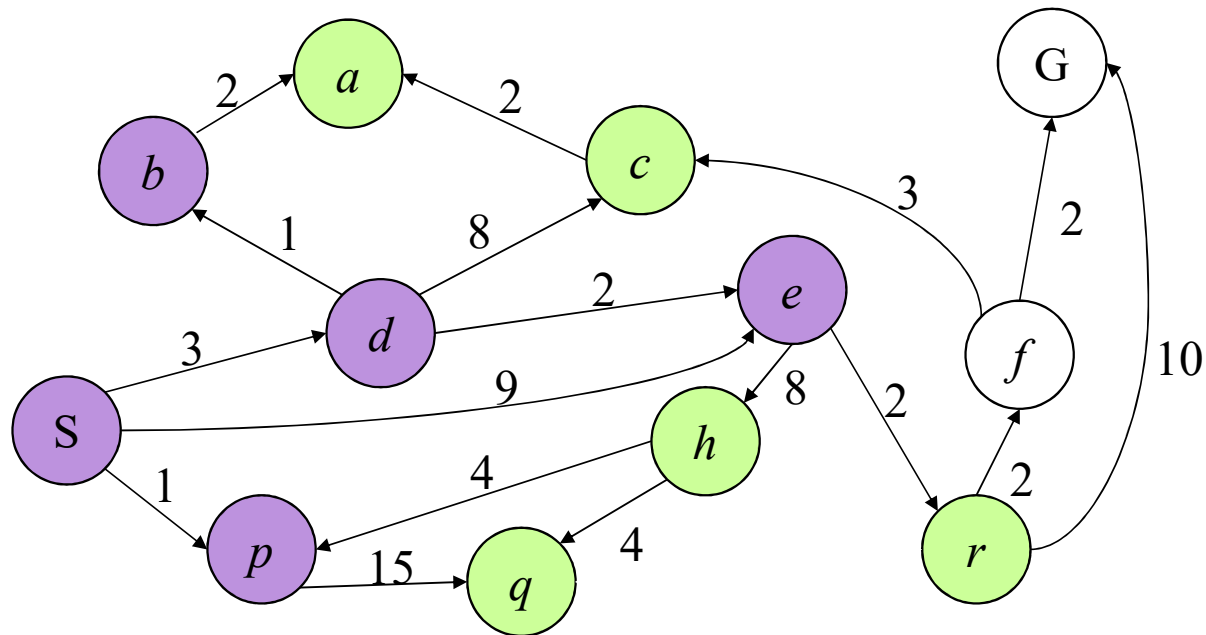
UCS (cont...)



x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	∞
h	∞
p	1
q	16
r	∞
G	∞



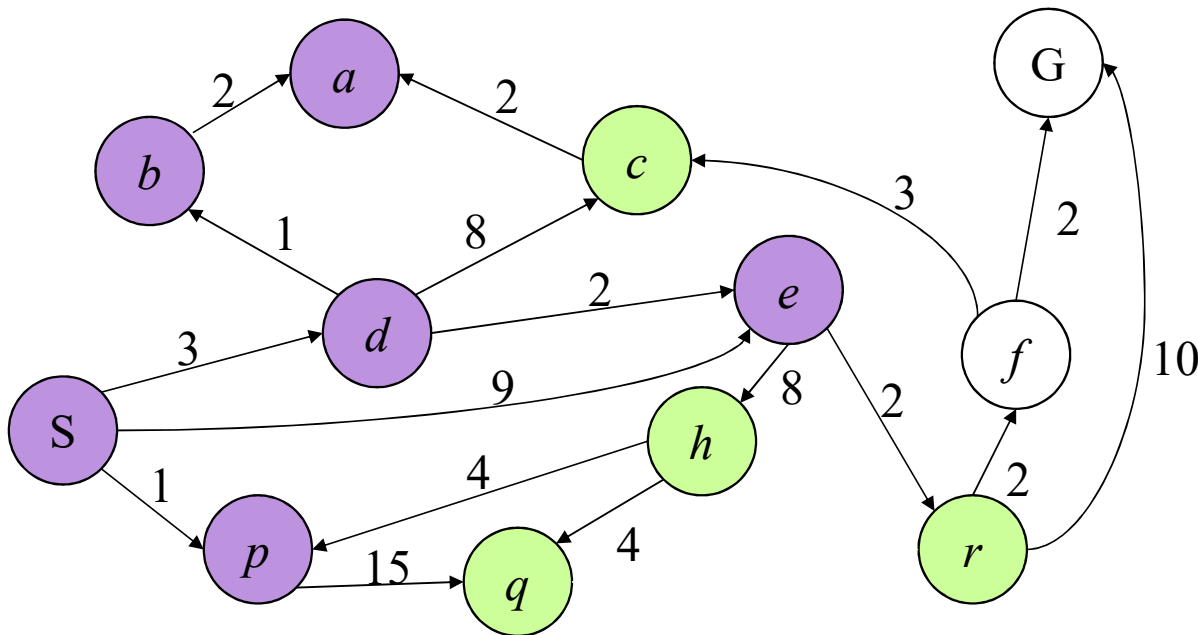
UCS (cont...)



x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	∞
h	13
p	1
q	16
r	7
G	∞



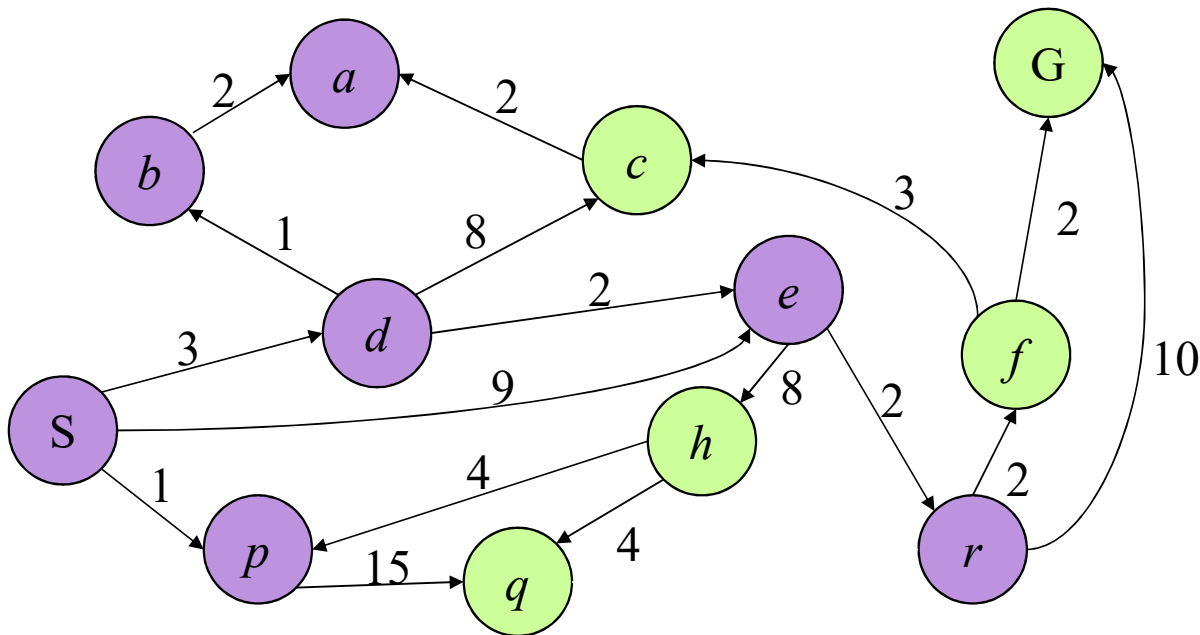
UCS (cont...)



x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	∞
h	13
p	1
q	16
r	7
G	∞



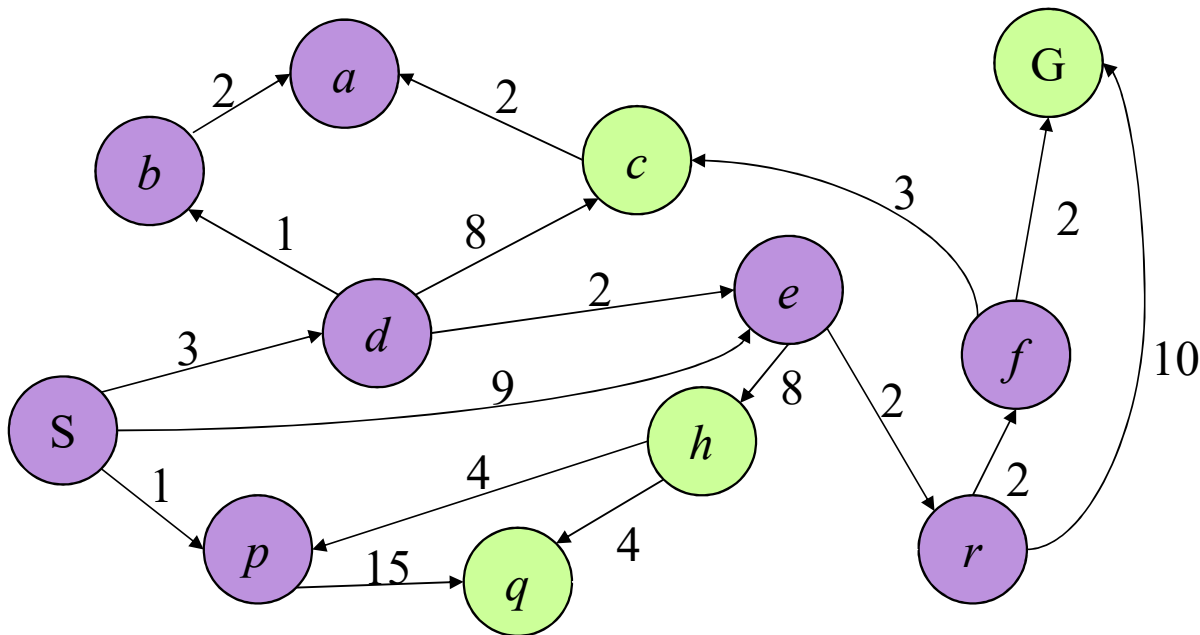
UCS (cont...)



x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	17



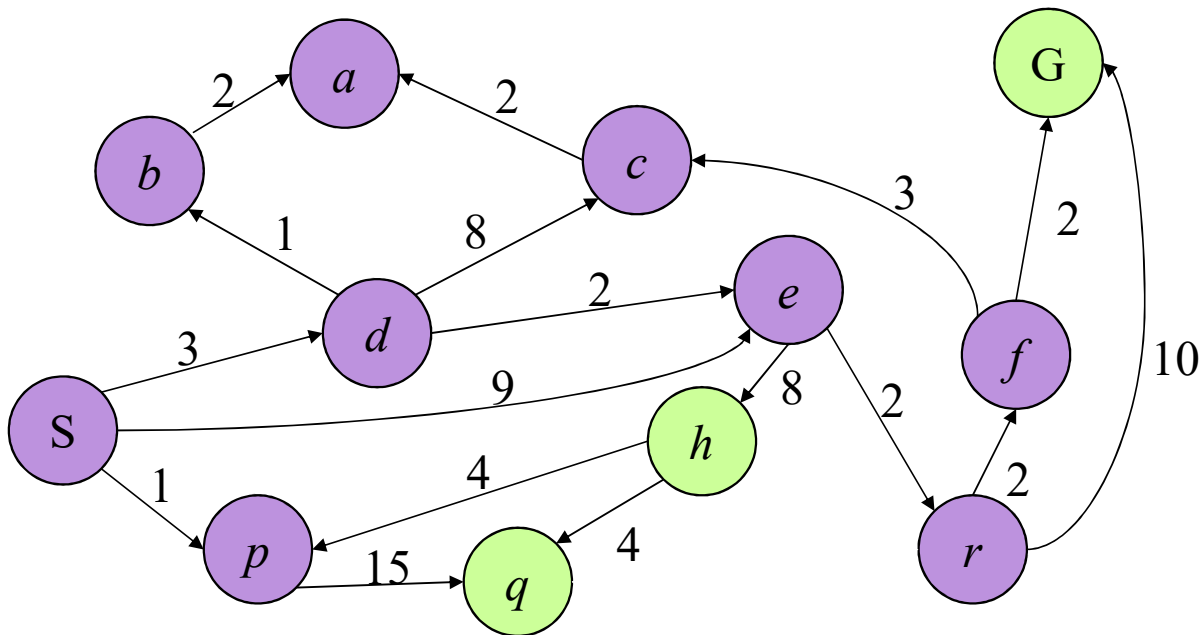
UCS (cont...)



x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	11



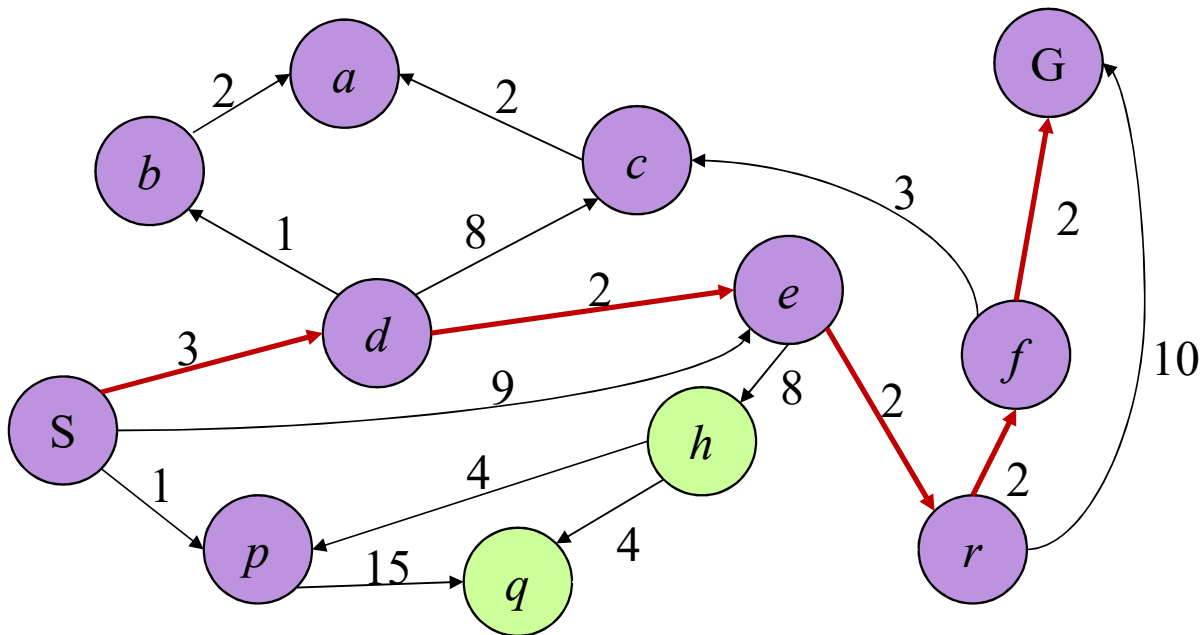
UCS (cont...)



x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	11



UCS (cont...)



x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	11

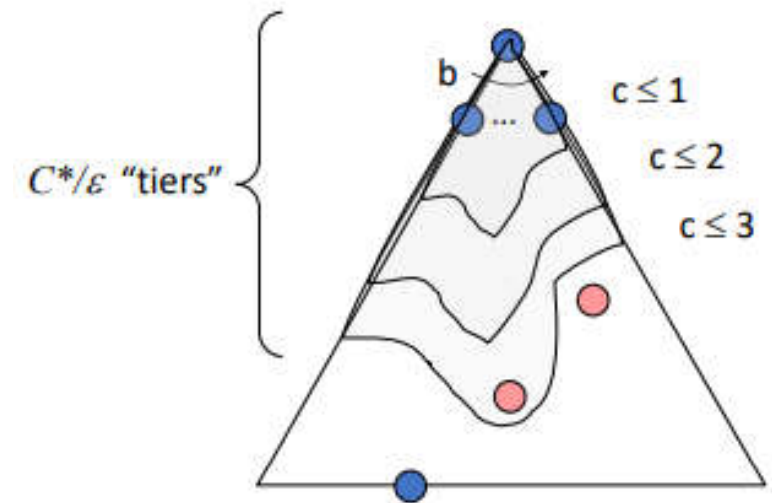


UCS properties

- What nodes does UCS expand?
 - Processes all nodes with cost less than cheapest solution!
 - If that solution costs C^* and arcs cost at least ε , then the “effective depth” is

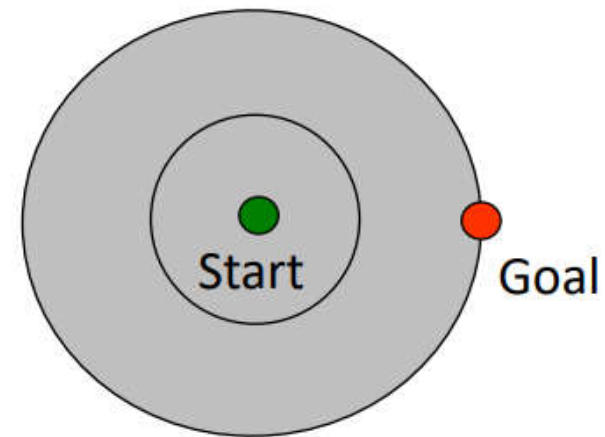
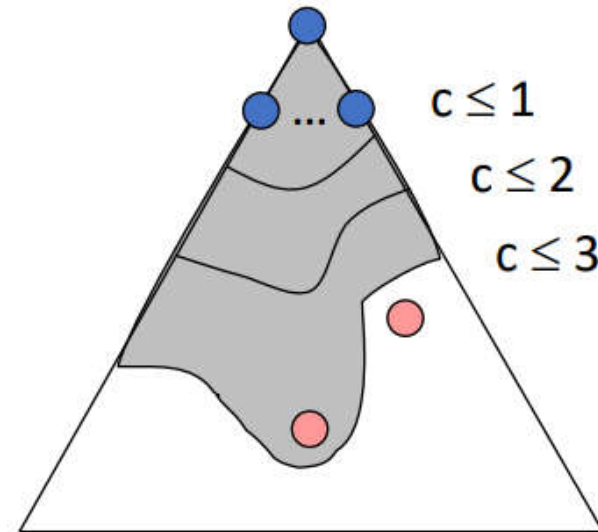
roughly C^* / ε

- Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)
- How much space does the frontier take?
 - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
 - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
 - Yes! (Proof next lecture via A^*)



Uniform cost issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
 - Explores options in every “direction”
 - No information about goal location
- We’ll fix that soon!



DFS/BFS/UCS? (Deep/light blue → high/low cost)





Topics Covered

- *Artificial Intelligence: A Modern Approach (4th Edition) by Stuart Russell and Peter Norvig. (Chapter -03)*

Any Question?